Prozessoptimierung unter Concept Drift: Adaptive Simulationsmodelle im industriellen Umfeld

Dem Fachbereich Maschinenbau und Verfahrenstechnik

der Rheinland-Pfälzischen Technischen Universität Kaiserslautern Landau

zur Erlangung des akademischen Grades

Doktor-Ingenieur (Dr.-Ing.)

vorgelegte

Dissertation

von

Herrn

M.sc. Pascal Marijan

aus Trier

2026

Betreuer der Dissertation: Prof. Dr.-Ing. Martin Ruskowski

Erklärung

Hiermit versichere ich, dass ich die vorliegende Arbeit selbständig verfasst und ohne unerlaubte, fremde Hilfe angefertigt habe. Alle Ausführungen, die aus anderen Schriften wörtlich oder sinngemäß übernommen wurden, sind kenntlich gemacht. Alle verwendeten Quellen sind im Literaturverzeichnis zitiert.

Mannheim, den 3. Oktober 2026

Pascal Marijan

Vorwort / Danksagung

Hier kommt das Vorwort oder die Danksagung hin

Weiterführung von Vorwort / Danksagung oder ...

Widmung

Inhaltsverzeichnis

Vorwort / Danksagung **III**

Kurzfassung **VII**

Abkürzungsverzeichnis **IX**

Symbolverzeichnis **X**

1 Einleitung **1**
1.1 Motivation . . . 1
1.2 Zielsetzung . . . 2

2 Grundlagen **4**
2.1 Vertikalrollenmühlen . . . 4
2.1.1 Funktionsweise . . . 4
2.1.2 Stellgrößen und Betriebspunkte . . . 6
2.1.3 GPpro . . . 8
2.2 Prozessmodellierung . . . 9
2.2.1 Systemidentifikation . . . 9
2.2.2 Modellarchitekturen . . . 11
2.2.3 Datenselektion . . . 14
2.2.4 Modellvergleich . . . 16
2.3 Concept Drift . . . 21
2.3.1 Terminologie . . . 21
2.3.2 Drift-Erkennung . . . 24
2.3.3 Drift-Adaption . . . 28

3 Generalisierung **31**
3.1 Synthetischer Datensatz . . . 32
3.1.1 Concept Drift . . . 33
3.1.2 Setpoints . . . 35
3.1.3 Signal-Inputs . . . 37
3.1.4 Rauschen . . . 38
3.1.5 Systemantwort . . . 39

3.1.6 Modellierung . . . 42
3.1.7 Drift-Erkennung . . . 47
3.1.8 Drift-Adaption . . . 51
3.2 Der Tennessee-Eastman-Process . . . 58
3.2.1 Concept Drift . . . 62
3.2.2 Generierung des Datenstroms . . . 64
3.2.3 Modellierung . . . 68
3.2.4 Drift-Erkennung . . . 74
3.2.5 Drift-Adaption . . . 78

4 Anwendung **83**
4.1 Concept Drift . . . 84
4.2 Datenerhebung . . . 85
4.3 Modellierung . . . 88
4.4 Kontextanalyse . . . 89
4.4.1 Nachweis . . . 89
4.4.2 Ursachen . . . 92
4.4.3 Beschaffenheit . . . 95
4.5 Drift-Erkennung . . . 95
4.6 Drift-Adaption . . . 95

5 Kapitel 5 **96**

Anhang **97**
A Tennessee-Eastman-Prozess . . . 97
B Vertikalrollenmühle . . . 99

Kurzfassung

Die Zementindustrie ist gegenwärtig für rund 7 % der globalen CO_2-Emissionen verantwortlich, wobei ein erheblicher Teil des elektrischen Energiebedarfs auf die Vermahlung entfällt. Vor diesem Hintergrund bietet die Betriebspunktoptimierung von Mahlanlagen ein signifikantes Potenzial zur Reduktion der spezifischen Energie, also der benötigten Energie pro Tonne produzierten Materials. Die Grundlage solcher Optimierungen bilden Simulationsmodelle, deren Performance jedoch im zeitlichen Verlauf aufgrund von Einflüssen wie Materialvariationen, Verschleiß und Wartungen beeinträchtigt wird. Im Rahmen der Untersuchung kann dieses Phänomen, aufgrund der Art der zugrunde liegenden Verschiebung der Datengrundlage als Real Concept Drift klassifiziert werden.

Durch die künstliche Simulation von Concept Drift wird das identifizierte Problem anhand des Wiener Modells generalisiert und zusätzlich auf den Benchmark-Datensatz des Tennessee Eastman Process übertragen, der für diesen Zweck modifiziert wird.

Kern der Arbeit ist die Untersuchung von Adaption der Modelle an veränderliche Bedingungen im industriellen Umfeld unter Verwendung neuronaler Netze, um die langfristige Simulationsgüte sicherzustellen. Dabei wird das Stability-Plasticity-Dilemma adressiert, also die Herausforderung des Modells, gleichzeitig neues Wissen aufzunehmen und vorhandenes Wissen zu erhalten.

Die Ergebnisse zeigen eine signifikante Verbesserung der Modellperformance durch die Adaption an veränderte Prozessbedingungen. Die vorliegende Arbeit liefert damit einen Beitrag zur Schaffung robuster maschineller Lernmodelle für die industrielle Praxis und zeigt messbare Vorteile für die Energieeffizienz vergleichbarer Prozesse auf.

Abkürzungsverzeichnis

ADWIN	Adaptive Windowing
CCR	Leitstand (Central Control Room)
CD	Concept Drift
CNN	Convolutional Neural Network
DDM	Drift-Erkennungsmethode (Drift Detection Method)
DOE	statistische Versuchsplanung (Design of Experiments)
EDDM	Early Drift Detection Method
EKF	erweiterter Kalman-Filter (Extended Kalman Filter)
GMM	Gaussian Mixture Model
IPC	Industrie-PC (Industrial PC)
KF	Kalman-Filter
KSWIN	Kolmogorov-Smirnov Windowing
LLM	Large Language Model
LR	Lineare Regression
LSTM	Long Short-Term Memory
LTI	lineares zeitinvariantes System (Linear Time-Invariant)
MAE	mittlerer absoluter Fehler (Mean Absolute Error)
MLP	Multi-Layer Perceptron
MSE	mittlerer quadratischer Fehler (Mean Squared Error)
NN	neuronales Netz
RMSE	Root Mean Squared Error (Wurzel des mittleren quadratischen Fehlers)
SPS	speicherprogrammierbare Steuerung
TEP	Tennessee Eastman Process
VRM	Vertikalrollenmühle

Symbolverzeichnis

Lateinische Buchstaben

$c[k]$	Driftsignal
$\bar{d}$	mittlerer Detektionsverzug
$\mathcal{D}$	Datensatz bzw. Datenfenster
e	Simulationsfehler (Residuum)
f	Dynamikteil: Zustandsübergang bzw. LTI-Impulsantwort
g	Ausgangsabbildung des Modells
h	Beobachtungsmodell des KF/EKF
J	Zielfunktion der Hyperparameter-Optimierung
K	Zeithorizont des Datenstroms
K_p	statische Verstärkung des PT_1-Gliedes
m	harmonisches Mittel der Fensterlängen
$p(\cdot)$	Wahrscheinlichkeitsdichte
T	Zeitkonstante des PT_1-Gliedes
u	Systemeingang
v	Zwischensignal des Wiener-Modells
w	Setpoint-Zeitreihe
W	ADWIN-Datenfenster
x	Systemzustand
y	Systemausgang

Griechische Buchstaben

α_i	Gewichtungskoeffizienten des Wiener-Modells
γ_τ	Verschiebungsanteil der Segmentgrenzen

$\delta,\ \delta'$	globales bzw. angepasstes Signifikanzniveau
$\eta_u,\ \eta_y$	injiziertes Rauschen am Ein- bzw. Ausgang
$\zeta_{\mathrm{p}},\ \zeta_{\mathrm{m}}$	Prozess- bzw. Messrauschen des KF/EKF
θ	Modellparameter
$\kappa,\ \kappa_w$	Driftkoeffizient am Ausgang bzw. Setpoint
λ	Schwellenwert
$\hat{\mu}_W$	empirischer Fenstermittelwert (ADWIN)
ν_j	normiertes Gewicht der Rauschkomponente
ρ	Recall der Sudden Drifts
σ	Standardabweichung (Streuung)
τ_i	innere Segmentgrenzen
φ	Regressionsvektor des KF/EKF
ω_*	Gewichte der Zielfunktion

Indizes und Zählgrößen

n_*	Dimension, Ordnung oder Länge
N_*	Anzahl
weitere	k (Zeitschritt), i, j (Laufindizes), $_d$ (Detektion), $_w$ (Setpoint), $_{\mathrm{p}}/_{\mathrm{m}}$ (Prozess/Messung), $_s$ (Abtastung), $_{\mathrm{a}}$ (Adaption), $_{\mathrm{min}}/_{\mathrm{max}}$ (Extrema)

Operatoren, Akzente und Superskripte

$\widehat{(\cdot)}$	Schätzwert bzw. identifiziertes Modell
$\bar{(\cdot)}$	zeitlicher Mittelwert
$\tilde{(\cdot)}$	drift-behaftete Größe
$(\cdot)^o$	rauschfreie Größe
$(\cdot)^{\mathrm{f}}$	gefilterte (geglättete) Größe
$(\cdot)^T$	Transponierte
$\lvert\cdot\rvert$	Betrag

1 Einleitung

TODO Kurze Einleitung dazu, was in dem Kapitel beschrieben wird.

1.1 Motivation

Im Kontext des Klimawandels, insbesondere im Industriesektor, erfährt der CO_2 Fußabdruck eine zunehmend zentralere Bedeutung. In diesem Kontext ist die Zementindustrie derzeit für rund 7 % der globalen CO_2 Emissionen verantwortlich [In18]. Angesichts der Tatsache, dass die globale Nachfrage nach Zement weiterhin wächst, besteht ein großes Interesse an einer Reduktion der Emissionen in diesem Sektor. Wie in Abbildung 1.1 am Beispiel von Deutschland dargestellt, können die Emissionen in der Zementindustrie in prozessbedingt und energiebedingt unterteilt werden. Ein signifikanter Anteil der prozessbedingten Emissionen ist auf die Freisetzung des CO_2 zurückzuführen, welches im Hauptbestandteil Kalkstein gebunden ist und während des Kalzinierungsprozesses freigesetzt wird. Eine Optimierung kann hier durch die Substitution des Kalksteins und des Klinkeranteils, beispielsweise durch das Beimischen von Recyclingmaterial, erzielt werden. Neben der Menge an CO_2, die durch den Kalzinierungsprozess für die Klinkerproduktion und durch die Wärmeerzeugung im Drehrohrofen emittiert wird, wird eine erhebliche Menge an elektrischer Energie benötigt, die dem energiebedingten Anteil zugeordnet wird [SB21]. Dabei werden etwa 60 % der elektrischen Energie für die Rohmaterialaufbereitung und die Klinkervermahlung benötigt [Af15]. So übersteigt die Leistung einer großen Vertikalrollenmühle 10 Megawatt. Dies verdeutlicht das große Optimierungspotenzial innerhalb des Mahlprozesses. Ein vielversprechendes Optimierungspotenzial bietet die Betriebspunktoptimierung. Dabei ist das Ziel die Parameter des Mahlbetriebs so zu wählen, dass die Vermahlung mit reduziertem Energieverbrauch erfolgt, ohne jedoch den Durchsatz oder die Qualität des Produkts zu beeinflussen. Konkret geht es dabei um die Verbesserung der spezifischen Energie, also der Energie pro Tonne Material. Diese Art der Optimierung gelingt mithilfe eines Modells, welches die Grundlage der Arbeit bildet und dessen Entwicklung einen essenziellen Teil der Untersuchung darstellt. Zur Bewertung des Modells kann der Modellfehler herangezogen werden, der die Differenz von Modellsimulation und Realwert beschreibt. Bei der Betrachtung des Modellfehlers der spezifischen Energie wie dargestellt in Abbildung 1.2 fällt auf, dass die Performance des Modells im zeitlichen Verlauf variiert. Dieses Phänomen lässt sich durch Concept Drift (CD) erklären und ist Motivation der Arbeit.

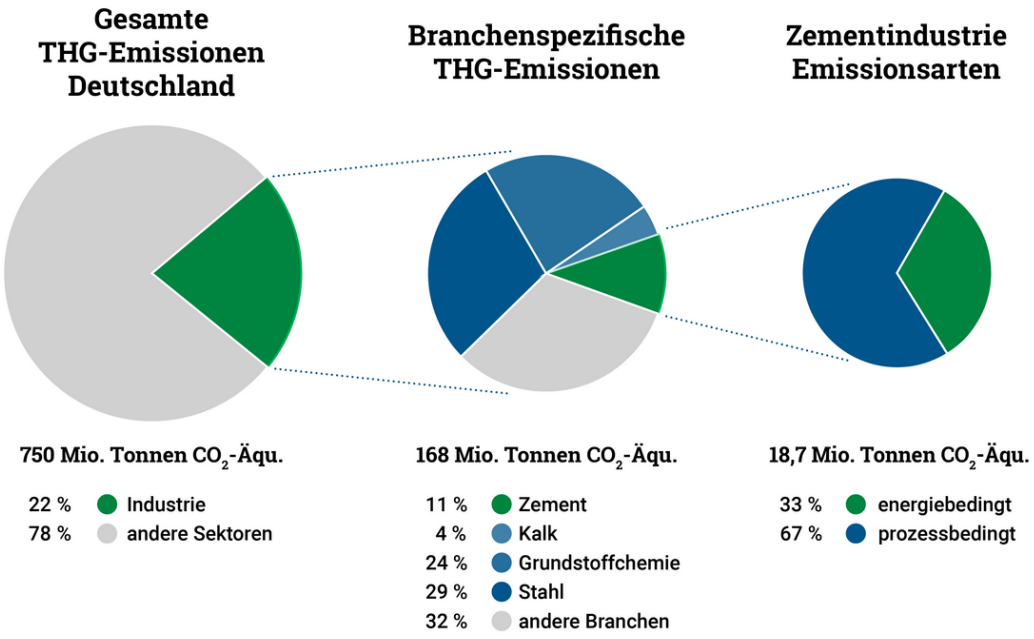


Abbildung 1.1: Treibhausgase (THG) der deutschen Industrie 2022. Grafik entnommen aus [Ko26]

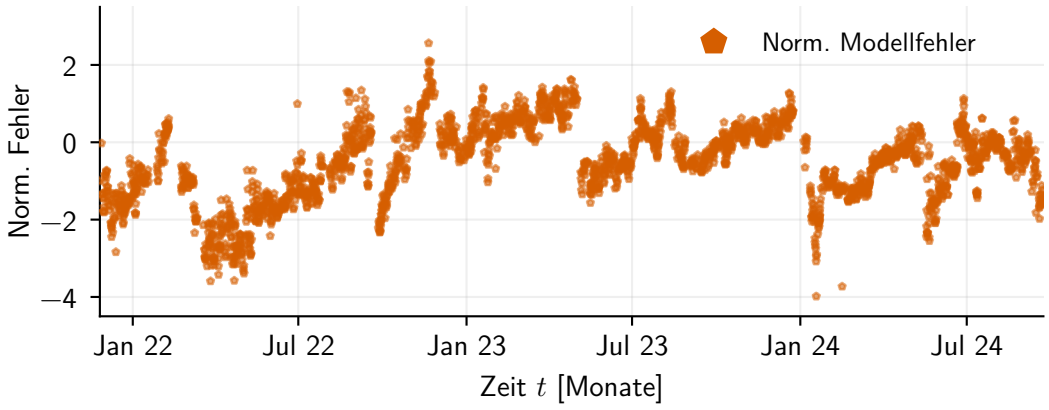


Abbildung 1.2: Normierter Modellfehler der spezifischen Energie im Produktionsbetrieb

1.2 Zielsetzung

Motiviert durch die Problemstellung aus soll die Dissertation durch die folgenden Forschungsleitfragen strukturiert werden:

1. Wie kann die Problemstellung des CD an einer industriellen Mahlanlage unter Wahrung ihrer Herausforderungen auf generalisierte Prozessdaten übertragen werden, um Erkennung und Adaption an einem bekannten Drift zu erproben?
2. Welche Form von CD liegt an einer realen Mahlanlage vor, und wie lässt er sich ohne Ground Truth aus Betriebs- und Wartungsdaten erschließen?
3. Welche messbaren Vorteile lassen sich durch die Adaption des Modells gegen CD im realen

Betrieb der Mahlanlage nachweisen?

2 Grundlagen

Die modellbasierte Betriebspunktoptimierung einer Vertikalrollenmühle erfordert ein grundlegendes Verständnis des zu optimierenden Prozesses, der eingesetzten Modellierungsverfahren sowie der Herausforderungen, die sich aus dem zeitvarianten Charakter des Prozesses ergeben. Dieses Kapitel legt die dafür notwendigen theoretischen Grundlagen und stellt den Stand der Technik dar. In Abschnitt 2.1 wird zunächst die Vertikalrollenmühle in ihrer technischen und systemischen Beschreibung eingeführt. Darauf aufbauend behandelt Abschnitt 2.2 die datengetriebene Prozessmodellierung, von der Systemidentifikation über relevante Modellarchitekturen bis hin zur Modellauswahl für die Betriebspunktoptimierung. Abschließend werden in Abschnitt 2.3 die Grundlagen zu CD erarbeitet, der als zentrales Phänomen die langfristige Modellgüte im realen Betrieb beeinflusst.

2.1 Vertikalrollenmühlen

Zunächst werden Aufbau und Funktionsweise der Vertikalrollenmühle erläutert sowie Stellgrößen und Betriebspunkte formal beschrieben. Ergänzend wird GPlink/GPpro als Erweiterung der Mühle zu einem cyberphysischen System dargestellt.

2.1.1 Funktionsweise

Vereinfacht gesprochen setzt sich Zement aus Kalkstein und Ton zusammen. Zur Zementherstellung erfolgt nach dem Abtragen der Gesteine zunächst eine Zerkleinerung mit Brecheranlagen, bevor das Gemisch zu Rohmehl vermahlen wird. Beim Brennen des Rohmehls im Drehrohrofen entsteht so genannter Zementklinker, der erneut vermahlen wird. Bei dem dabei entstehenden Produkt handelt es sich um das Endprodukt, Zement. In Abbildung 2.1 sind die beschriebenen Prozessschritte der Zementherstellung schematisch dargestellt. Die Mühlen sind dabei farblich in Gelb zu erkennen. Neben der Rohmehlmühle (a) und der Zementmühle (c) ist auch eine Kohlemühle (b) dargestellt, die oftmals bei kohlebefeuertem Drehrohrofen eingesetzt wird. Im Laufe der Zeit haben sich zur Vermahlung mehrere Verfahren etabliert. Neben der Kugelmühle wird hauptsächlich die Vertikalrollenmühle (VRM) eingesetzt, die aufgrund ihres hohen Wirkungsgrads als Stand der Technik gelten [Sc01]. Dennoch gibt es auch bei VRM ein erhebliches Potenzial zur Energieoptimierung. So kann beispielsweise eine modulare Bauweise

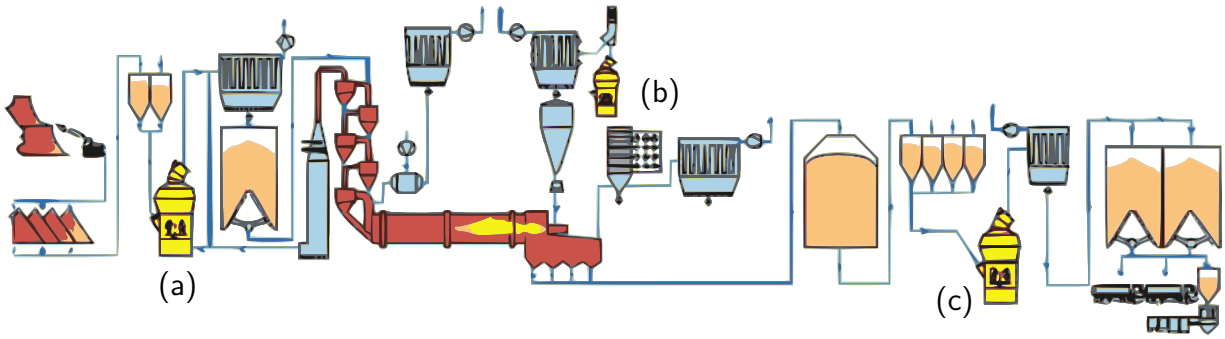


Abbildung 2.1: Schematische Abbildung der konsekutiven Prozessschritte in der Zementherstellung, Abbildung von Gebr. Pfeiffer SE

der Mühle den Prozess durch eine Reduzierung der Stillstandzeiten optimieren [Wo17]. Neben der Zementindustrie kommen VRM auch in der Kohleindustrie zum Einsatz. Zu den gängigen Mahlgütern zählen Kalkstein, Kohle, Ton, Gips und Schlacke. Das Hauptziel des Mahlvorgangs besteht darin, das Eingangsmaterial auf ein gewünschtes Maß zu zerkleinern. Während die Größe des Produkts erst nach dem Prozess gemessen werden kann, werden mehrere Sensoren eingesetzt, um den Zustand des Systems zu überwachen, sodass die resultierende Partikelgröße geschätzt werden kann. Der prinzipielle Aufbau einer VRM ist in Abbildung 2.2 schematisch dargestellt. Grundsätzlich besteht eine VRM aus einem rotierenden Mahlteller, auf dem die Walzen durch ihr Eigengewicht nach unten gedrückt werden. Häufig wird die Kraft in Richtung Mahlteller durch ein hydraulisches System unterstützt. Die Verschleißteile, einschließlich der Walzen und der Oberfläche des Mahltellers, bestehen in der Regel aus austauschbaren Teilen aus Hartgusslegierungen wie NiHard, die für ihre extreme Härte bekannt sind. Das Aufgabematerial wird von oben durch die Schurre über einen Trichter in die Mitte des Mahltellers eingeführt. Durch die Zentrifugalkräfte wird das Material auf dem Mahlteller zum Rand bewegt, wo es von den Walzen erfasst und zerkleinert wird. So entsteht ein Gemisch aus grobem und feinem Mahlgut. Dieses Gemisch bildet ein so genanntes „Mahlbett“ auf dem Mahlteller, das einen direkten Kontakt zwischen den Walzen und dem Tisch verhindert. Ein durch einen Ventilator erzeugter Luftstrom tritt oberhalb des Antriebs am äußeren Rand des Mahltellers ein und transportiert das feine Material nach oben durch die Mahlkammer. Der Sichter oberhalb der Schurre trennt dann das Grobgut vom Feingut. Das feine Material verlässt die Mühle, während das grobe Material über den Trichter zurück auf den Mahlteller fällt. Material, das über den Rand des Mahltellers fällt, wird über ein externes Zirkulationssystem wieder in die Mühle zurückgeführt.

Bei der in dieser Arbeit betrachteten Mühle handelt es sich um eine VRM vom Typ MVR 2500 C-4 der Firma Gebr. Pfeiffer SE. Diese MVR wird für die Zerkleinerung von Hochofenschlacke verwendet, einem Nebenprodukt der Stahlindustrie, mit dem ein Anteil des Klinkers im Zement substituiert werden kann. Mit vier gleich großen Mahlwalzen und einem Mahltellerdurchmesser von $2{,}5\,\mathrm{m}$ ist die MVR 2500 C-4 für Durchsatzleistungen von bis zu 30 t/h ausgelegt und wird von einem 750-kW-Elektromotor angetrieben.

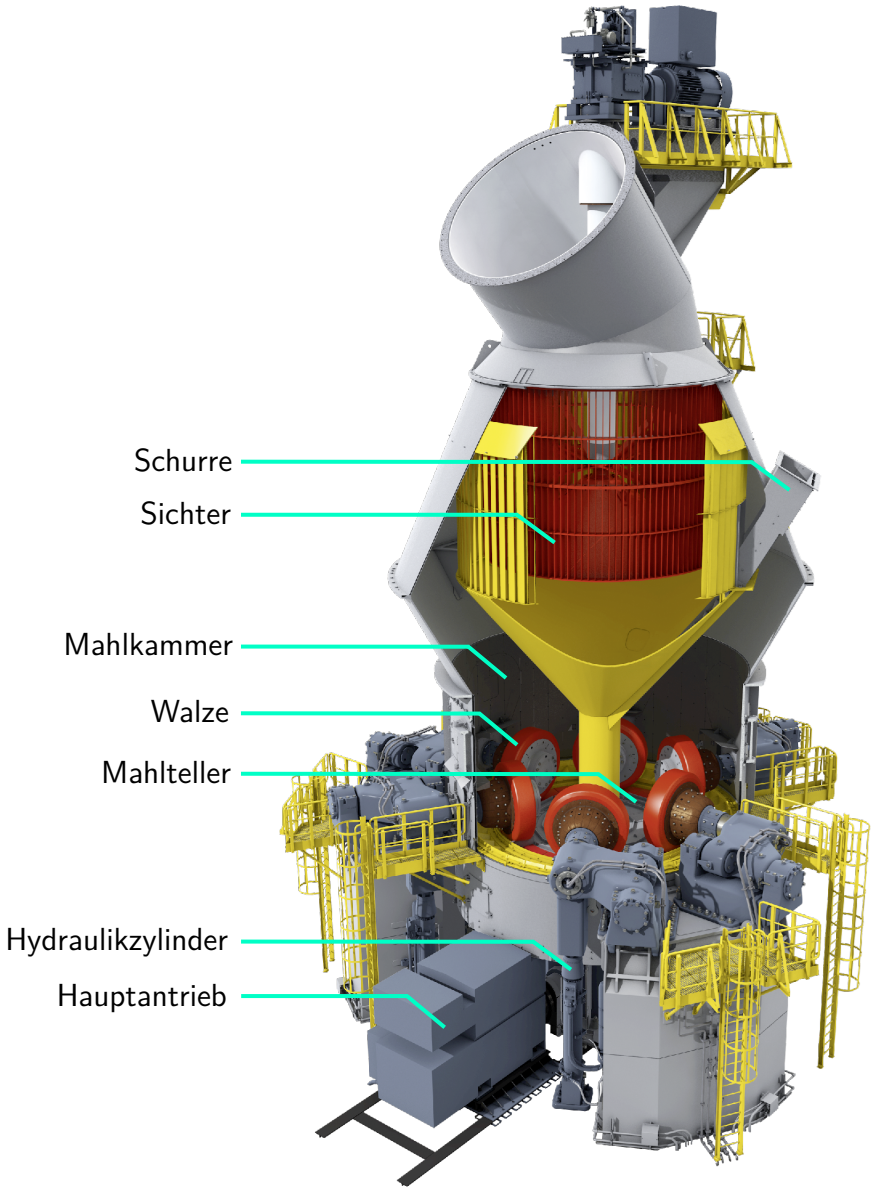


Abbildung 2.2: Schematische Abbildung einer VRM vom Typ MVR der Firma Gebr. Pfeiffer SE

2.1.2 Stellgrößen und Betriebspunkte

Zum Betrieb der Mühle stehen dem Betreiber, im Folgenden als Operator bezeichnet, mehrere Stellgrößen zur Verfügung, mit denen der Prozess gezielt beeinflusst werden kann. Dabei legen die Operatoren die Sollwerte, die so genannten Setpoints, der jeweiligen Aggregate fest, die anschließend durch eine speicherprogrammierbare Steuerung (SPS) geregelt werden. Eine konkrete Kombination aller eingestellten Stellgrößen zu einem bestimmten Zeitpunkt wird als Betriebspunkt bezeichnet. Jeder Betriebspunkt beschreibt somit eine eindeutig definierte Konfiguration der Anlage, die sich durch die Werte aller relevanten Stellgrößen charakterisieren lässt. Formal lässt sich ein Betriebspunkt als Vektor

$$w = [w_1, w_2, \dots, w_n]^T \in \mathbb{R}^n \tag{2.1}$$

auffassen, wobei die einzelnen Komponenten w_i die jeweiligen Stellgrößen der Mühle repräsentieren. Zu diesen Stellgrößen zählen unter anderem:

- **Die Aufgabemenge**
  Abhängig von Mühlengröße und Produkt können die Mengen des Edukts oder der Edukte über mehrere Förderbänder variiert werden. Die Messung erfolgt mittels Bandwagen.
- **Die Mahltellergeschwindigkeit**
  Sofern der Hauptantrieb mit einem Frequenzumrichter ausgestattet ist, kann die Drehzahl des Mahltellers variiert werden. Aus wirtschaftlichen Gründen wird bei einigen Anlagen jedoch auf den Einsatz eines Frequenzumrichters verzichtet.
- **Die Sichtergeschwindigkeit**
  Über die Sichtergeschwindigkeit wird die Drehzahl des Sichterrades geregelt. Mit zunehmender Drehzahl wird der Abscheidegrad erhöht, sodass weniger grobes Material mit dem Luftstrom zu den Filtern transportiert wird.
- **Die Ventilatorgeschwindigkeit**
  Die Ventilatorgeschwindigkeit beeinflusst den Luftvolumenstrom innerhalb der Anlage. In manchen Anlagen wird der Luftvolumenstrom direkt als Stellgröße durch den Operator vorgegeben.
- **Der Spanndruck**
  Der Spanndruck bezeichnet den hydraulischen Druck, mit dem die Walzen auf den Mahlteller wirken. Aufgrund von Leckagen im Hydrauliksystem weist der Spanndruck eine charakteristische Hysterese auf, wobei das Hydraulikaggregat bei Unterschreiten definierter Grenzwerte automatisch nachregelt.
- **Die Heißgastemperatur**
  Der in der Regel mit fossilen Brennstoffen betriebene Heißgaserzeuger ist für die Einstellung der Temperatur des Luftvolumenstroms verantwortlich.
- **Die Klappenstellungen**
  Die Frischluft- und Rezirkulationsklappen dienen insbesondere der Abführung von Feuchtigkeit und Wärme aus dem System und beeinflussen damit sowohl den thermischen als auch den strömungstechnischen Zustand der Mühle.

Da jede Stellgröße innerhalb eines definierten Wertebereichs variiert werden kann, spannt die Gesamtheit aller Stellgrößen einen hochdimensionalen Stellgrößenraum $\mathcal{W} = \{w \in \mathbb{R}^n\}$ auf. Entsprechend existiert eine sehr große Anzahl möglicher Betriebspunkte, in denen die Mühle betrieben werden kann. Die Identifikation geeigneter Betriebspunkte innerhalb dieses hochdimensionalen Raums stellt eine zentrale Herausforderung für den effizienten Betrieb der Mühle dar und soll durch modellbasierte Betriebspunktoptimierung adressiert werden.

2.1.3 GPpro

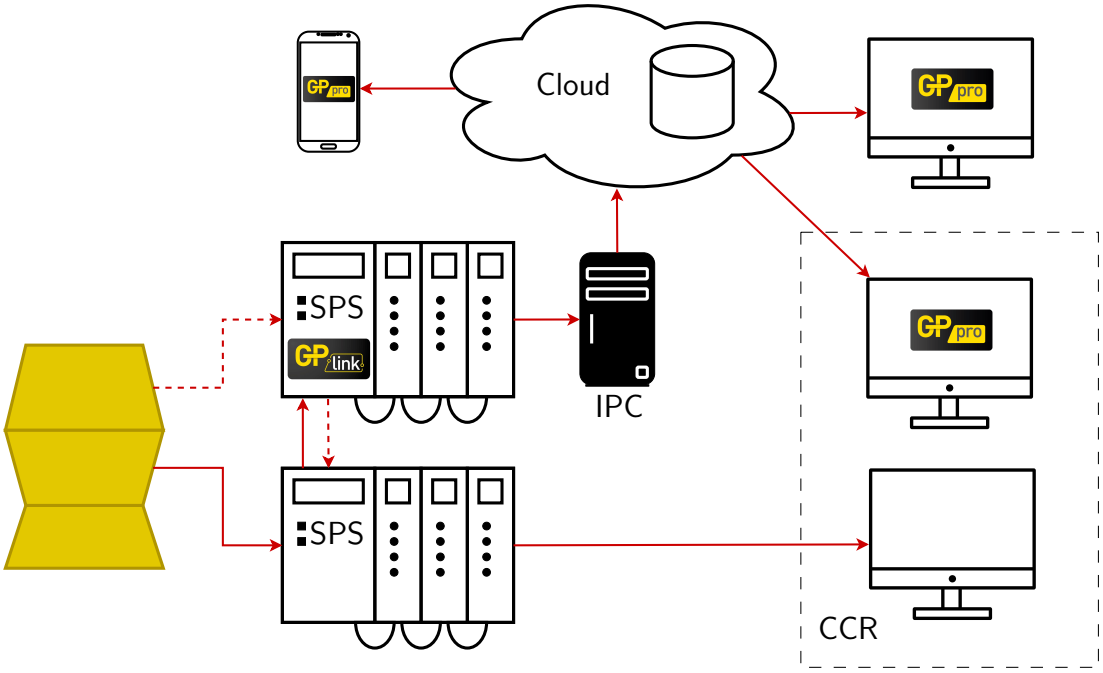


Abbildung 2.3: Schematische Abbildung der Datenverarbeitung mit GPlink/GPpro

GPlink/GPpro bezeichnet das Digitalisierungsprogramm der Firma Gebr. Pfeiffer SE, mit dem die Mahlanlagen zu cyberphysischen Systemen erweitert werden. Cyberphysische Systeme zeichnen sich dadurch aus, dass sie physikalische Prozesse eng mit rechnergestützten Verfahren zur Datenerfassung, -verarbeitung und -visualisierung koppeln und diese über Kommunikationsnetze verfügbar machen [Le08]. Die Architektur von GPlink/GPpro umfasst Hardware- und Softwarekomponenten; die verwendeten Bezeichnungen sind historisch gewachsen und daher nicht ganz trennscharf. In dieser Arbeit wird vereinfachend die Hardware-Infrastruktur als GPlink und die darauf aufbauenden Software- und Webdienste als GPpro bezeichnet. Die prinzipielle Funktionsweise des Systems ist in Abbildung 2.3 schematisch dargestellt. Die Sensordaten der Mahlanlage werden zunächst konventionell über SPS des Kunden erfasst. Ergänzend hierzu ist eine zusätzliche SPS installiert, die so genannte GPlink-SPS. Diese übernimmt zwei zentrale Aufgaben: Zum einen empfängt sie ausgewählte Prozesssignale aus der Kunden-SPS mit einer Frequenz von $1\,\mathrm{Hz}$, zum anderen stellt sie zusätzliche Messdaten aus einer erweiterten Sensorik bereit, die Bestandteil von GPlink ist, und überträgt diese an die Kunden-SPS zurück. Auf diese Weise wird die bestehende Automatisierungsstruktur erweitert, ohne sie grundlegend zu verändern. Die von der GPlink-SPS erfassten Daten werden über eine OPC-UA-Schnittstelle an einen Industrie-PC (Industrial PC) (IPC) übertragen. Der IPC wiederum fungiert als Bindeglied zwischen der lokalen Automatisierungsebene und der Cloud-Infrastruktur. Die Daten werden so an cloudbasierte Dienste weitergeleitet, wo sie gespeichert und weiterverarbeitet werden. In der Cloud sind mehrere, mittels Docker containerisierte Dienste implementiert. Dazu zählen unter anderem *Kafka* zur Datenübertragung, *InfluxDB* zur Speicherung der Zeitreihendaten, *PostgreSQL* zur Ablage verarbeiteter Daten, *Grafana* zur Visualisierung sowie *Python*-basierte

Dienste zur Datenanalyse. Die vom IPC gesendeten Prozessdaten werden über Kafka in die Cloud übertragen und in einer Influx-Datenbank gespeichert. Auf dieser Datenbasis werden mit Python Berechnungen, Analysen und Auswertungen durchgeführt, deren Ergebnisse anschließend in einer PostgreSQL-Datenbank gespeichert werden. Sowohl Roh- als auch verarbeitete Daten werden über ein webbasiertes Frontend bereitgestellt und mit Grafana visualisiert. Über die Cloud-Infrastruktur ist ein weltweiter Zugriff möglich. Die Operatoren und weitere Stakeholder erhalten dadurch ortsunabhängig Einblick in relevante Prozessgrößen, Kennzahlen und Auswertungen. Für die Operatoren im Leitstand (Central Control Room) (CCR) stehen die Prozessdaten somit auf zwei Wegen zur Verfügung: Zum einen weiterhin direkt über die Visualisierung der Kunden-SPS, zum anderen ergänzend über GPpro mit erweiterten Analyse-, Historisierungs- und Visualisierungsfunktionen. GPlink/GPpro erweitert damit die klassische Automatisierungsebene um eine digitale Service- und Analyseplattform und bildet eine Grundlage für datengetriebene Prozessüberwachung und -optimierung.

2.2 Prozessmodellierung

Aufbauend auf der Beschreibung der Mühle wird im Folgenden die Prozessmodellierung eingeführt, beginnend mit grundlegenden Aspekten der Systemidentifikation, einer kompakten Einführung in neuronale Netze und abschließend der Modellauswahl zur Betriebspunktoptimierung.

2.2.1 Systemidentifikation

Der Vermahlungsprozess lässt sich, wie viele ingenieurwissenschaftliche Prozesse, als dynamisches System beschreiben. Obwohl reale Systeme ihrem Kern nach zeitkontinuierlich sind, wird für deren Systemidentifikation auf zeitdiskrete Modelle zurückgegriffen. Der Grund dafür ist der zeitdiskrete Charakter der Sensoren, die zur Datenerfassung verwendet werden. In der Systemidentifikation können Systeme anhand verschiedener Eigenschaften klassifiziert werden [Ke11]. Dazu zählen:

- die Kausalität,
- der Prozesscharakter (deterministisch oder stochastisch),
- die Zeit(in)varianz,
- die (Nicht-)Linearität,
- die Dynamik.

Grundsätzlich kann ein System entweder durch ein Zustandsraummodell oder ein Input-Output-Modell beschrieben werden. Ausgehend von einem Datensatz $\mathcal{D} = \{(u[k],\, y[k])\}_{k=0}^{N-1}$ aus N

gemessenen Input-Output-Paaren kann das System S durch folgende Zustandsraumdarstellung beschrieben werden:

$$\begin{aligned} x[k+1] &= f(x[k], u[k]) \\ y^o[k] &= g(x[k]) \ , \end{aligned} \tag{2.2}$$

wobei $x[k] \in \mathbb{R}^{n_x}$ der Zustand des Systems, $y^o[k] \in \mathbb{R}^{n_y}$ der rauschfreie Output und $u[k] \in \mathbb{R}^{n_u}$ der Input jeweils zum Zeitschritt k ist. Weiterhin sind $f(\cdot)$ und $g(\cdot)$ die Zustands- bzw. Output-Mappings [Ke11]. Der gemessene Output $y[k]$ wird durch ein mittelwertfreies, farbiges oder weißes Rauschen $\eta[k]$ verfälscht, d. h. $y[k] = y^o[k] + \eta[k]$. Dieses Zustandsraummodell kann entweder „klassisch“ nach Linearisierung in den Frequenzbereich überführt oder in der „modernen“ Identifikation direkt im Zeitbereich betrachtet werden.
Alternativ kann statt des Zustandsraumansatzes eine Input-Output-Modellstruktur verwendet werden. Der durch ein Modell geschätzte Output sei dabei mit $\hat{y}[k]$ bezeichnet. Modelle, die sowohl die gemessenen Inputs als auch die gemessenen Outputs zur Vorhersage verwenden, werden als Prädiktionsmodelle bezeichnet:

$$\hat{y}[k] = g(y[k-1], \dots, y[k-n_a],\ u[k-1], \dots, u[k-n_b];\ \theta) \ , \tag{2.3}$$

wobei n_a bzw. n_b die Länge der jeweils berücksichtigten Historien festlegen und $g(\cdot)$ das durch $\theta \in \mathbb{R}^{n_\theta}$ parametrisierte Output-Mapping ist [FP20]. Im Gegensatz dazu werden Modelle, bei denen ausschließlich die gemessenen Inputs auf den Output gemappt werden, Simulationsmodelle genannt [FP20]. Ein solcher Fall liegt vor, wenn der Output aus einer endlichen Historie vergangener Inputs berechnet wird:

$$\hat{y}[k] = g(u[k-1], \dots, u[k-n_b];\ \theta) \tag{2.4}$$

Wird darüber hinaus die Dynamik des Systems im Modell vernachlässigt, spricht man von einem statischen Modell. Der Zustand x - und damit das Gedächtnis des Systems, über das vergangene Inputs nachwirken - wird dabei vernachlässigt, sodass der Output ausschließlich vom aktuellen Input abhängt:

$$\hat{y}[k] = g(u[k];\ \theta) \ , \tag{2.5}$$

wobei $u[k]$ den System-Input zum Zeitpunkt k darstellt. Die Abweichung $e[k] = y[k] - \hat{y}[k]$ zwischen Messung und Modelloutput beschreibt den (Simulations-)Fehler. Ist $g(\cdot)$ linear in den Parametern θ, spricht man von linearer Regression; andernfalls ist $g(\cdot)$ ein nichtlineares Mapping, das den Input mit dem Output in Beziehung setzt.

Zur Identifikation des in Abschnitt 2.1 beschriebenen Vermahlungsprozesses kann zunächst festgehalten werden, dass es sich um einen kausalen Prozess handelt, da alle relevanten Ausgangsgrößen ausschließlich von aktuellen und vergangenen Eingangsgrößen abhängen und keine zukünftigen Informationen benötigen. Obwohl die zugrunde liegenden physikalischen Gesetzmäßigkeiten prinzipiell deterministisch sind, wird das reale Anlagenverhalten durch zufällige Einflüsse wie schwankende Materialeigenschaften, Umwelteinflüsse sowie Prozess- und Messrauschen geprägt, sodass der Prozess praktisch stochastischen Charakter besitzt. Zudem ist der Prozess

zeitvariant, da sich die Systemdynamik infolge von Verschleiß, Materialwechseln, Wartungsmaßnahmen und veränderten Betriebsbedingungen kontinuierlich verändert. Weiterhin weist die Vermahlung eine ausgeprägte Nichtlinearität auf, die sich unter anderem in betriebspunktabhängigem Verhalten und Wechselwirkungen zwischen Materialfluss, Druck, Temperatur und Energieeintrag äußert. Schließlich handelt es sich um einen dynamischen Prozess, da interne Speicher- und Verzögerungseffekte, insbesondere durch das Mahlbett und thermische Trägheiten, dazu führen, dass der aktuelle Prozesszustand wesentlich von der Systemhistorie abhängt. Der Vermahlungsprozess ist folglich als kausaler, stochastischer, zeitvarianter, nichtlinearer und dynamischer Prozess zu klassifizieren.

Eine physikalisch motivierte Zustandsraumbeschreibung dieses Systems würde eine detaillierte Abbildung der zugrunde liegenden mechanischen, thermischen und materialabhängigen Wechselwirkungen erfordern. Aufgrund der hohen Prozesskomplexität sowie der Vielzahl nur schwer oder gar nicht messbarer Zustandsgrößen ist die Herleitung eines hinreichend genauen physikalischen Modells jedoch mit erheblichem Aufwand und Unsicherheiten verbunden. Bisherige Versuche, ein solches Modell zu entwickeln, führten zu keinem Erfolg. Ein datenbasierter Input-Output-Modellierungsansatz hingegen ermöglicht es, das dynamische Prozessverhalten direkt aus gemessenen Ein- und Ausgangsdaten zu identifizieren. Dieser Ansatz eignet sich insbesondere für komplexe industrielle Prozesse mit zeitvarianten stochastischen Einflüssen und erlaubt eine Modellierung des Systems ohne Kenntnis der internen Zustände. Als Modellstruktur dieses Ansatzes wurde das neuronale Netz gewählt, welches eine flexible Approximation komplexer, nichtlinearer Zusammenhänge erlaubt.

2.2.2 Modellarchitekturen

Durch die prominente Präsenz großer Sprachmodelle (engl. Large Language Model (LLM)) im öffentlichen Diskurs finden die Begriffe „Künstliche Intelligenz“, „Machine Learning“ und „neuronale Netze“ zunehmend Verwendung im allgemeinen Sprachgebrauch. Diese Begriffe bezeichnen jedoch eine Vielzahl unterschiedlicher Modellklassen, deren inhaltliche Breite durch eine Reduktion auf LLMs nicht angemessen abgebildet wird. Gleichzeitig ist im industriellen Kontext zu beobachten, dass zum Teil auch vergleichsweise einfache oder etablierte algorithmische Ansätze unter diesen Begriffen subsumiert werden, was zu einer begrifflichen Verwässerung beiträgt und die fachliche Trennschärfe weiter reduziert. Um dieser Unschärfe entgegenzuwirken, sollen im Folgenden die für diese Arbeit relevanten Modellstrukturen eingeführt und voneinander abgegrenzt werden.

Zunächst werden dazu Verfahren betrachtet, die nicht in die Kategorie der neuronales Netz (NN) fallen, aber durch ihre Einfachheit auszeichnen und häufig als Baseline dienen.
Dabei sei als erstes die Lineare Regression (LR) aufgeführt, die den Zusammenhang zwischen einer abhängigen Zielgröße und einer oder mehreren unabhängigen Inputvariablen durch eine

lineare Gleichung beschreibt. Sie ist leicht interpretierbar, bei komplexeren, nicht-linearen Zusammenhängen stößt sie jedoch an ihre Grenzen und kann hier lediglich zur Approximation dienen.
Die Lasso-Regression erweitert dieses Konzept der LR um einen Regularisierungsterm [Ti96]. Dieser bestraft große Modellgewichte und kann diese auf Null setzen, was nicht nur Overfitting entgegenwirkt, sondern auch zu einer automatischen Feature-Auswahl führt.

Ein weiteres Verfahren mit linearer Modellannahme ist der der Kalman-Filter (KF), ein rekursives Zustandsschätzverfahren auf Basis eines Zustandsraummodells gemäß (2.2) [Ka60]. Der KF ist ein rekursives Schätzverfahren, das den verborgenen internen Zustand eines linearen dynamischen Systems aus einer Reihe von verrauschten Messungen ermittelt. Der Algorithmus arbeitet dabei in zwei iterativen Schritten:

- **Prädiktionsschritt**
  Schätzung des Systemzustands für den folgenden Zeitschritt basierend auf einer Zustandsgleichung des mathematischen Systemmodells
- **Korrekturschritt**
  Abgleichen der Schätzung mit der tatsächlichen Messung über die Messgleichung

Unter der Annahme, dass das System linear ist und das Rauschen einer Normalverteilung unterliegt, liefert der Kalman-Filter so theoretisch die mathematisch optimale Zustandsschätzung.

Beim erweiterter Kalman-Filter (Extended Kalman Filter) (EKF) handelt es sich um eine Erweiterung des KF, der das Konzept in nicht-lineare Zustandsraum-Modelle überführt. Dazu werden System- und Messgleichungen im jeweiligen Arbeitspunkt linearisiert [WB+95]. Dies geschieht durch eine Taylor-Reihenentwicklung erster Ordnung. Durch die lokale Linearisierung kann die Prädiktions- und Korrekturlogik des KF durch den EKF auf komplexere, nicht-lineare Problemstellungen angewendet werden.

Während die zuvor genannten Verfahren entweder auf linearen Annahmen beruhen oder explizite mathematische Systemmodelle erfordern, bieten NN einen rein datengetriebenen Ansatz zur Modellierung komplexer, nicht-linearer Zusammenhänge. Grundsätzlich lassen sich maschinelle Lernverfahren auf verschiedene Art und Weise kategorisieren. Eine fundamentale Unterscheidung beschreibt die zwischen überwachtem Lernen (engl. Supervised Learning) und unüberwachtem Lernen (engl. Unsupervised Learning) [BN06]. Beim unüberwachten Lernen erhält das Modell ausschließlich Input-Daten (engl. *Features*), ohne dass eine zugehörige Zielgröße (engl. *Label*) vorgegeben ist. Durch das Modell können so intrinsische Strukturen in den Daten analysiert oder Muster und Korrelationen identifiziert werden.

Demgegenüber basiert das Modelltraining beim überwachten Lernen auf einem Datensatz, der aus Features und den dazugehörigen, bekannten Labels besteht. Man spricht auch von gelabelten Daten. Das Ziel ist es, eine Funktion zu erlernen, die den Fehler zwischen der Mo-

dellvorhersage und dem tatsächlichen Zielwert minimiert, um präzise Vorhersagen für neue, ungesehene Daten treffen zu können. Die vorliegende Arbeit beschränkt sich auf überwachtes Lernen, da die betrachtete Problemstellung eine Regressionsaufgabe darstellt und die betrachteten Systemgrößen sensorisch gemessen und somit als gelabelte Daten für das Modelltraining verfügbar sind.

Die grundlegendste Form eines künstlichen neuronalen Netzes ist das Multi-Layer Perceptron (MLP), welches zur Klasse der Feedforward Neural Networks gehört [Go16]. Der Informationsfluss in einem MLP ist streng gerichtet und verläuft von der Eingabe zur Ausgabe, ohne Rückkopplungen. Die Architektur gliedert sich in drei Hauptkomponenten, die jeweils aus einem oder mehreren so genannten Neuronen bestehen:

- **Input Layer** (Eingabeschicht)
  Erfasst die Input-Daten u
- **Hidden Layers** (Verborgene Schichten)
  Jedes Neuron dieser Schichten ist mit allen Neuronen der vorherigen Schicht verbunden, berechnet eine gewichtete Summe von deren Outputs und transformiert diese mit einer vorgegebenen Aktivierungsfunktion.
- **Output Layer** (Ausgabeschicht)
  Gibt die Prädiktion des Outputs $\hat{y}$ zurück. Bei Regressionsaufgaben besteht sie typischerweise aus einem einzelnen Neuron mit linearer Aktivierungsfunktion.

Sowohl die Anzahl der Neuronen in den Layern als auch die Wahl der Aktivierungsfunktionen sind variabel und können als Hyperparameter des Modells spezifiziert werden. Das Training erfolgt durch den Backpropagation-Algorithmus [RHW86]. Dabei wird nach einer Vorhersage über eine Verlustfunktion der Unterschied zwischen dem tatsächlichen Output y und dem prädizierten Output $\hat{y}$ berechnet. Anschließend wird der Fehler rückwärts durch das Netz propagiert. Mithilfe von gradientenbasierten Optimierungsverfahren (engl. gradient descent) werden die Gewichte iterativ angepasst, um so den Gesamtfehler zu minimieren [Go16]. Klassischerweise ist das MLP ein statisches Modell. Um es für die Regression von Zeitreihendaten nutzbar zu machen und Dynamiken zu erfassen, kann ein Sliding-Window-Ansatz verwendet werden [BBL12]. Dabei wird ein Datenfenster festgelegter Breite aus vergangenen Zeitschritten zu einem Vektor transformiert und dem MLP als statische Eingabe übergeben. So kann das Modell historische Abhängigkeiten in seine Vorhersage einbeziehen.

Dem MLP gegenüber stehen neuronale Architekturen, die zeitliche Abhängigkeiten nicht ausschließlich über die Eingangsrepräsentation, sondern auch über ihre interne Struktur erfassen. Convolutional Neural Network (CNN) wurden ursprünglich für die Bilderkennung entwickelt [Le02]. Sie nutzen Faltungsschichten (engl. Convolutional Layers), in denen Filter über die Eingabedaten gleiten, um lokale Features zu extrahieren, wobei die Gewichte geteilt werden. Obwohl ursprünglich für 2-dimensionale Bilddaten konzipiert, lassen sich CNNs auf Zeitreihen-

daten anwenden. Dabei werden 1D-CNNs eingesetzt, bei denen ein Filter entlang der Zeitachse gleitet [Ki21]. Dadurch erkennt und extrahiert das Modell automatisch lokale zeitliche Muster und Trends innerhalb der Zeitreihe.

Für die Modellierung komplexer, langfristiger zeitlicher Abhängigkeiten finden typischerweise rekurrente neuronale Netze Anwendung. Eine besonders leistungsfähige Form ist das Long Short-Term Memory (LSTM) [HS97]. Im Gegensatz zu Feedforward-Netzen besitzen LSTMs Rückkopplungsschleifen, die einen internen Zustand über die Zeit aufrechterhalten. Durch eine Zellstruktur von so genannten Input-, Output- und Forget-Gates steuert das LSTM gezielt, welche Informationen behalten und welche vergessen werden. Dadurch sind LSTMs prädestiniert für Zeitreihendaten, da sie die Dynamik der Zeitreihen nativ abbilden.

2.2.3 Datenselektion

Bei der Modellierung realer industrieller Prozesse spielen die erfassten experimentellen Daten eine essenzielle Rolle, da sie in datengetriebenen Ansätzen die primäre Informationsquelle darstellen. Gleichzeitig ist die Datenerhebung in Produktionsumgebungen typischerweise durch technische Randbedingungen erschwert, sodass eines der zentralen Ziele darin besteht, die Datengrundlage so zu gestalten, dass der Informationsgehalt maximiert wird. Für den vorliegenden Anwendungsfall der Mühle ist zudem zu berücksichtigen, dass der eigentliche Materialzerkleinerungsprozess nicht direkt messbar ist. Stattdessen müssen indirekte Indikatoren herangezogen werden, die das Prozessverhalten über messbare Betriebsparameter beschreiben.

Die in dieser Arbeit betrachtete Zielgröße ist der spezifische Energiebedarf, also der Gesamtenergiebedarf pro Tonne produzierten Produkts, der hier auf die drei größten Hauptaggregate: Hauptantrieb, Ventilator und Sichter beschränkt ist. Um aus der Vielzahl verfügbarer Sensorsignale diejenigen Größen zu bestimmen, die einen signifikanten Einfluss auf diese Zielgröße besitzen, stützt sich die Sensorauswahl auf eine Kombination aus ingenieurwissenschaftlichem Fachwissen sowie datengetriebenen Analyseverfahren wie Principal Component Analysis und Mutual-Information-Analyse. Die resultierende Menge relevanter Stell- und Messgrößen entspricht den in Abschnitt 2.1.2 eingeführten Parametern. Dabei werden die Stellgrößen als exogene Setpoints durch die Operatoren vorgegeben, während die resultierenden Prozesssignale sensorisch erfasst und als Messgrößen in das Modell eingehen. Mit der Identifikation der für die Zielgröße relevanten Stellgrößen stellt sich die Frage nach der geeigneten Modellstruktur zur Beschreibung des Prozesses.

Wie bereits einleitend in Abschnitt 1.1 beschrieben, liegt das Ziel der Modellierung des Mahlprozesses darin, den Prozess durch Betriebspunktoptimierung zu verbessern. Dazu sollen in einem gegebenen Betriebspunkt die partiellen Ableitungen der Zielgröße nach den Eingangsgrößen bestimmt werden, um diejenigen Stellgrößen zu identifizieren, die lokal den größten Einfluss auf die Zielgröße ausüben. Auf Basis dieser Information kann anschließend eine Anpassung der

entsprechenden Stellgrößen im realen Prozess erfolgen. Vor diesem Hintergrund stellt sich die Frage, welche Modellarchitektur mit Blick auf den Informationsgehalt der verfügbaren Daten für diese Aufgabe am geeignetsten ist. Einerseits liegen Zeitreihendaten vor, deren sequenzieller Charakter prinzipiell dynamische Information enthält. Andererseits ist die Betriebspunktoptimierung in erster Linie an eingeschwungenen, quasi-stationären Zuständen interessiert, sodass eine Reduktion auf stationäre Betriebspunkte denkbar ist. Die Modellkomplexität soll daher empirisch überprüft werden.

In Abschnitt 2.2.1 wurde der Mahlprozess als dynamischer Prozess identifiziert. Außerdem wurde das NN als datenbasierter Input-Output-Modellierungsansatz des Prozesses gewählt. Die Entscheidung für ein NN als Modellstruktur lässt jedoch offen, ob zur Abbildung des Prozesses ein dynamisches Modell gemäß (2.4) oder ein statisches Modell gemäß (2.5) verwendet werden soll. Während statische Input-Output-Modelle den Systemausgang ausschließlich als Funktion der aktuellen Eingangsgrößen beschreiben, berücksichtigen dynamische Input-Output-Modelle zusätzlich die zeitliche Abhängigkeit vergangener Ein- und Ausgangswerte. Damit weisen statische Modelle aufgrund der fehlenden zeitlichen Rekursion grundsätzlich eine geringere strukturelle Komplexität auf als dynamische Modelle [Ne20]. Vor diesem Hintergrund stellt sich die zentrale Frage, welche Modellkomplexität für den vorliegenden Prozess unter den gegebenen Informationsbedingungen adäquat ist. Die verfügbaren Messdaten mit einer Abtastrate von $1\,\mathrm{Hz}$ bilden sowohl eingeschwungene, quasi-stationäre Betriebszustände als auch Übergänge zwischen diesen ab, die als transient bezeichnet werden. Die Daten im quasi-stationären Zustand schwanken, bedingt durch Messrauschen und stochastische Prozesseinflüsse, um einen Mittelwert. In diesem Fall liegt ein klassisches Regressionsproblem vor, bei dem die zeitliche Reihenfolge der Datenpunkte vernachlässigt wird. Demgegenüber besteht bei transienten Zuständen die Erwartung, dass sich zusätzliche Information aus der zeitlichen Abfolge der Messwerte ergibt, da das dynamische Verhalten des Systems infolge einer Stellgrößenänderung sichtbar wird. Daraus ergibt sich eine Aufteilung der Datengrundlage in zwei Datensätze.

Für den Datensatz der quasi-stationären Zustände ist es essenziell, einen möglichst breiten und gleichmäßig verteilten Datenraum abzudecken, um robuste Aussagen über den stationären Zusammenhang zwischen Ein- und Ausgangsgrößen zu ermöglichen. Zu diesem Zweck kann statistische Versuchsplanung (engl. statistische Versuchsplanung (Design of Experiments) (DOE)) herangezogen werden. Konkret findet dazu der so genannte Latin-Hypercube-Ansatz Anwendung, bei dem Stichproben aus einem Parameterraum so gezogen werden, dass jeder Parameter über seinen definierten Wertebereich gleichmäßig verteilt ist [MBC00]. Innerhalb der minimalen und maximalen Grenzen aller betrachteten Stellgrößen wird so ein zufällig verteilter Datenraum mit einer vorgegebenen Anzahl von Stichproben erzeugt. Die resultierenden Betriebspunkte werden anschließend hinsichtlich ihrer technischen Umsetzbarkeit geprüft und gefiltert. Im Rahmen eines Feldexperiments an der in Abschnitt 2.1.1 beschriebenen Mühle konnten auf diese Weise insgesamt 18 Betriebspunkte realisiert werden, welche den Datenraum des regulären Produkti-

onsbetriebs signifikant erweitern. Die Anforderung an einen stationären Prozesszustand bestand bei dem Experiment darin, dass nach der letzten Änderung einer Stellgröße alle Regelgrößen für einen Zeitraum von mindestens zehn Minuten unverändert bleiben mussten.

Für den zweiten Datensatz, der die transienten Prozesszustände abbilden soll, reicht die Anzahl der im Experiment erzeugten Übergänge zwischen stationären Betriebspunkten nicht aus. Daher ist eine Ergänzung des Datensatzes durch Ereignisse aus dem laufenden Produktionsbetrieb erforderlich. Um sicherzustellen, dass der Prozess während dieser Zeiträume keinen grundlegenden Veränderungen unterliegt, ist die Datenauswahl auf einen Zeitraum weniger Wochen um das Experiment zu beschränken, in dem die Modellperformance des statischen Modells als konstant beobachtet wird (vgl. Abbildung 1.2). Die Identifikation der transienten Bereiche erfolgt in diesem Kontext anhand der Setpoint-Änderungen ausgewählter Stellgrößen, die durch die Operatoren initiiert werden. Hinsichtlich der zeitlichen Länge der betrachteten Intervalle ist a priori jedoch unklar, über welchen Zeitraum der zeitliche Verlauf der Daten einen relevanten Einfluss auf die Zielgröße ausübt. Um diesen Einfluss zu untersuchen, werden für jeden Übergang vor und nach der Setpoint-Änderung Zeitfenster mit den Längen 30, 60 und 120 Sekunden betrachtet. Die Auswertung zeigt, dass die Intervalllänge innerhalb dieser Grenzen lediglich einen marginalen Einfluss auf die Modellgüte besitzt. Vor diesem Hintergrund wurden für alle weiteren Untersuchungen Zeiträume von $60\,\mathrm{s}$ verwendet.
Da einige Stellgrößen im Produktionsbetrieb deutlich häufiger verändert werden als andere, ist es darüber hinaus notwendig, die Anzahl dieser häufig wechselnden Signale zu reduzieren. Auf diese Weise wird eine möglichst uniforme Verteilung der betrachteten transienten Zustände über alle relevanten Stellgrößen angestrebt, um eine Verzerrung des Datensatzes zu vermeiden.

2.2.4 Modellvergleich

Aufbauend auf der Trennung der beiden Datensätze soll nun die Leistungsfähigkeit verschiedener statischer und dynamischer Modellansätze systematisch miteinander verglichen werden. Als Grundlage dazu dient eine umfassende Grid-Search-basierte Hyperparameter-Optimierung, die es erlaubt, sowohl unterschiedliche Modelltypen als auch relevante Vorverarbeitungsschritte zu evaluieren [Ma26]. Vor dem eigentlichen Modelltraining ist eine einheitliche Vorverarbeitung beider Datensätze erforderlich. Dazu werden die Daten zunächst in Trainings-, Validierungs- und Testdatensätze im prozentualen Verhältnis 70 : 20 : 10 aufgeteilt. Anschließend werden Ausreißer identifiziert und aus den Daten entfernt. Um den Einfluss hochfrequenter Störungen zu reduzieren, werden verrauschte Signale außerdem je nach Hyperparameter-Konfiguration entweder mithilfe eines digitalen Butterworth-Filters oder durch einen gleitenden Mittelwert geglättet [OS10]. Abschließend erfolgt eine Normalisierung der Daten auf Basis manuell festgelegter Minima und Maxima, um eine vergleichbare Skalierung sicherzustellen.

Im Rahmen der Hyperparameter-Optimierung werden neben den unterschiedlichen Varianten

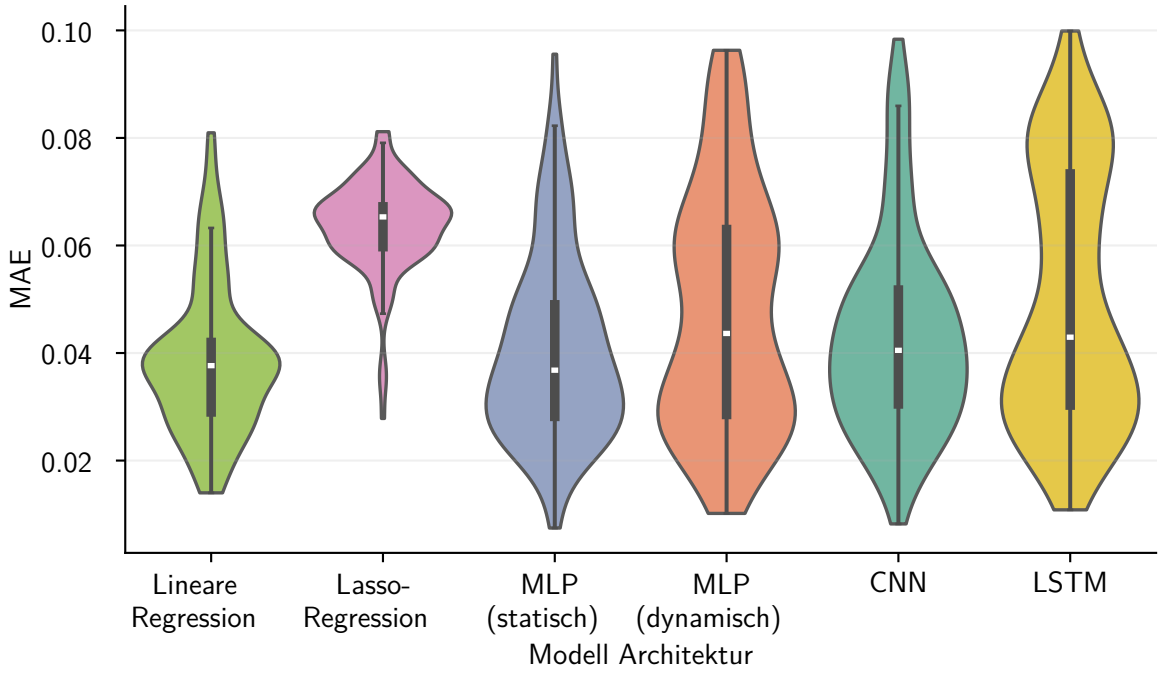
Tabelle 2.1: Übersicht der verwendeten Modelltypen

Modellname	**Dynamik-Charakter**	**Aktivierungsfunktion**
Lineare Regression	statisch	–
Lasso-Regression	statisch	–
`MLP`	statisch	Tanh
`MLP`	dynamisch	Tanh
`CNN`	dynamisch	ReLU
`LSTM`	dynamisch	Tanh

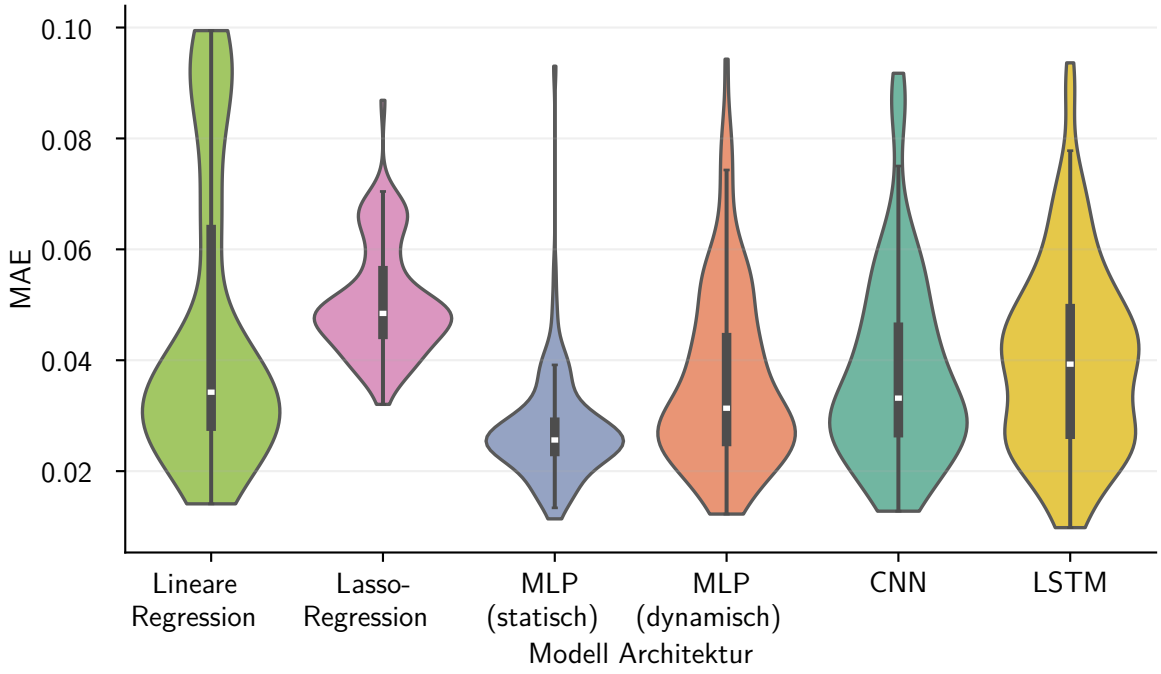
zur Datenglättung für die zeitabhängigen Modellansätze verschiedene Fensterbreiten berücksichtigt. Der Modellvergleich und dessen Bewertung erfolgt innerhalb des Hyperparameter-Tuning-Prozesses, indem mehrere Modelltypen auf beiden Datensätzen trainiert und bewertet werden. Eine konsistente und vergleichbare Bewertung der Modelle wird gewährleistet, da beide Datensätze dieselben Features und dasselbe Label enthalten. Zur Sicherstellung der Reproduzierbarkeit sämtlicher Experimente für alle Trainingsläufe wird ein fester Zufallsstartwert (*Seed*) verwendet. Insgesamt umfasst der Vergleich sechs unterschiedliche Modellklassen, die entweder statischen oder dynamischen Charakter besitzen und in Tabelle 2.1 zusammengefasst sind. Zusammenfassend werden bei der Hyperparameter-Optimierung mittels Grid-Search-Strategie folgende Parameter variiert:

- der Modelltyp (siehe Tabelle 2.1)
- die Art der Datenglättung (keine Glättung, digitale Filterung, gleitender Mittelwert)
- die Anzahl der Neuronen in den Hidden-Layern (8, 16 und 64), sofern für den Modelltyp anwendbar
- die Fensterbreite mit den Werten $5\,\mathrm{s}$, $10\,\mathrm{s}$, $20\,\mathrm{s}$, $30\,\mathrm{s}$ und $60\,\mathrm{s}$

Durch die Kombination dieser Parameter ergeben sich insgesamt über 500 unterschiedliche Modell- und Hyperparameterkonfigurationen. Die Bewertung der Modelle erfolgt anhand des mittleren quadratischen Fehlers (engl. mittlerer quadratischer Fehler (Mean Squared Error) (`MSE`)) und des mittleren absoluten Fehlers (engl. mittlerer absoluter Fehler (Mean Absolute Error) (`MAE`)). Unabhängig davon, auf welchem der beiden Datensätze ein Modell trainiert ist, werden beide Metriken stets auf den Testdaten beider Datensätze berechnet. Diese wechselseitige Evaluation ermöglicht eine robuste Einschätzung der Generalisierungsfähigkeit der Modelle sowohl für stationäre als auch für transiente Prozesszustände. Alle Trainings- und Evaluationsläufe wurden auf einem vom Deutschen Forschungszentrum für Künstliche Intelligenz (DFKI) bereitgestellten Linux-Hochleistungsrechner durchgeführt.



(a) Quasi-stationäre Testdaten



(b) Transiente Testdaten

Abbildung 2.4: Kombinierte Violin- und Boxplots der `MAE`-Verteilung aller Modelle auf quasi-stationären und transienten Testdaten

Tabelle 2.2: Vergleich der besten Modellarchitekturen auf stationären und transienten Daten

	Stationäre Daten		**Transiente Daten**	
Modelltyp	**Bester `MAE`**	**Δ Best**	**Bester `MAE`**	**Δ Best**
Lineare Regression	0,01400	+87,7%	0,01410	+43,7%
Lasso-Regression	0,02783	+273,3%	0,03205	+226,6%
`MLP` (statisch)	0,00745	**0,0%**	0,01140	+16,2%
`MLP` (dynamisch)	0,01017	+36,5%	0,01226	+25,0%
`CNN`	0,00822	+10,3%	0,01278	+30,3%
`LSTM`	0,01084	+45,4%	0,00981	**0,0%**

Zur Bewertung der im Rahmen der Hyperparameter-Grid-Search untersuchten Modelle werden alle Konfigurationen auf trainingsdisjunkten Testdaten beider Datensätze evaluiert. Zunächst ist festzuhalten, dass sich der eingesetzte Butterworth-Filter als die effektivste Glättungstechnik erweist, da er die Vorhersageleistung sämtlicher Modellarchitekturen konsistent verbessert. In Bezug auf die Performance der jeweiligen Modellarchitekturen lässt sich grundsätzlich festhalten, dass die erzielten Performances aller Modellarchitekturen sehr dicht beieinander liegen. Die Unterschiede im mittleren absoluten Fehler (`MAE`) manifestieren sich häufig erst in der dritten oder vierten Nachkommastelle. So wurden geringfügig abweichende Ergebnisse in einem anderen Durchgang der gleichen Versuchsreihe in [Ma26] erreicht. Diese hohe Ergebnisdichte impliziert, dass viele Modelle aus statistischer Sicht als gleichwertig betrachtet werden können, wobei minimale Abweichungen plausibel auf geringe Schwankungen im Trainingsprozess zurückzuführen sind.

Grundsätzlich schneiden lineare Modelle, also die lineare Regression und die Lasso-Regression, schlechter ab als ihre nichtlinearen Pendants. Ihre dennoch vergleichsweise gute Leistung deutet auf eine insgesamt moderate Systemkomplexität hin. Gleichzeitig zeigt der direkte Vergleich, dass lineare Modelle nicht in der Lage sind, die vorhandenen nichtlinearen Zusammenhänge in gleichem Maße zu erfassen wie komplexere Modellarchitekturen.

Unter Berücksichtigung der beiden unterschiedlichen Datensätze sind die besten Modellperformances basierend auf dem jeweils minimalen Simulationsfehler für alle untersuchten Modellarchitekturen in Tabelle 2.2 zusammengefasst. Zusätzlich ist für jedes Modell die relative Abweichung Δ zum jeweils besten Modell innerhalb derselben Kategorie angegeben. Ergänzt werden die tabellarischen Ergebnisse durch die Verteilungsplots in Abbildung 2.4. Während Abbildung 2.4a die Ergebnisse auf den quasi-stationären Testdaten darstellt, zeigen die Plots in Abbildung 2.4b die der transienten Testdaten. Bei den Plots handelt es sich um eine Kombinati-

on aus Violin- und Boxplots. Hierbei ermöglicht der Violinplot durch seine Kerndichteschätzung die Visualisierung der strukturellen Eigenschaften der Verteilung. Zusätzlich bietet der Boxplot eine statistische Einordnung und berücksichtigt auch untere „Ausreißer“, die zur Identifikation der besten Ergebnisse relevant sind.

Die Analyse der quasi-stationären Testdaten ergibt, dass sich die Fehlerverteilungen der nicht-linearen Modelle nur marginal unterscheiden. Dennoch ist bei den besten Konfigurationen eine Überlegenheit von Feedforward-Architekturen erkennbar. Insbesondere erzielt das statische `MLP` hier die beste Performance über alle durchgeführten Experimente. Jedoch zeigen auch die dynamischen Modelle, die zeitliche Fenster berücksichtigen, auf den quasi-stationären Daten insgesamt gute Ergebnisse. Insbesondere das `CNN` weist eine vergleichbare Performance auf. Das `LSTM` hingegen weist mit seiner rekurrenten Architektur sowohl eine schlechtere Best-Performance als auch eine insgesamt nach oben verschobene Fehlerverteilung auf (siehe Abbildung 2.4a). Dies deutet darauf hin, dass die interne Zustandsrückkopplung des `LSTM` bei stationären und zeitlich invarianten Mustern keinen zusätzlichen Informationsgewinn liefert, sondern zu einem Overfitting auf Rauschanteile führt.

Bei der Betrachtung der transienten Testdaten, die durch dynamische zeitliche Veränderungen charakterisiert sind, ergibt sich ein anderes Bild. Erwartungsgemäß erzielt hier das `LSTM` den geringsten Simulationsfehler, da seine Architektur explizit für die Modellierung sequenzieller Abhängigkeiten konzipiert ist. Die Anzahl der Neuronen je Hidden-Layer in den Modellen mit der besten Performance liegt zwischen 8 und 16. Diese relativ geringe Anzahl deutet darauf hin, dass auch bei Auswahl des komplexeren dynamischen Modelltyps eine geringe Komplexität ausreichend ist. Bemerkenswert ist jedoch die hohe Leistungsfähigkeit des statischen `MLP` auch auf dem dynamischen Datensatz. Das bestperformenste `MLP` liegt nur knapp hinter dem deutlich komplexeren `LSTM`. Darüber hinaus zeigt die Verteilung der MAEs aller statischen `MLP`-Konfigurationen eine hohe Dichte im unteren Fehlerbereich, was in Abbildung 2.4b deutlich wird. Dass die nicht-linearen dynamischen Modelle in den transienten Bereichen nicht signifikant besser performen, lässt vermuten, dass die Datenerfassung mit lediglich einem Hertz die Dynamik des Systems nicht ausreichend abbildet.

Für die Auswahl einer geeigneten Modellarchitektur zur Beschreibung des betrachteten Prozesses lässt sich zusammenfassend festhalten, dass keine Architektur auf beiden Datensätzen gleichermaßen als überlegen identifiziert werden kann. Daher ist die Generalisierungsfähigkeit über beide Domänen hinweg ein zentrales Auswahlkriterium, welches für den Einsatz des statischen `MLP` spricht. Die Fehlerverteilungen zeigen außerdem, dass dieses Modell eine hohe Dichte guter Ergebnisse aufweist und somit weniger sensitiv gegenüber suboptimalen Hyperparameter-Wahlen reagiert. Es erweist sich folglich als robuster im Trainingsprozess im Vergleich zu den übrigen Architekturen. Darüber hinaus ist zu berücksichtigen, welcher Systemzustand für die angestrebte Anwendung von höherer Relevanz ist. Diese Frage lässt sich zwar nicht allgemeingültig beantworten, im Kontext der Betriebspunktoptimierung kommt dem stationären Zustand

jedoch eine größere Bedeutung zu. Schließlich ist bei vergleichbarer Leistungsfähigkeit gemäß dem Prinzip der Parsimonie das einfachere Modell zu bevorzugen. Vor diesem Hintergrund stellt das statische MLP einen geeigneten und ausgewogenen Allrounder für die vorliegende Problemstellung dar.

2.3 Concept Drift

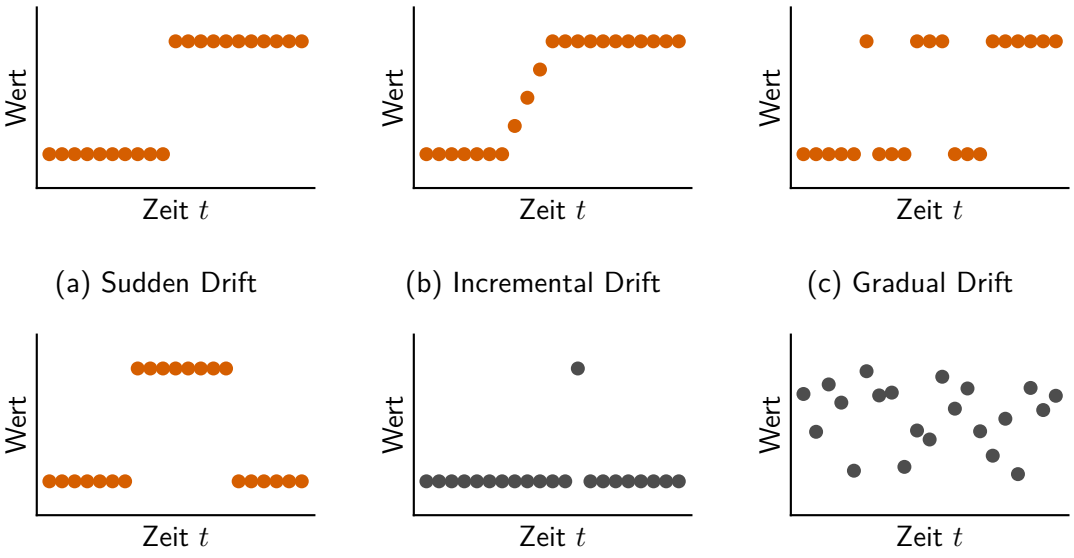
Datengetriebene Modelle wie das in Abschnitt 2.2 entwickelte statische MLP setzen implizit voraus, dass die zugrunde liegende Datenverteilung über die Zeit stabil bleibt. In realen Industrieumgebungen ist diese Annahme jedoch häufig verletzt, da sich Prozesse infolge von Verschleiß, Materialwechseln oder veränderten Betriebsbedingungen kontinuierlich verändern können. Dieses Phänomen wird als CD bezeichnet und stellt eine zentrale Herausforderung für den dauerhaften Einsatz datengetriebener Modelle dar. Aufbauend auf der in Abschnitt 2.2 eingeführten Prozessmodellierung werden in diesem Abschnitt die theoretischen Grundlagen zum Umgang mit CD erarbeitet. Dazu wird zunächst in Abschnitt 2.3.1 eine einheitliche Terminologie etabliert und die verschiedenen Ausprägungen von CD mathematisch abgegrenzt. Anschließend werden in Abschnitt 2.3.2 ausgewählte Methoden zur Erkennung von CD im Datenstrom vorgestellt. Abschließend behandelt Abschnitt 2.3.3 Strategien zur Modelladaption unter CD.

2.3.1 Terminologie

Viele Anwendungen der datengetriebenen Prozessmodellierung arbeiten in der industriellen Praxis in dynamischen Umgebungen, in denen die zugrunde liegenden Prozesse nicht streng stationär sind. Werden die Ursachen dieser Dynamiken während des Modelltrainings nicht erfasst, die den Prozess jedoch erheblich beeinflussen, spricht man von einem so genannten *Hidden Context* [WK96]. Darüber hinaus ist es möglich, dass sich die dem Modell zugrunde liegende Zielfunktion über die Zeit hinweg verändert [HPC12]. In der Praxis werden diese veränderlichen Einflüsse bei der Modellentwicklung jedoch häufig nicht ausreichend berücksichtigt. Dies kann sich dann im laufenden Betrieb in einem Leistungsabfall des Modells äußern.

Wie sich dieser Leistungsabfall phänomenologisch darstellen kann, ist in Abbildung 2.5 veranschaulicht. Dabei werden vier typische Arten unterschieden, nämlich (a) die plötzliche Veränderung (engl. *Sudden Drift*), (b) die inkrementelle Veränderung (engl. *Incremental Drift*), (c) die schleichende Veränderung (engl. *Gradual Drift*) sowie (d) die wiederkehrende Veränderung (engl. *Recurring Drift*). Diese systematischen Veränderungen sind jedoch von (e) Ausreißern und (f) statistischem Rauschen abzugrenzen.

Obwohl das beschriebene Phänomen der Modelldegradation in der Literatur weitreichend be-



(a) Sudden Drift (b) Incremental Drift (c) Gradual Drift

(d) Recurring Drift (e) Ausreißer (f) Rauschen

Abbildung 2.5: Phänomenologisches Verhalten von Datenverschiebungen

kannt ist, werden für die verschiedenen Ausprägungen oft unterschiedliche und teils inkonsistente Begriffe verwendet. In manchen Fällen finden sich sogar identische Begrifflichkeiten für gänzlich unterschiedliche Effekte [BAK22]. Häufig verwendete Begriffe stellen in diesem Kontext *Dataset Shift* und *Concept Drift* dar. Grundlegend ist jedoch jede Art von Drift oder Shift durch eine spezifische Veränderung der zugrunde liegenden Datenverteilung gekennzeichnet. Der Begriff des Dataset Shift wird dabei verwendet, um das Abweichen der Datenverteilung zwischen einem statischen Trainings- und einem Test-Szenario zu beschreiben [Qu08]. Concept Drift hingegen bezieht sich primär auf den zeitlichen Verlauf von Verteilungsänderungen im Kontext von kontinuierlichen Datenströmen und Online-Learning-Szenarien [Ga14]. Webb et al. bezeichnen den Concept Drift im Bereich des Stream-Learnings als das direkte Pendant zum Dataset Shift im Batch-Learning [We16].

Obwohl es sich bei der vorliegenden Problemstellung methodisch um Batch-Learning handelt, resultiert die in Abschnitt 1.1 beschriebene Veränderung aus einem zeit-kontinuierlichem Datenstrom. Daher fokussiert sich diese Arbeit auf den Concept Drift. Dazu soll zunächst definiert werden, was unter *Concept* zu verstehen ist. Gama et al. definieren in dem Zusammenhang ein Concept C_t als multivariate Wahrscheinlichkeitsverteilung der Features X und der Labels y zu einem bestimmten Zeitpunkt t [Ga14; We16]. Damit gilt:

$$C_t = P_t(X, y) \tag{2.6}$$

Im Zusammenhang der Definition des Concept Drifts ist anzumerken, dass auch hier in der Literatur einige Begriffe existieren, die als unterschiedliche Ausprägungsformen des Concept Drifts verstanden werden können. Dazu zählen *Permanent Drift*, *Temporary Drift*, *Sampling Shift*, *Feature Change*, *Conditional Change*, *Global Drift*, *Label Shift* und *Target Shift*, um nur eine Auswahl zu nennen [BAK22]. Das Fehlen einer einheitlichen Terminologie wurde 2012

Tabelle 2.3: Terminologie und ihre zugehörigen mathematischen Definitionen

Terminologie	**Mathematische Definition**	
Covariate Shift	$P_t(X) \neq P_{t+\Delta t}(X)$	(2.7)
Prior probability Shift	$P_t(y) \neq P_{t+\Delta t}(y)$	(2.8)
Concept Drift	$P_t(X, y) \neq P_{t+\Delta t}(X, y)$	(2.9)
Real Concept Drift	$P_t(y \mid X) \neq P_{t+\Delta t}(y \mid X)$	(2.10)
Actual Drift	$P_t(y \mid X) \neq P_{t+\Delta t}(y \mid X)$ und $P_t(X) = P_{t+\Delta t}(X)$	(2.11)
Virtual Drift	$P_t(y \mid X) = P_{t+\Delta t}(y \mid X)$ und $P_t(X) \neq P_{t+\Delta t}(X)$	(2.12)

erstmals maßgeblich von Moreno-Torres et al. thematisiert [Mo12].

Um die terminologische Unschärfe aufzulösen, ist es daher zielführend, die verschiedenen Effekte mathematisch voneinander abzugrenzen. Einen solchen Ansatz verfolgen Bayram et al., die die Begriffe der Literatur anhand ihrer mathematischen Definitionen strukturieren [BAK22]. Basierend auf dieser Systematik sind die verschiedenen Arten von Drifts in Tabelle 2.3 aufgeführt, wobei für jede mathematische Definition je ein in der Literatur verwendeter Begriff ausgewählt wurde. Demzufolge lässt sich Concept Drift formal gemäß (2.9) als die Veränderung der multivariaten Verteilung P der Features X und der Labels y zwischen zwei Zeitpunkten t und $t+\Delta t$ beschreiben. Wird lediglich die Veränderung der Verteilung der Features betrachtet, also der veränderlichen Kovariablen (engl. covariate), ist gemäß (2.7) die Rede von *Covariate Shift*. Analog wird beim *Prior probability Shift* gemäß (2.8) ausschließlich die Veränderung der Zielgröße betrachtet. Für die genauere Betrachtung der verschiedenen Formen von Concept Drift sind *Real Concept Drift* gemäß (2.10), *Actual Drift* gemäß (2.11) und *Virtual Drift* gemäß (2.12) zu unterscheiden. Ihnen gemein ist die Betrachtung der a-posteriori-Wahrscheinlichkeitsverteilung der Labels $P_t(y \mid X)$ zu verschiedenen Zeitpunkten. Gemäß dem Satz von Bayes lässt sich die bedingte Wahrscheinlichkeit wie folgt umformulieren:

$$P_t(y \mid X) = \frac{P_t(X \mid y) \cdot P_t(y)}{P_t(X)} \tag{2.13}$$

Dabei bezeichnet $P_t(y \mid X)$ die a-posteriori-Wahrscheinlichkeitsverteilung der Labels und $P_t(X)$ die Wahrscheinlichkeitsverteilung der Eingabedaten. $P_t(y)$ stellt weiter die a-priori-Wahrscheinlichkeitsverteilung der Labels und $P_t(X|y)$ die bedingte bedingte Wahrscheinlichkeitsdichteverteilung (engl. Likelihood) dar.

2.3.2 Drift-Erkennung

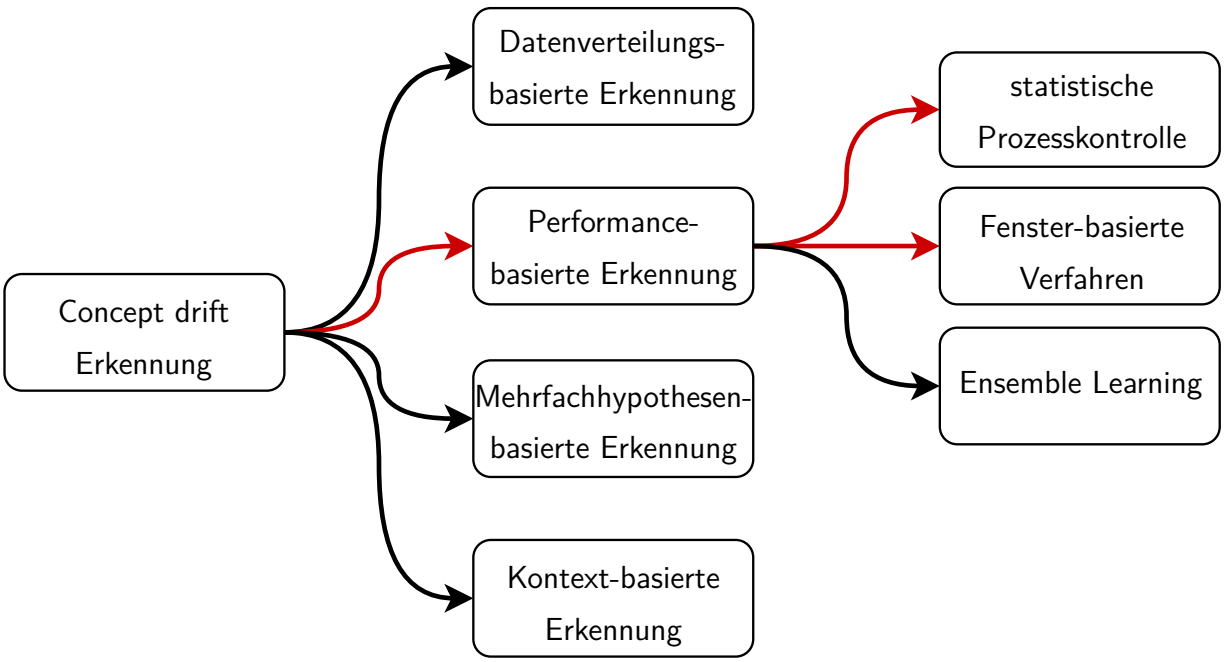


Abbildung 2.6: Methoden zur Erkennung von CD angelehnt an [BAK22]

Um einen CD gemäß (2.9) in einem kontinuierlichen Datenstrom festzustellen, existiert in der Literatur eine Vielzahl von Erkennungsverfahren, die als Drift-Detektoren bezeichnet werden. Im Kern basieren die Detektionsmethoden unabhängig von ihrer algorithmischen Implementierung auf statistischen Hypothesentests [BAK22]. Hierfür wird kontinuierlich eine Teststatistik berechnet, welche die strukturelle Ähnlichkeit oder Abweichung zwischen einer historischen Referenzdatenmenge $\mathcal{D}_t$ und einer aktuell erhobenen Datenmenge $\mathcal{D}_{t+w}$ zu einem späteren Zeitpunkt $t+w$ quantifiziert. Beide Datenmengen bestehen aus Paaren von Eingangsvariablen X und zugehörigen Zielvariablen y und stellen empirische Stichproben der jeweiligen zugrunde liegenden gemeinsamen Wahrscheinlichkeitsverteilungen dar.

Die zugrunde liegende Nullhypothese H_0 nimmt stets an, dass sich die wahren Wahrscheinlichkeitsverteilungen der beiden Datenmengen nicht signifikant unterscheiden – folglich also kein CD vorliegt. Formal lässt sich dies durch einen Hypothesentest und eine Teststatistik D ausdrücken:

$$H_0 : P(\mathcal{D}_t) = P(\mathcal{D}_{t+w}) \quad \text{gegen} \quad H_1 : P(\mathcal{D}_t) \neq P(\mathcal{D}_{t+w}) \tag{2.14}$$

Um diese Hypothese zu überprüfen, wird ein empirischer Wert der Teststatistik $D(\mathcal{D}_t, \mathcal{D}_{t+w})$ berechnet und mit einem vordefinierten Schwellenwert λ abgeglichen [DR09]. Solange $D(\mathcal{D}_t, \mathcal{D}_{t+w}) \leq \lambda$ gilt, kann H_0 nicht verworfen werden und das System geht von einer stabilen Verteilung aus. Wird der Schwellenwert hingegen überschritten, also $D > \lambda$, wird die Nullhypothese zum gewählten Signifikanzniveau verworfen und das System deklariert einen CD.

Abhängig von der Teststatistik können verschiedene Methoden der Drift-Erkennung unterschieden werden, wie in Abbildung 2.6 dargestellt [BAK22]. Die rot markierten Pfade stellen dabei

diejenigen Ansätze dar, die in dieser Arbeit verwendet wurden. Allen Drift-Detektionsmethoden liegt dieselbe Nullhypothese zugrunde, wonach keine signifikante Änderung der Datenverteilung vorliegt. Die Unterschiede ergeben sich hingegen aus der Wahl der Teststatistik und der Art der Auswertung des Datenstroms.

Bei der *Datenverteilungs-basierten Erkennung* werden Distanzmaße verwendet, um die Ähnlichkeit zwischen den Datenverteilungen in zwei verschiedenen Zeitfenstern abzuschätzen [KBG04]. Zwar ist dieser Ansatz auch für ungelabelte Daten geeignet, es wird jedoch nicht berücksichtigt, ob die Änderungen tatsächlich die Vorhersageleistung des Modells beeinflussen [Ga14]. *Multiple hypothesis-based Drift-Detektoren* stellen hybride Verfahren dar, die mehrere Detektionsmethoden parallel oder aufeinanderfolgend auswerten [Lu18]. Während parallele Ansätze Entscheidungsregeln kombinieren, nutzen hierarchische Verfahren eine Struktur aus Warnung und Validierung. Unter *Kontext-basierter Drift-Erkennung* lassen sich solche Ansätze zusammenfassen, die klassische Detektoren um kontextuelle Informationen aus System oder Modell erweitern [BAK22]. Der Begriff kann hier als Sammelbegriff für unterschiedliche methodische Ausprägungen verstanden werden.

Ansätze der *Performance-basierten Erkennung* verfolgen Abweichungen im Output-Fehler des Modells, um Veränderungen zu erkennen. Ihr Hauptvorteil besteht darin, dass so Drifts nur detektieren werden, wenn auch die Leistung beeinträchtigt wird. Wie in Abbildung 2.6 veranschaulicht, lassen sich diese Methoden je nach der zur Erkennung von Leistungseinbrüchen verwendeten Strategie weiter in folgende Kategorien einteilen: *statistische Prozesskontrolle*, *Fenster-basierte Verfahren* und *Ensemble-Learning*.

Bei der statistischen Prozesskontrolle wird die Qualität des (kontinuierlichen) Lernprozesses anhand der zeitlichen Entwicklung der Fehlerrate überwacht. Überschreitet die Fehlerrate eine statistisch definierte Schwelle, wird ein `CD` detektiert. Die wohl am häufigsten zitierte Methode dieser Kategorie ist die so genannte **acDDM** von Gama et al. [Ga04]. Drift-Erkennungsmethode (Drift Detection Method) (`DDM`) überwacht die Fehlerrate im Datenstrom und basiert auf der Annahme, dass jede Modellentscheidung entweder korrekt oder fehlerhaft ist. Entsprechend werden Fehlklassifikationen als Bernoulli-Zufallsvariable modelliert, sodass pro Beobachtung lediglich zwei Ausprägungen (Richtig oder Falsch) möglich sind. Nach i Beobachtungen werden die Fehlerrate p_i sowie deren Standardabweichung

$$\sigma_i = \sqrt{\frac{p_i(1-p_i)}{i}} \tag{2.15}$$

geschätzt. Unter der Annahme eines stationären Zustands und für eine hinreichend große Anzahl an Beobachtungen kann die zugrunde liegende Binomialverteilung durch eine Normalverteilung approximiert werden. `DDM` speichert dabei die bislang minimal beobachteten Fehlerraten $p_{\min}$ und Standardabweichungen $\sigma_{\min}$, die fortlaufend aktualisiert werden, sofern $p_i + \sigma_i < p_{\min} + \sigma_{\min}$. Ein Warnzustand wird dann ausgelöst, wenn die aktuelle Fehlerrate signifikant vom bisherigen Minimum abweicht, konkret wenn $p_i + \sigma_i \geq p_{\min} + 2\ \sigma_{\min}$. Über-

schreitet die Abweichung eine Schwelle von $p_i + \sigma_i \geq p_{\min} + 3\ \sigma_{\min}$, wird ein CD detektiert. Der Warnzustand erlaubt dabei eine frühzeitige Indikation eines potenziellen Drifts und wird häufig zur Anpassung des Beobachtungsfensters genutzt.

Eine Erweiterung der DDM stellt die **Early Drift Detection Method (EDDM)** dar [Ba06]. EDDM wurde insbesondere zur verbesserten Erkennung gradueller Konzeptdrifts entwickelt, ohne dabei bei abrupten Driftformen an Performance einzubüßen. Im Gegensatz zu DDM analysiert EDDM die zeitlichen Abstände zwischen aufeinanderfolgenden Fehlklassifikationen. Die zentrale Annahme ist dabei, dass ein gradueller Drift zu einer sukzessiven Verkürzung der Abstände zwischen aufeinanderfolgenden Fehlklassifikationen führt. Für jedes Zeitintervall i wird daher die mittlere Distanz zwischen zwei Fehlklassifikationen p'_i sowie deren Standardabweichung σ'_i geschätzt. Während des Lernprozesses speichert EDDM die Referenzwerte $p'_{\max}$ und $\sigma'_{\max}$, sofern $p'_i + 2\ \sigma'_i > p'_{\max} + 2\,\sigma'_{\max}$. Diese Referenzwerte entsprechen dem Zustand, in dem das Modell das zugrunde liegende Konzept am besten approximiert. Zur Drift-Erkennung sei die dimensionslose Kennzahl

$$r_i = \frac{p'_i + 2\,\sigma'_i}{p'_{\max} + 2\,\sigma'_{\max}} \tag{2.16}$$

definiert, welche den aktuellen Abstandsausdruck relativ zum bislang maximal beobachteten Referenzwert beschreibt. Auf Basis dieser Größe werden zwei Schwellenwerte unterschieden:

- Ein Warnzustand wird ausgelöst, wenn $r_i < \alpha$.
- Ein Konzeptdrift wird detektiert, wenn $r_i < \beta$, wobei $\beta < \alpha$.

Nach Erreichen des Driftzustands werden die Referenzwerte $p'_{\max}$ und $\sigma'_{\max}$ zurückgesetzt.

Bei Fenster-basierten Verfahren wird ein Datenstrom in Referenz- und Detektionsfenster geteilt. Drifts werden anhand signifikanter Unterschiede von statistischen Tests zwischen beiden Fenstern erkannt. Die Fenster können dabei zeit- oder größenbasiert definiert und flexibel positioniert werden. Ein Fenster-basierter Ansatz ist das **Adaptive Windowing (ADWIN)** von Bifet et al. (2007) [BG07]. Dabei wird CD anhand sequentieller statistischer Hypothesentests über ein adaptives Datenfenster erkannt. Ziel des Algorithmus ist es, ein Fenster W so groß wie möglich zu halten, solange die Nullhypothese einer konstanten Erwartung innerhalb dieses Fensters nicht verworfen werden kann. Dazu wird W fortlaufend in zwei aufeinanderfolgende Teilfenster W_0 und W_1 mit Längen n_0 und n_1 unterteilt, sodass $n = n_0 + n_1$. Mit der Zerlegung des aktuellen Fensters wird geprüft, ob sich die beobachteten Mittelwerte $\hat{\mu}_{W_0}$ und $\hat{\mu}_{W_1}$ signifikant unterscheiden. Die Entscheidungsregel basiert auf der Hoeffding-Schranke ε_{cut}, mit

$$\varepsilon_{\text{cut}} = \sqrt{\frac{1}{2m} \cdot \ln(\frac{4}{\delta'})}\ , \quad m = \frac{1}{{}^1\!/_{n_0} + {}^1\!/_{n_1}} \quad \text{und} \quad \delta' = \frac{\delta}{n}\ , \tag{2.17}$$

wobei m das harmonische Mittel der Fensterlängen darstellt und δ das globale Signifikanzniveau des Hypothesentests bezeichnet. Das angepasste Niveau δ' wird zur Kontrolle multipler Hypothesentests eingeführt. Überschreitet die Differenz der Mittelwerte den Schwellenwert ε_{cut},

wird die Nullhypothese verworfen und auf einen CD geschlossen, woraufhin der ältere Fensteranteil W_0 entfernt wird. Andernfalls bleibt das Fenster unverändert, sodass sich das Fenster durch neue Datenpunkte sukzessive vergrößert. Als Schätzung der aktuellen Erwartung gibt ADWIN kontinuierlich den empirischen Mittelwert $\hat{\mu}_W$ des aktuellen Fensters aus. Das zugrunde liegende Prinzip von ADWIN ist unabhängig vom konkreten statistischen Test formuliert und erlaubt alternative Formen einer Teststatistik.

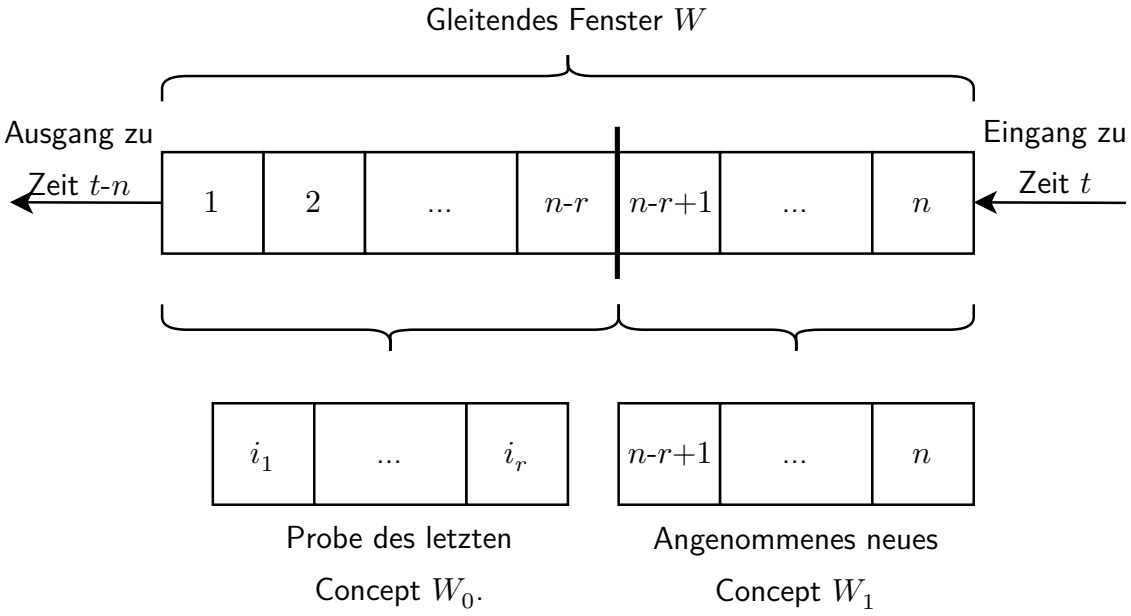


Abbildung 2.7: Speicherstrategie von Kolmogorov-Smirnov Windowing (KSWIN) angelehnt an [RHS20]

Ein weiterer Fenster-basierter Ansatz zur Erkennung von CD ist das **KSWIN** von Raab et al. (2020) [RHS20]. KSWIN basiert auf einem Gleitfenster fester Größe, das kontinuierlich die zuletzt beobachteten Datenpunkte eines Datenstroms speichert und mithilfe eines nichtparametrischen statistischen Tests auf Verteilungsänderungen untersucht. Hierzu wird ein Fenster W der Größe n geführt, das mit jedem neu eintreffenden Datenpunkt aktualisiert wird, indem der älteste Wert entfernt und der neue Wert hinzugefügt wird. Aus diesem Fenster werden zwei Teilfenster konstruiert: Ein aktuelles Detektionsfenster W_1, das die r zuletzt eingetroffenen Datenpunkte enthält, sowie ein Referenzfenster W_0, das ebenfalls aus r Datenpunkten besteht. Diese werden uniform zufällig aus dem älteren Bereich des Fensters W gezogen, der nicht zu W_1 gehört. Auf diese Weise soll W_0 die historische Datenverteilung approximieren, ohne Annahmen über deren Form zu treffen. Die Speicherstrategie ist in Abbildung 2.7 veranschaulicht.

Zur Drift-Erkennung wird der *Kolmogorov-Smirnov-Test* zwischen den empirischen Verteilungsfunktionen der beiden Teilfenster W_0 und W_1 durchgeführt [Lo11]. Der Test basiert auf der maximalen absoluten Abweichung

$$\mathrm{dist}(W_0, W_1) = \sup_x \left| F_{W_0}(x) - F_{W_1}(x) \right| \ , \tag{2.18}$$

wobei F_{W_0} und F_{W_1} die empirischen kumulativen Verteilungsfunktionen der jeweiligen Fenster darstellen. Überschreitet diese Distanz einen durch ein Signifikanzniveau bestimmten Schwellenwert, wird die Nullhypothese gleicher Verteilungen verworfen und auf einen CD geschlossen.

2.3.3 Drift-Adaption

Wie einleitend in Abschnitt 1.1 exemplarisch dargestellt, kann die Modellperformance durch CD beeinflusst werden. Durch Methoden zur Erkennung von CD, wie sie in Abschnitt 2.3.2 beschrieben sind, lässt sich der Drift zwar detektieren, das durch den Drift beeinträchtigte Modell bleibt dabei jedoch unverändert. Zur Verbesserung der Performance bedarf es folglich einer Anpassung des Modells. Einen umfassenden und bis heute maßgeblichen Überblick über Strategien zur Anpassung von Modellen unter CD, der in der Literatur häufig als Referenz herangezogen wird, liefern Gama et al. in ihrer Survey-Arbeit zur CD-Adaption [Ga14].

Zunächst hängt die konkrete Ausgestaltung einer Anpassung gegenüber CD davon ab, welche Informationen im Datenstrom vorgehalten und wie sie verwaltet werden. Bei der Speicherung von Informationen im Datenstrom können grundlegend zwei Speichertypen unterschieden werden: dem Kurz- und dem Langzeitspeicher. Beim Kurzzeitspeicher handelt es sich um die Daten selbst, die verarbeitet werden. Der Langzeitspeicher hingegen stellt eine Generalisierung von Daten dar, nämlich das Modell. Beide Speichergrundlagen sind essenziell für die Handhabung von CD. Der Kurzzeitspeicher bestimmt, welche aktuellen Informationen beim Anpassungsprozess verwendet werden sollen, etwa einzelne Datenpunkte oder gleitende Fenster fixer oder variabler Größe. Ergänzend regeln Vergessensmechanismen, wie ältere Informationen verworfen werden. Hier wiederum kann zwischen abrupten und graduellen Ansätzen unterschieden werden. Ein schnelleres Vergessen erhöht in diesem Zusammenhang zwar die Reaktionsfähigkeit gegenüber Drifts, geht jedoch mit einer höheren Anfälligkeit gegenüber Rauschen einher.

Der zweite Speichertyp – das Modell – steht im Zentrum der Anpassung. Ziel modellbasierter Anpassungsstrategien ist es, die im Modell verankerte Generalisierung an veränderte Zusammenhänge anzupassen und dadurch die Prädiktionsgüte unter Driftbedingungen zu verbessern bzw. wiederherzustellen. Bei der Adaption gilt als eine besondere Herausforderung das so genannte *Stability-plasticity-dilemma*, dem Spagat zwischen der Fähigkeit eines Modells, neues Wissen aufzunehmen (Plastizität) und vorhandenes Wissen zu erhalten (Stabilität). Hierbei das richtige Gleichgewicht zu finden ist von zentraler Bedeutung, da zu viel Plastizität zu katastrophalem Vergessen führen kann, also dem Verlust bereits erlernter Informationen, wenn neue Informationen erfasst werden. Gleichzeitig kann zu viel Stabilität dafür sorgen, dass Anpassungen an Drifts nicht wirksam sind.

Zur Anpassung eines Modells an veränderte Datenverteilungen existieren grundsätzlich zwei Strategien. Einerseits kann ein vollständiges Neutrainieren erfolgen, bei dem auf Basis gepufferter, aktualisierter Daten ein neues Modell erzeugt und das bisherige Modell verworfen

wird. Dieser Ansatz wird häufig als *Full Relearning* bezeichnet und dient in der Forschung als robuste Referenz, ist jedoch mit erhöhtem Rechenaufwand und Datenbedarf verbunden. Insbesondere bei Notwendigkeit von statistischer Versuchsplanung zur Datenakquise stellt dies eine große Herausforderung dar. Andererseits kann das bestehende Modell inkrementell angepasst werden, indem neue Beobachtungen schrittweise in das Modell integriert werden. Dieses Vorgehen ist dem Bereich des *Continual Learning* bzw. des *Incremental Learning* zuzuordnen und zielt darauf ab, eine kontinuierliche Modelladaption ohne vollständig neues Modelltraining zu ermöglichen. Gerade unter CD stellt diese Strategie jedoch besondere Anforderungen, da fortlaufende Updates das Risiko katastrophalen Vergessens erhöhen und ein sorgfältiges Austarieren von Stabilität und Plastizität erfordern. In der Praxis spiegelt sich dieser Zielkonflikt in einem Trade-off zwischen Adaptionsgeschwindigkeit, Robustheit und Validierbarkeit wider. Hybride Ansätze versuchen hier eine Lösung dieses Zielkonfliktes zu bieten [Ga04].

Neben der Art der Modellanpassung ist entscheidend, wann bzw. wodurch diese ausgelöst wird. Abhängig davon, ob eine Anpassung gezielt durch erkannte Veränderungen erfolgt oder unabhängig davon kontinuierlich durchgeführt wird, können folgende Ansätze unterschieden werden:

- **Aktive Anpassung (engl. auch informed adaptation)**
  Die ergebnisgetriebene Anpassung wird durch den Output der Drift-Erkennung ausgelöst und kann entsprechend zielgerichtet erfolgen. Dieser reaktive Charakter macht sie jedoch vollständig abhängig von der Detektionsqualität, da nicht erkannte Drifts keine Anpassung auslösen. Die Methode eignet sich daher vor allem für plötzliche, klar identifizierbare Drifts.
- **Passive Anpassung (engl. auch blind adaptation)**
  Die Anpassung erfolgt kontinuierlich und unabhängig von etwaigen Driftinformationen. Der Ansatz ist sehr einfach anzuwenden, da es keiner Drift-Erkennung bedarf. Bei zu niedriger Anpassungsfrequenz besteht jedoch das Risiko, auf bereits eingetretene Drifts nicht zu reagieren. Bei zu hoher Anpassungsfrequenz wird das Modell ohne vorliegenden Drift ohne Notwendigkeit angepasst, was zu Overfitting führen kann und das katastrophale Vergessen begünstigt. Diese Methode ist proaktiv und daher besonders geeignet für schleichende oder nicht klar trennbare Drifts, da die Anpassung kontinuierlich und unabhängig von einer Drift-Erkennung erfolgt.

Der Langzeitspeicher kann darüber hinaus auch so gestaltet sein, dass mehrere Modelle parallel vorgehalten und angepasst werden. Ein etablierter Ansatz ist dabei das Ensemble-Learning, bei dem eine Zusammenstellung aus mehreren Modellen gemeinsam eine Vorhersage generiert. Adaptive Ensembles basieren in dem Kontext auf der Annahme, dass Daten während einer Veränderung aus einer gemischten Verteilung stammen, die als gewichtete Kombination von Verteilungen betrachtet werden kann, die die Zielkonzepte charakterisieren. Jedes einzelne Modell bildet dabei eine Verteilung ab [SK07]. Die endgültige Vorhersage ergibt sich typischer-

weise als gewichtete Kombination der Einzelvorhersagen, wobei die Gewichtung die Leistung der jeweiligen Modelle auf den neuesten Daten widerspiegelt und im zeitlichen Verlauf angepasst wird. Bei wiederkehrenden Konzeptdrifts bieten Ensemble-Ansätze in Kombination mit einer geeigneten Modellverwaltung besondere Vorteile. Dadurch kann der mit erneutem Training verbundene Rechenaufwand deutlich reduziert und die Reaktionsfähigkeit des Systems auf bereits bekannte Konzepte verbessert werden. Die konkrete Behandlung solcher rekurrenten Drifts ist dabei stark vom eingesetzten Verfahren und der Driftcharakteristik abhängig.

3 Generalisierung

Grundlage dieser Arbeit ist ein Regressionsmodell, das, wie in Abschnitt 2.2 beschrieben, mit einem DOE-basierten Datensatz aus Sensordaten im stationären Zustand eines Prozesses der Prozessindustrie trainiert wurde. Als geeignete Modellarchitektur wurde hierfür ein MLP identifiziert. Die Güte dieses Modells wird jedoch durch CD beeinträchtigt, dessen Grundlagen in Abschnitt 2.3 allgemein dargestellt sind. Verändert sich das zugrunde liegende Prozessverhalten über die Zeit, weicht die Datengrundlage zunehmend von der Trainingsverteilung ab, sodass die Modellperformance sinkt, wie in Kapitel 1 skizziert. Diese Problemstellung soll nicht an die Besonderheiten der Vertikalrollenmühle gebunden bleiben und soll daher zunächst von der spezifischen Anwendung gelöst und auf allgemeine prozesstechnische Fragestellungen übertragen werden. Dieses Kapitel widmet sich damit der ersten Leitfrage aus Abschnitt 1.2. Auf dem so generalisierten Problem setzt der Lösungsansatz an, der dem drift-bedingten Leistungsabfall durch eine kontinuierliche Adaption des Modells begegnet.

Damit eine Datengrundlage diese generalisierende Rolle erfüllen kann, muss sie einerseits hinreichend allgemein sein, um eine Generalisierung zu ermöglichen, und andererseits genügend Ähnlichkeit zum ursprünglichen Prozess aufweisen, um methodische Konsistenz sicherzustellen. Grundlage einer validen Generalisierung ist daher die Auswahl einer geeigneten Datengrundlage, die folgende Voraussetzungen erfüllt:

- **Abbildung realer Prozesscharakteristik**
  Der Datensatz muss reale physikalische Eigenschaften wie Systemträgheit, Sensorrauschen und komplexe Korrelationen zwischen den Variablen abbilden, wie sie für verfahrenstechnische Anlagen charakteristisch sind.
- **Klassifizierung der Variablen und Kausalität**
  Der Datensatz muss eine klare Unterscheidung zwischen Stellgrößen, Messgrößen und Störgrößen sowie eine von den Eingangsgrößen abhängige Zielgröße bereitstellen, um die Kausalität des Modells korrekt abzubilden.
- **Steuerbarkeit und Betriebspunktabdeckung**
  Die Setpoints müssen aktiv ansteuerbar sein, um Betriebspunkte gezielt zu erzeugen, und es muss eine hinreichend große Anzahl unterschiedlicher, durch Stellgrößen definierter Betriebszustände vorliegen bzw. erzeugbar sein. Das ist essenziell, um eine statistische Versuchsplanung zu simulieren und eine robuste Aufteilung in Trainings-, Validierungs-

und Testdaten zu ermöglichen.

- **Erfassbarkeit stationärer Betriebspunkte**
  Da das Modell auf stationären Betriebspunkten arbeitet, muss die Abtastrate ausreichend hoch sein, um stationäre Zustände zuverlässig von transienten Phasen trennen und je Betriebspunkt eine ausreichende Zahl an Messpunkten für die Extraktion bereitstellen zu können.
- **Abbildbarkeit von CD**
  Der Datensatz muss über einen hinreichenden Zeithorizont Szenarien enthalten oder deren Simulation ermöglichen, in denen sich das Prozessverhalten signifikant ändert, sodass sowohl die Driftentwicklung als auch die wiederholte Evaluation der Adaptionsmechanismen abgebildet werden können.
- **Drift-freie Referenzdaten**
  Für das initiale Training des Basismodells ist ein Zeitraum erforderlich, in dem der Prozess stabil und ohne systematische Abweichungen abläuft.
- **Kontinuierliche Verfügbarkeit der Zielgröße**
  Die Zielgröße muss fortlaufend vorliegen, da die Adaption das Modell auf den driftbehafteten Labels nachtrainiert.
- **Öffentliche Verfügbarkeit und Reproduzierbarkeit**
  Zur Gewährleistung der Vergleichbarkeit der Ergebnisse muss der Datensatz öffentlich zugänglich, die Datenerhebung transparent dokumentiert und die Ergebnisse reproduzierbar sein.

Die Generalisierung erfolgt in zwei Stufen zunehmender Realitätsnähe. Zunächst wird in Abschnitt 3.1 ein synthetischer Datenstrom erzeugt, der die genannten Voraussetzungen unter vollständig kontrollierten Bedingungen per Konstruktion erfüllt und den CD mit bekannter Ground Truth vorgibt. Darauf aufbauend betrachtet Abschnitt 3.2 mit dem Tennessee Eastman Process (TEP) einen realitätsnäheren, öffentlich verfügbaren Benchmark-Prozess, der reale, gekoppelte Prozessdynamik und Regelstrukturen einbringt und die Voraussetzungen weitgehend erfüllt, sodass allein der CD modifiziert werden muss.

3.1 Synthetischer Datensatz

Als erster Schritt zur Generalisierung wird ein synthetischer Datenstrom erzeugt, der unter kontrollierten Bedingungen die Erprobung des Adaptionsansatzes bei CD ermöglicht. Dabei sollen die wesentlichen Eigenschaften prozesstechnischer Messdaten synthetisch nachgebildet werden. Abbildung 3.1 gibt einen Überblick über die zugrunde liegende Simulationsstrategie. Die drei Hauptkomponenten - die exogenen Setpoints, die Aktuatordynamik und die Systemantwort -

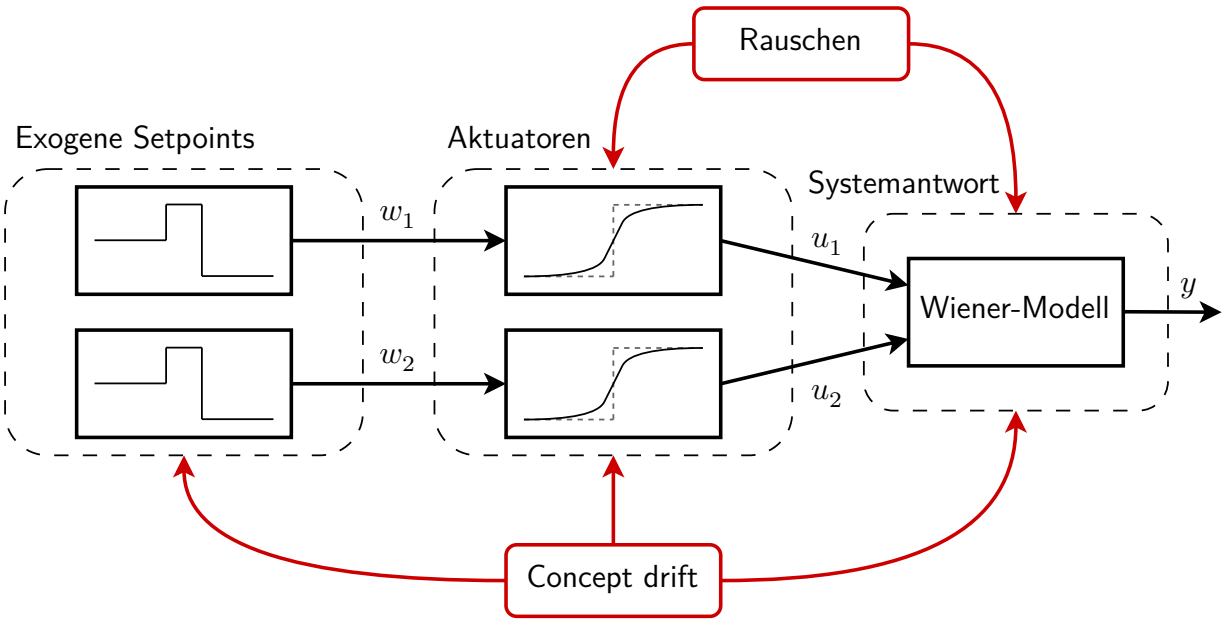


Abbildung 3.1: Übersicht zur Simulationsstrategie

werden dabei von CD beeinflusst, dessen Modellierung in Abschnitt 3.1.1 beschrieben wird. Darauf aufbauend wird in Abschnitt 3.1.2 die Erzeugung stückweise konstanter exogener Setpoints erläutert. Diese durchlaufen PT_1-Glieder, die als Spezialfall der in Abschnitt 2.2.1 eingeführten Zustandsraumdarstellung die Trägheit der Aktuatoren nachbilden. Deren Modellierung ist in Abschnitt 3.1.3 ausgeführt. Das heterogene Rauschmodell, beschrieben in Abschnitt 3.1.4, überlagert die erzeugten Signale additiv als Beobachtungsrauschen und wirkt damit unabhängig vom CD. Die rauschfreien Aktuatorsignale dienen als Eingang eines Wiener-Modells, das eine gewichtete Summation, ein weiteres PT_1-Glied zur Abbildung der Prozessträgheit und eine statische Nichtlinearität in Serie schaltet; sein Ausgang y bildet die beobachtete Systemantwort. Das Wiener-Modell wird in Abschnitt 3.1.5 näher spezifiziert und auf den vorliegenden Fall angewandt. Die gesamte Datenstromsynthese wurde in Python implementiert. Der erzeugte Datenstrom bildet die Grundlage der anschließenden Verarbeitungskette aus Modellierung, Detektion und Adaption. Zunächst wird der synthetische Prozess in Abschnitt 3.1.6 modelliert, wobei ein MLP sowie ein Kalman- und ein Extended-Kalman-Filter gegenübergestellt werden. Aufbauend auf dem Simulationsfehler des MLP behandelt Abschnitt 3.1.7 die Detektion des CD. Abschnitt 3.1.8 schließt die Kette mit der Adaption des Modells an den drift-behafteten Prozess ab.

3.1.1 Concept Drift

Einen typischen Mechanismus in der industriellen Praxis, den CD hervorrufen kann, stellen kontinuierliche Veränderungen infolge von Einflüssen wie Verschleiß und sich verändernden äußeren Bedingungen dar. Wie in Abbildung 2.5 dargestellt, können sich diese Einflüsse in Form von *Incremental Drift* äußern. Dem gegenüber stehen kurzfristige Maßnahmen wie Wartungs-

oder Instandhaltungsereignisse, die *Sudden Drifts* verursachen können. Bei der Synthese des Driftsignals $c[k]$ liegt der Fokus auf diesen beiden charakteristischen Phänomenen.

Das zeitabhängige Driftsignal $c[k]$ dient dabei als Grundlage für alle driftabhängigen Modifikationen des synthetischen Datenstroms. Es nimmt zu jedem Zeitschritt k einen skalaren Wert an und beeinflusst sowohl die Parameter der PT_1-Glieder als auch die Setpoints sowie den stationären Arbeitspunkt der Systemantwort. Die beiden genannten Drifttypen werden durch die Bildung von Segmenten kombiniert. Innerhalb jedes Segments nimmt $c[k]$ einen linearen Verlauf negativer Steigung an, was einen *Incremental Drift* repräsentiert. An den Grenzen zwischen den Segmenten hingegen wird durch einen sprungartigen Übergang auf ein neues Ausgangsniveau ein *Sudden Drift* erzeugt. Diese Struktur erzeugt ein sägezahnartiges Muster, das beide Driftformen miteinander kombiniert.

Formal erstreckt sich das Signal über eine Gesamtlänge von K Zeitschritten. Die Wahl der Abtastfrequenz orientiert sich dabei am Ausgangsbeispiel der Mühle mit GPlink, das mit einer Frequenz von $1\,\mathrm{Hz}$ arbeitet. Die entsprechende Abtastzeit $T_s = 1\,\mathrm{s}$ wird auf ein Kalenderjahr angewandt, dessen Zeitschritte auf Zeitstempel ab dem 1. Januar 1970 abgebildet werden, dem Nullpunkt der Unix-Zeit. Ein Kalenderbezug trägt für die Auswertung keine Information, weshalb der Nullpunkt der üblichen Zeitrechnung von Rechnersystemen als neutraler Beginn dient. Die Segmentierung dieses Zeitbereichs folgt einem festen Raster mit der nominellen Segmentdauer $T_{\mathrm{seg}} = 1752\,\mathrm{h}$, die als Wartungs- beziehungsweise Kalibrierintervall zu verstehen ist. Im betrachteten Kalenderjahr liegen damit 5 Driftsegmente vor. Da ein Wartungstermin in der Praxis um einige Tage schwankt, werden die inneren Segmentgrenzen $\tau_1 < \tau_2 < \cdots$ gegenüber dem Raster um maximal $\pm\gamma_\tau \cdot T_{\mathrm{seg}}/2$ zufällig verschoben, wobei γ_τ den Verschiebungsanteil bezeichnet. Zusammen mit dem festen Anfangspunkt $\tau_0 = 0$ und dem Ende des Zeitbereichs bei K sind damit alle Segmentgrenzen definiert.

Innerhalb des i-ten Segments wird $c[k]$ durch lineare Interpolation (*Incremental Drift*) vom Startniveau c_i^+ zum Zielniveau c_i^- geformt:

$$c[k] = c_i^+ + \frac{c_i^- - c_i^+}{\tau_i - \tau_{i-1}}\,(k - \tau_{i-1}), \qquad k \in [\tau_{i-1},\, \tau_i) \tag{3.1}$$

Das Zielniveau $c_i^- \sim \mathcal{U}(c_{\min},\, c_{\min}/4)$ wird aus dem negativen Teilbereich gezogen, sodass $c[k]$ innerhalb jedes Segments monoton fällt. An der Segmentgrenze τ_i erfolgt ein sprungartiger Übergang (*Sudden Drift*) zum Startniveau des folgenden Segments. Sämtliche Startniveaus $c_i^+ \sim \mathcal{U}(c_{\max}/4,\, c_{\max})$ werden unabhängig aus dem positiven Teilbereich gezogen, während das letzte Segment ohne abschließenden Sprung ausläuft. Das Driftsignal ist damit durch die Parameter $c_{\min}$, $c_{\max}$, T_{seg} und γ_τ vollständig charakterisiert. In der Simulation werden $c_{\min} = -0{,}5$, $c_{\max} = 0{,}5$ und $\gamma_\tau = 0{,}2$ verwendet. Abbildung 3.2 zeigt den resultierenden Verlauf des Driftsignals $c[k]$ über ein Kalenderjahr. Trotz des diskreten Charakters von $c[k]$ erfolgt die Darstellung als kontinuierlicher Verlauf, da der stückweise lineare Verlauf so unmittelbar erkennbar wird. Jedes Segment erscheint dabei als eigene Linie, die vom oberen zum unteren

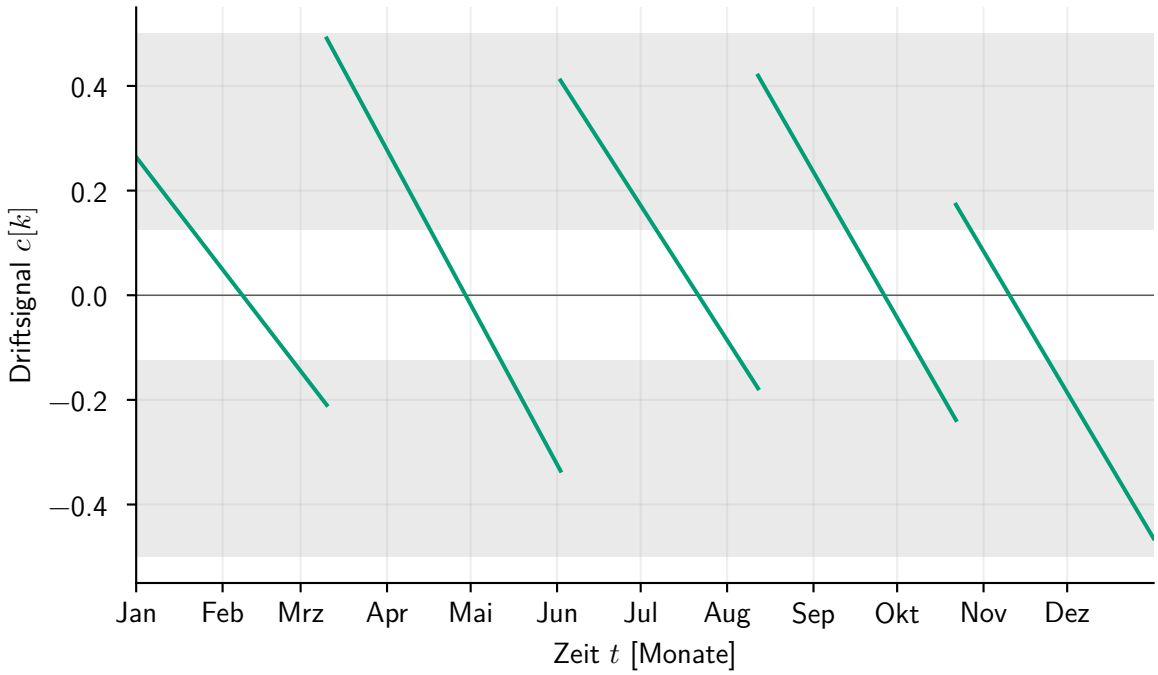


Abbildung 3.2: Verlauf des synthetischen Driftsignals $c[k]$ über ein Kalenderjahr

grau hinterlegten Band abfällt und so den *Incremental Drift* abbildet. Die Lücken zwischen den Linien markieren die sprungartigen Anstiege des *Sudden Drift* an den Segmentgrenzen. Da c_{i+1}^{+} stets aus dem oberen und c_i^{-} stets aus dem unteren Band gezogen wird, sind die beiden Drifttypen im Signalverlauf klar voneinander unterscheidbar und das Signal deckt den Wertebereich $[c_{\min}, c_{\max}]$ näherungsweise ab.

3.1.2 Setpoints

In industriellen Prozessen stellt der Setpoint den vom Anlagenoperator vorgegebenen Sollwert dar, auf den die beteiligten Aggregate und Regelkreise eingestellt werden. Der Setpoint definiert somit den angestrebten Betriebszustand der Anlage und wird in der Praxis abhängig von betrieblichen Zielgrößen sowie beobachteten Prozessbedingungen angepasst. Es wird angenommen, dass der Operator die Setpoints auch in Reaktion auf drift-bedingte Änderungen des Anlagenverhaltens anpasst. Folglich soll der indirekte Einfluss von `CD` auf die Setpoints berücksichtigt werden. Formal ergibt sich die resultierende, durch `CD` beeinflusste Setpoint-Zeitreihe $\tilde{w}[k]$ als Überlagerung einer Basis-Zeitreihe $w[k]$ und einer driftinduzierten Korrektur:

$$\tilde{w}[k] = w[k] + \kappa_w \cdot c_w[k], \quad k = 0, 1, \ldots, K-1 \,, \tag{3.2}$$

wobei k den diskreten Zeitindex mit Abtastzeit T_s bezeichnet, sodass $t = k \cdot T_s$. Weiter stellen κ_w den Setpoint-Driftkoeffizienten und $c_w[k]$ das segmentweise konstante Driftsignal dar. Die Basis-Setpoint-Zeitreihe $w[k]$ wird als stückweise konstante Folge definiert und stellt die exogene Anregung des Systems dar. Zwischen aufeinanderfolgenden Änderungsindizes $0 = k_0 < k_1 < \cdots < k_n = K$ nimmt sie jeweils einen konstanten Wert $w[k] = s_i$ für

$k \in [k_{i-1},\, k_i)$, $i = 1, \dots, n$, an. Die Werte s_i werden zufällig aus einem vorgegebenen Wertebereich $(r_{\min}, r_{\max})$ gezogen. Daraus resultiert eine stückweise konstante Setpoint-Zeitreihe, die als Eingangsgröße für die Generierung von Prozesssignalen dient. Die daraus abgeleiteten Signale approximieren Betriebszustände, sodass das Systemverhalten unter sich verändernden Umgebungsbedingungen untersucht werden kann.

Die Generierung erfolgt durch das Ziehen von N_{c} zufälligen, eindeutigen inneren Änderungsindizes $k_1 < \cdots < k_{N_{\mathrm{c}}}$ aus dem Indexvektor $\mathcal{K} = \{1, \dots, K-1\}$, die zusammen mit den festen Rändern $k_0 = 0$ und $k_n = K$ die insgesamt $n = N_{\mathrm{c}} + 1$ Indexsegmente begrenzen. Jedem Segment wird ein zufällig aus dem vorgegebenen Wertebereich gezogener Setpoint-Wert zugewiesen.

Diese Anpassung der Setpoint-Zeitreihe als indirekte Reaktion auf CD wird über das Driftsignal $c_w[k]$ modelliert. Da der Operator nur zu den Änderungsindizes eingreift, wird das Driftsignal $c[k]$ am jeweiligen Segmentanfang k_{i-1} abgetastet und über das gesamte Segment $[k_{i-1},\, k_i)$ konstant gehalten (Zero-Order Hold):

$$c_w[k] = c[k_{i-1}], \quad k \in [k_{i-1},\, k_i), \quad i = 1, \dots, n \tag{3.3}$$

Der Setpoint-Driftkoeffizient κ_w skaliert die Stärke der Operatorreaktion und wird relativ zur Spannweite $(r_{\max} - r_{\min})$ des Setpoint-Bereichs gewählt. Die driftinduzierte Korrektur $\kappa_w \cdot c_w[k]$ hat damit dieselbe Segmentstruktur wie $w[k]$ und ist zwischen zwei Änderungsindizes konstant. Dadurch bildet die resultierende Setpoint-Zeitreihe $\tilde{w}[k]$ sowohl die vorgegebene Anregungsstruktur ab als auch die Reaktion des Operators auf nicht-stationäre Prozessbedingungen.

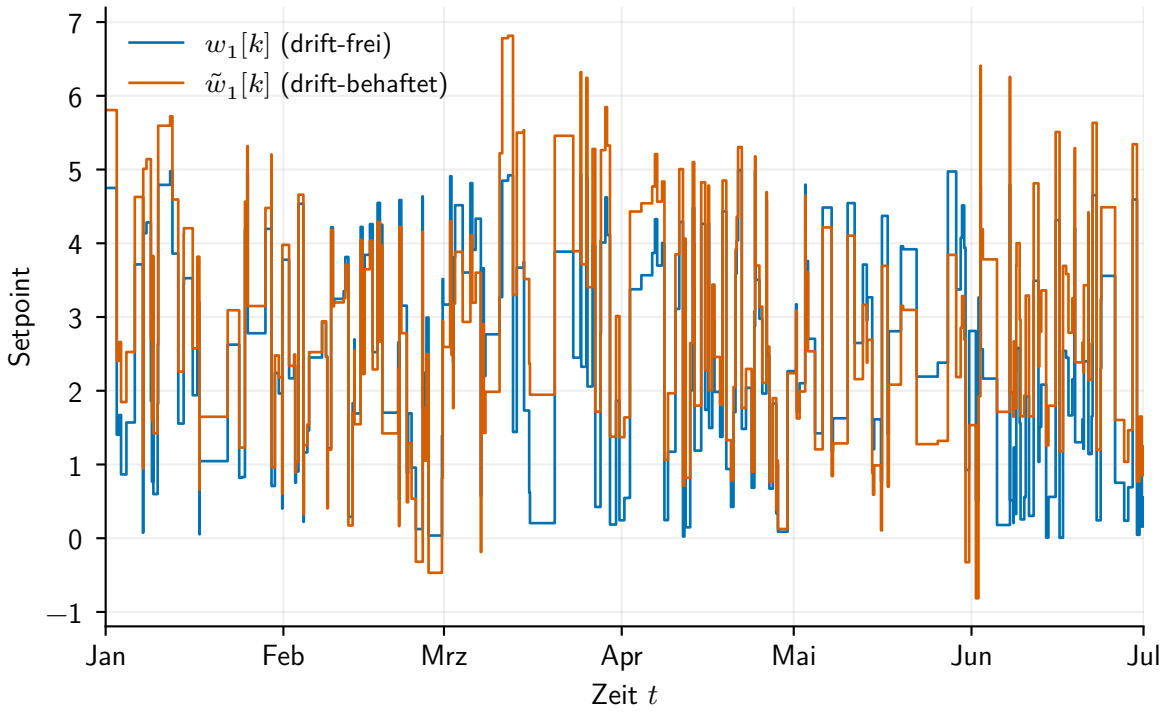


Abbildung 3.3: Einfluss von CD auf eine Setpoint-Zeitreihe über ein halbes Jahr

Abbildung 3.3 zeigt exemplarisch eine Basis-Setpoint-Zeitreihe $w[k]$ sowie die daraus resultierende, durch CD beeinflusste Setpoint-Zeitreihe $\tilde{w}[k]$ für einen Zeitraum von einem halben Jahr. Analog zur Darstellung der Drifts in Abbildung 3.2 sind die Signale hier trotz des diskreten Charakters der Zeitreihe kontinuierlich dargestellt, um den stückweise konstanten Verlauf und die Übergänge hervorzuheben. Die Basis-Zeitreihe besitzt dabei einen Wertebereich von $(r_{\min}, r_{\max}) = (0, 5)$ sowie eine Anzahl von Setpoint-Änderungen über ein Jahr von $N_{\mathrm{c}} = 500$. Die driftinduzierte Korrektur ist mit dem Driftkoeffizienten $\kappa_w = 4$ berechnet, was $80\,\%$ der Spannweite des Setpoint-Bereichs entspricht. Die Abbildung verdeutlicht, dass die durch CD beeinflusste Setpoint-Zeitreihe gegenüber der Basis-Zeitreihe systematisch verschoben wird und dabei sowohl Erweiterungen des Wertebereichs als auch veränderte Lagen der stationären Segmente auftreten.

3.1.3 Signal-Inputs

Zur Simulation der Prozesssignale wird ein lineares dynamisches System erster Ordnung (PT_1-Glied) verwendet, welches das verzögerte Anpassungsverhalten vieler technischer Systeme in kompakter Form beschreibt. Es erlaubt die Approximation von Einschwingvorgängen, wie sie beispielsweise in thermischen Systemen, Speicherprozessen oder mechanisch trägen Aggregaten auftreten. Dadurch können realistische Übergänge zwischen stationären Betriebszuständen erzeugt werden. Zwar sind die transienten Übergänge für die nachfolgende Modellierung anhand eines stationären MLP nicht unmittelbar relevant, werden aber aus Gründen der Realitätsnähe und Reproduzierbarkeit mitberechnet. Das PT_1-Glied kann dabei als Spezialfall der in Abschnitt 2.2.1 eingeführten allgemeinen Zustandsraumdarstellung interpretiert werden. Während Gleichung (2.2) jedoch eine allgemein nicht-lineare Systembeschreibung zugrunde legt, wird beim PT_1-Glied ein vereinfachtes Modell mit linearer Dynamik betrachtet. Konkret wird angenommen, dass die Zustandsänderung proportional zur Abweichung zwischen aktuellem Zustand und dem Eingangssignal ist. Daraus ergibt sich die (zeit-kontinuierliche) Differentialgleichung

$$\dot{u}(t) = -\frac{1}{T}\, u(t) + \frac{K_p}{T}\, w(t)\ , \tag{3.4}$$

wobei T die Zeitkonstante, K_p die statische Verstärkung, $w(t)$ das Eingangssignal in Form der zeit-kontinuierlichen Setpoint-Zeitreihe und $u(t)$ den Ausgang des PT_1-Gliedes bezeichnen.

Mit dem konstanten Abtastintervall T_s und dem Zeitindex k ergibt sich so die zeitdiskrete rekursive Darstellung

$$u^o[k] = \frac{T}{T + T_s}\, u^o[k-1] + \frac{K_p \cdot T_s}{T + T_s}\, w[k]\ , \tag{3.5}$$

wobei $w[k]$ die diskrete Setpoint-Zeitreihe und $u^o[k]$ das resultierende rauschfreie Signal zum Zeitschritt k bezeichnen. $u^o[k]$ beschreibt damit die dynamische Antwort des PT_1-Gliedes auf die Setpoint-Zeitreihe und dient als Eingangssignal für das nachfolgend betrachtete System. Es werden $N_{\mathrm{in}} = 2$ Eingangssignale erzeugt, die jeweils ein eigenes PT_1-Glied durchlaufen.

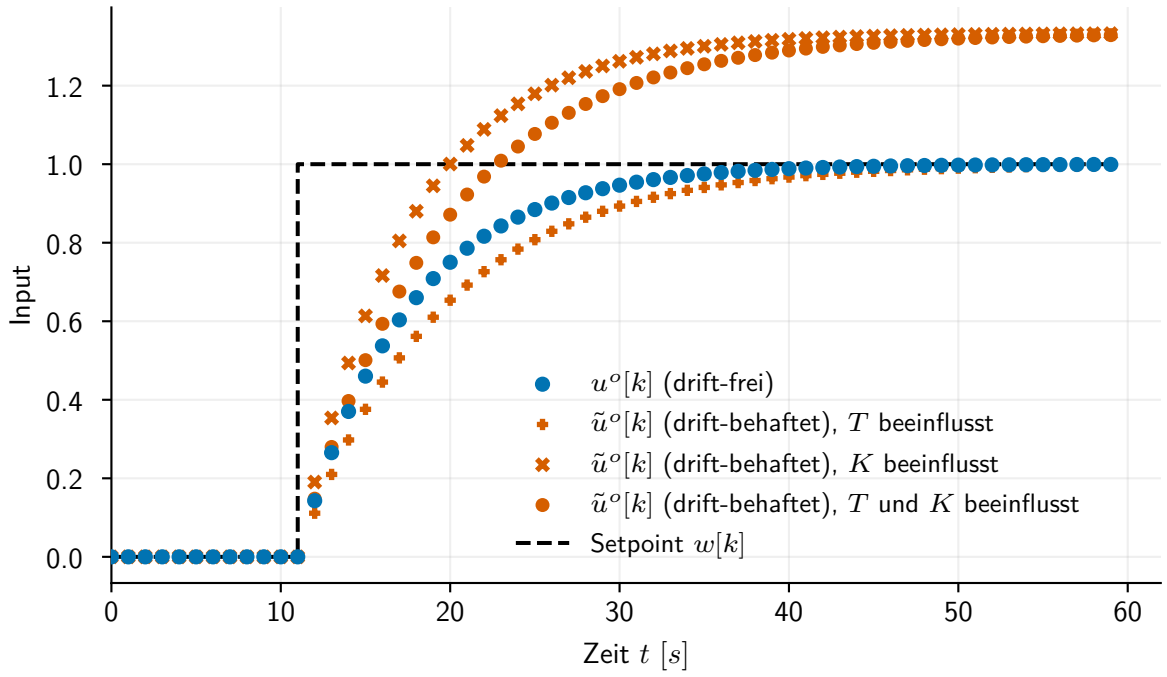


Abbildung 3.4: Qualitative Darstellung des Einflusses von CD auf ein PT_1-Glied

Im Kontext von CD wird angenommen, dass sich der Drift in einer Veränderung der dynamischen Systemeigenschaften und veränderten stationären Zuständen äußert. Die Parameter K_p und T aus Gleichung (3.5) werden dazu als zeitvariant modelliert und hängen vom CD-Wert $c[k]$ zum jeweiligen Zeitschritt ab:

$$K_p[k] = K_p \cdot (1 + c[k]), \quad T[k] = T \cdot (1 + c[k]), \quad \forall k \tag{3.6}$$

Für die drift-behafteten Signale wird der Setpoint $\tilde{w}[k]$ (vgl. Abschnitt 3.1.2) als Eingangsgröße verwendet. Die resultierenden drift-behafteten Ausgangssignale der PT_1-Glieder werden im Folgenden als $\tilde{u}_i^o[k]$ bezeichnet.

Abbildung 3.4 stellt qualitativ den Einfluss von CD auf das Verhalten des PT_1-Gliedes dar. Während das System ohne Drift ein konstantes Einschwingverhalten aufweist, führt die driftabhängige Variation der Parameter $T[k]$ und $K_p[k]$ zu einer veränderten Dynamik, die sich in unterschiedlichen Anstiegszeiten und veränderten stationären Werten äußert. Dabei ist der Einfluss des CD hier exemplarisch durch eine proportionale Skalierung der Parameter mit einem konstanten Driftfaktor $c = \frac{1}{3}$ modelliert.

3.1.4 Rauschen

Reale Sensordaten unterliegen typischerweise mehreren überlagerten Störquellen unterschiedlicher Charakteristik wie feinem thermischen Grundrauschen, gelegentlichen Messimpulsen sowie selteneren Ausreißerereignissen. Zur realistischen Synthese des Datenstroms wird der Signal-Input daher zusätzlich mit einem Rauschterm überlagert. Ein einfaches additives weißes gauß-

sches Rauschen mit einer einzigen Standardabweichung σ ist dabei nicht in der Lage, diese Heterogenität adäquat abzubilden. Daher basiert das verwendete Rauschmodell auf einem Gaussian Mixture Model (GMM), das N_{q} überlagerte Rauschquellen mit unterschiedlichen Standardabweichungen und Auftrittswahrscheinlichkeiten beschreibt. Die Wahrscheinlichkeitsdichtefunktion des Rauschterms η lautet:

$$p(\eta) \;=\; \sum_{j=1}^{N_{\mathrm{q}}} \nu_j \cdot \mathcal{N}\big(\eta \,\big|\, 0,\, \sigma_j^2\big)\,, \qquad \sum_{j=1}^{N_{\mathrm{q}}} \nu_j = 1, \quad \nu_j \geq 0\,, \tag{3.7}$$

wobei ν_j das normierte Gewicht und σ_j die Standardabweichung der j-ten Komponente bezeichnen. $\mathcal{N}\big(\eta \,\big|\, 0,\, \sigma_j^2\big)$ bezeichnet dabei die Normalverteilung mit Mittelwert Null und Varianz σ_j^2, d. h. jede Komponente modelliert mittelwertfreies Rauschen mit komponentenspezifischer Streuung. Das Modell ist zeitinvariant, d. h. die Gewichte und Standardabweichungen bleiben über alle Zeitschritte konstant. Zum Zeitschritt k ergibt sich somit der gestörte Ausgangswert $u[k]$ aus dem rauschfreien Wert $u^o[k]$ und einem unabhängig aus $p(\eta)$ gezogenen Rauschterm:

$$u[k] \;=\; u^o[k] + \eta_u[k], \qquad \eta_u[k] \;\sim\; p(\eta) \tag{3.8}$$

Die konkret verwendeten Komponentenkonfigurationen für die beiden synthetisierten Eingangssignale sind in Tabelle 3.1 zusammengefasst. Abbildung 3.5 zeigt exemplarisch in Blau den

Tabelle 3.1: Parametrisierung des GMM-Rauschmodells je Eingangssignal

Eingangssignal	**Komponente**	σ_j	ν_j
Eingangssignal $u_1^o[k]$	Grundrauschen	0,02	0,6
	Störimpulse	0,12	0,35
	Ausreißer	0,5	0,05
Eingangssignal $u_2^o[k]$	Grundrauschen	0,003	0,9
	Störimpulse	0,02	0,08
	Ausreißer	0,2	0,02

Verlauf des Signals $u_1^o[k]$ sowie in Grau den der verrauschten Variante $u_1[k]$. Das GMM führt zu zufälligen Streuungen um das Referenzsignal, wobei sowohl im stationären Bereich als auch während des Übergangs vereinzelt größere Ausreißer auftreten.

3.1.5 Systemantwort

Die Systemantwort beschreibt grundsätzlich, wie ein technisches System auf eine Eingangsfolge reagiert und dabei eine messbare Ausgangsgröße erzeugt. Zur Modellierung stehen Ansätze unterschiedlicher Komplexität zur Verfügung. Für den vorliegenden Anwendungsfall wird ein

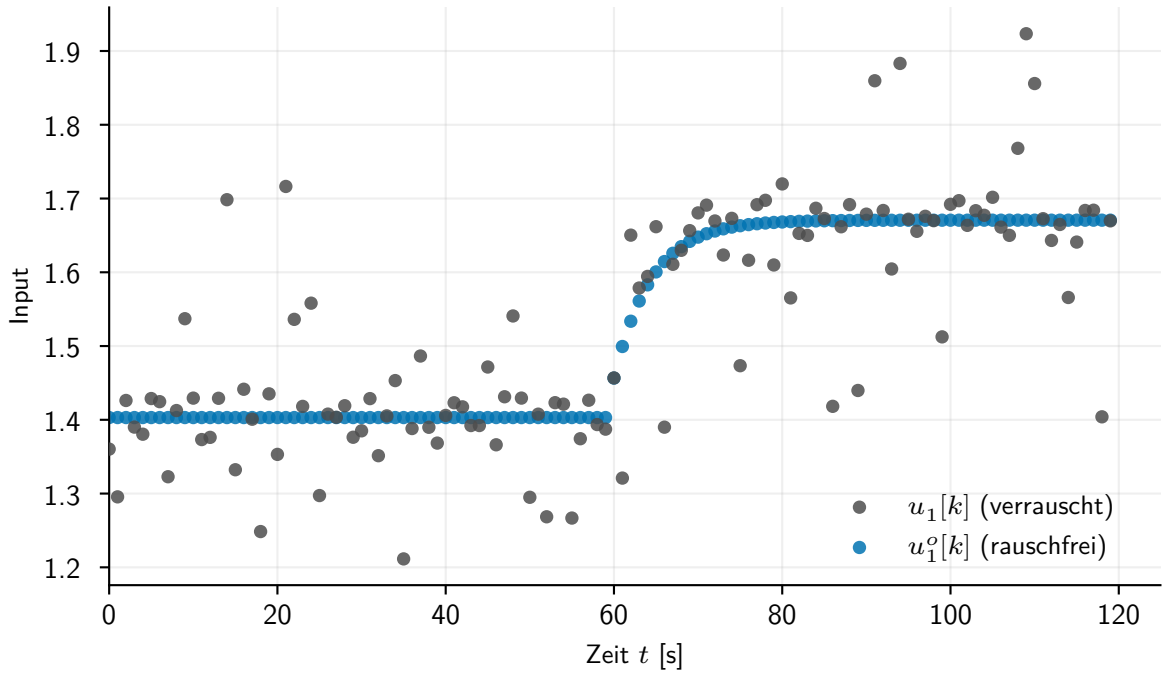


Abbildung 3.5: Zeitverlauf des PT_1-Ausgangssignals $u_1^o[k]$ und des mit gemischtem gaußschen Rauschen überlagerten Signals $u_1[k]$ während eines transienten Übergangs

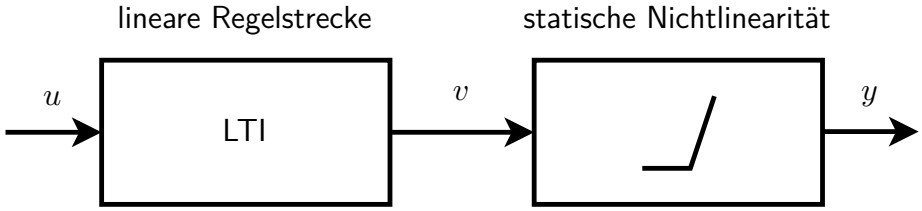


Abbildung 3.6: Struktur des Wiener-Modells

Modell gesucht, das einen guten Kompromiss zwischen Realitätsnähe und Modellkomplexität bietet. Blockorientierte nichtlineare Modelle bieten hierfür einen geeigneten Ansatz, da sie lineare dynamische und statische nichtlineare Blöcke in Serie schalten [GB10]. Das 1930 eingeführte *Hammerstein-Modell* ordnet dabei eine statische Nichtlinearität einem Block eines lineares zeitinvariantes System (Linear Time-Invariant) (LTI) vor. Die Vertauschung dieser Blöcke führt zum 1958 eingeführten *Wiener-Modell*, bei dem zunächst der LTI-Block das Eingangssignal dynamisch formt, bevor die statische Nichtlinearität $g(\cdot)$ wirkt [SL19; Wi58]. Die Struktur des Wiener-Modells ist in Abbildung 3.6 grafisch veranschaulicht.

Das Wiener-Modell kann eine breite Klasse nichtlinearer Systeme approximieren und findet in der Praxis u. a. bei chemischen Prozessen und biologischen Systemen Anwendung [GB10]. Für den vorliegenden Aufbau ist es besonders geeignet, da der Prozess auf die bereits dynamisch geformten Eingangssignale aus Abschnitt 3.1.3 träge und nichtlinear reagieren soll und damit genau der Blockfolge des Wiener-Modells folgt. Im Wiener-Modell werden die Eingangssignale mit der Impulsantwort des LTI-Blocks gefaltet, gewichtet aufsummiert und durch die statische

Nichtlinearität geführt. Formal lässt sich der Ausgang als

$$y^o[k] = g(v^o[k]), \qquad v^o[k] = \sum_{i=1}^{N_{\text{in}}} \alpha_i \cdot (f * u_i^o)[k] \tag{3.9}$$

schreiben, wobei α_i die Gewichtungskoeffizienten, f die Impulsantwort des LTI-Blocks und $v^o[k]$ den rauschfreien Ausgang des linearen Teilsystems bezeichnen.

Als LTI-Block kommt ein weiteres PT_1-Glied gemäß Gleichung (3.5) zum Einsatz, das die Trägheit des Prozesses nachbildet und mit der Prozesszeitkonstante T_y sowie der Verstärkung $K_p = 1$ parametrisiert ist. Die Faltung lässt stationäre Werte damit unverändert und glättet lediglich die Übergänge zwischen Betriebspunkten. Die Prozessdynamik wird als drift-frei angenommen. Als statische Nichtlinearität wird eine linear-kubische Kennlinie $g(v) = v + \beta v^3$ mit $\beta > 0$ gewählt, wie sie beispielsweise progressive Federkennlinien oder Verfestigungsverhalten technischer Systeme abbildet und in der Literatur zum Wiener-Modell etabliert ist. Die Kennlinie ist streng monoton und invertierbar, sodass Drift-bedingte Verschiebungen durchgängig auf die Systemantwort wirken.

Im Fall von CD wirkt der Drift auf zwei Wegen auf die Systemantwort. Zum einen verändert sich durch die zeitvariante Parametrierung der PT_1-Glieder (vgl. Abschnitt 3.1.3) das Verhalten der Eingangssignale. Zum anderen geht das Driftsignal, gewichtet mit dem Driftkoeffizienten κ_y, als zusätzlicher Eingang in das Modell ein und bewirkt eine drift-bedingte Verschiebung des stationären Arbeitspunkts der Ausgangsgröße. Das PT_1-Glied des LTI-Blocks selbst bleibt dabei unverändert, sodass dessen Zeitinvarianz gewahrt bleibt:

$$\tilde{v}^o[k] = \sum_{i=1}^{N_{\text{in}}} \alpha_i \cdot (f * \tilde{u}_i^o)[k] \; + \; \kappa_y \cdot (f * c)[k] \tag{3.10}$$

Die drift-behaftete Systemantwort ergibt sich entsprechend gemäß Gleichung (3.9) als $\tilde{y}^o[k] = g(\tilde{v}^o[k]) = \tilde{v}^o[k] + \beta \left(\tilde{v}^o[k]\right)^3$.

Analog zur Vorgehensweise in Abschnitt 3.1.4 wird die Systemantwort abschließend mit additivem GMM-Rauschen überlagert. Die beobachteten, verrauschten Signale lauten

$$y[k] = y^o[k] + \eta_y[k], \qquad \tilde{y}[k] = \tilde{y}^o[k] + \tilde{\eta}_y[k], \qquad \eta_y[k],\, \tilde{\eta}_y[k] \sim p(\eta)\ , \tag{3.11}$$

wobei $p(\eta)$ die in Gleichung (3.7) definierte Gaussian-Mixture-Dichte bezeichnet.

Abbildung 3.7 zeigt exemplarisch den Verlauf der verrauschten drift-freien Systemantwort $y[k]$ sowie der verrauschten drift-behafteten Systemantwort $\tilde{y}[k]$ in einem Zeitfenster um einen *Sudden Drift*. Die Linearkombination der beiden Eingangssignale erfolgt dabei mit den Gewichtungskoeffizienten $\alpha_1 = \frac{2}{5}$, $\alpha_2 = \frac{1}{3}$ und $\beta = \frac{1}{5}$, der Driftkoeffizient beträgt $\kappa_y = \frac{3}{8}$ und die Zeitkonstante der Prozessdynamik $T_y = 3\,\mathrm{s}$. Die systematische Verschiebung des Ausgangsniveaus, die durch den direkten Drifteinfluss sowie die veränderte PT_1-Dynamik verursacht wird, ist dabei gut zu erkennen.

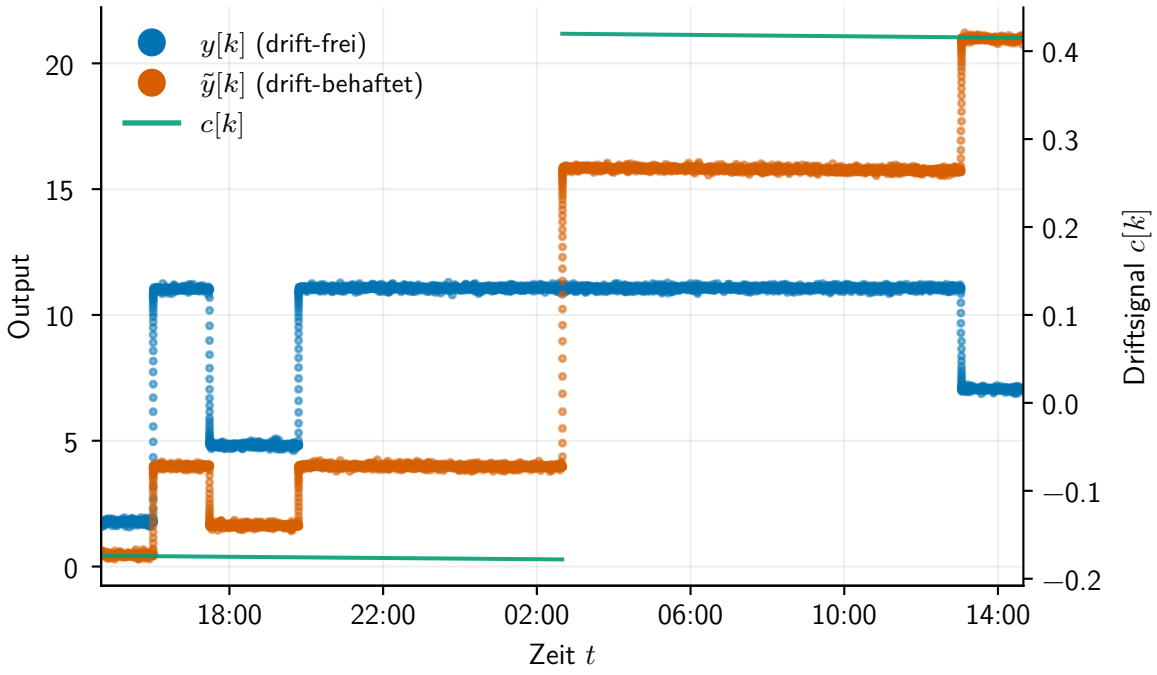


Abbildung 3.7: Vergleich der verrauschten drift-freien und drift-behafteten Systemantworten $y[k]$ und $\tilde{y}[k]$ am Übergang eines *Sudden Drifts*

3.1.6 Modellierung

Nachdem ein industrieller Prozess anhand des synthetischen Datensatzes in den vorigen Abschnitten nachempfunden wurde, soll dieser Prozess nun modelliert werden. Dazu werden drei Modellansätze auf einem drift-freien Referenzdatensatz trainiert bzw. initialisiert und anschließend auf den Datenstrom angewandt. Die Beschränkung des Trainings auf drift-freie Daten entspricht dabei der Anforderung an drift-freie Referenzdaten.

Die Auswahl der Modellansätze fußt auf den in Kapitel 2 eingeführten Architekturen. Für den Mühlen-Anwendungsfall wurde das statische `MLP` in Abschnitt 2.2.4 bereits als geeigneter Kompromiss identifiziert. Mit dem Ziel der Generalisierung dieses Anwendungsbeispiels auf allgemein industrielle Prozesse ist die Beibehaltung dieser Modellarchitektur naheliegend. Ergänzend werden der Kalman-Filter und der Extended Kalman-Filter (vgl. Abschnitt 2.2.2) als Vergleichsverfahren herangezogen. Beide Verfahren zählen zu den klassischen Ansätzen der rekursiven Zustands- und Parameterschätzung linearer bzw. schwach nichtlinearer dynamischer Systeme und finden in der Prozessindustrie weiterhin breite Anwendung [Ka60; WB+95]. Im Gegensatz zum `MLP`, das als Black-Box-Modell ohne strukturelles Vorwissen auskommt, nutzen die Filterverfahren die bekannte Struktur des zugrunde liegenden Systems dabei explizit aus. Der Vergleich erlaubt damit eine Aussage darüber, ob der drift-bedingte Leistungsabfall des Modells ein spezifisches Problem datengetriebener Black-Box-Modelle ist oder auch klassische, strukturbasierte Schätzverfahren betrifft.

Vor dem Training werden die verrauschten Signale mit einem gleitenden Mittelwert über ein

Fenster von $60\,\mathrm{s}$ geglättet, analog zur Glättung der realen Sensordaten in Abschnitt 2.2.4. Die geglätteten Signale werden durch das Superskript f markiert, sodass $u_i^{\mathrm{f}}[k]$ bzw. $\tilde{u}_i^{\mathrm{f}}[k]$ und $y^{\mathrm{f}}[k]$ bzw. $\tilde{y}^{\mathrm{f}}[k]$ die gefilterten Inputs und Outputs darstellen. Als Trainingsdaten dient ein unabhängiger, drift-freier Referenzdatensatz, der analog zur Datenselektion im Mühlen-Anwendungsfall nach dem Latin-Hypercube-Ansatz wie in Abschnitt 2.2.3 beschrieben zur Simulation eines `DOE` erzeugt wird. Die Stellgrößen werden dabei gleichmäßig über ihren Wertebereich gezogen und die quasi-stationären Betriebspunkte gemäß den Abschnitten 3.1.3, 3.1.4 und 3.1.5 simuliert. Beim Modelltraining bilden die drift-freien Eingangssignale $u_i^{\mathrm{f}}[k]$ die Features und die ebenfalls drift-freie Systemantwort $y^{\mathrm{f}}[k]$ das Label. Die Features werden vor dem Training, analog zum Anwendungsfall in Abschnitt 2.2.4, mittels Min-Max-Normierung auf den Wertebereich $[0, 1]$ skaliert.

Das `MLP` folgt der in Abschnitt 2.2.2 eingeführten Feedforward-Struktur und besteht aus zweiHidden-Layern mit jeweils 32 Neuronen und Tanh-Aktivierung sowie einem linearen Ausgabeneuron. Diese bewusst minimale Komplexität ist an die geringe Dimensionalität des synthetischen Systems angepasst und steht im Einklang mit der Erkenntnis aus Abschnitt 2.2.4, dass Netze geringer Komplexität für derartige Regressionsaufgaben ausreichen. Das Training erfolgt mit dem Adam-Optimierer und dem `MSE` als Verlustfunktion über maximal 500 Epochen bei einer Batchgröße von 32. Dabei werden $20\,\%$ der Trainingsdaten zur Validierung zurückgehalten.

Als rekursiver Parameterschätzer wird der Kalman-Filter formuliert. Hierzu wird ausgenutzt, dass die linear-kubische Kennlinie $g(v) = v + \beta v^3$ streng monoton und damit eindeutig invertierbar ist. Die transformierte Messung $z[k] = g^{-1}(y^{\mathrm{f}}[k])$ ist linear in den Gewichtungskoeffizienten der Linearkombination. Angewandt auf die geglätteten Signale ergibt sich mit dem Parametervektor $\theta = [\theta_1, \theta_2]^T$ und dem Regressionsvektor $\varphi[k]$ das lineare Beobachtungsmodell

$$z[k] = g^{-1}(y^{\mathrm{f}}[k]) = \varphi[k]^T \theta[k] + \tilde{\zeta}_{\mathrm{m}}[k], \qquad \varphi[k] = \left[u_1^{\mathrm{f}}[k],\ u_2^{\mathrm{f}}[k]\right]^T , \tag{3.12}$$

wobei $\tilde{\zeta}_{\mathrm{m}}[k] \approx \zeta_{\mathrm{m}}[k] / g'(v[k])$ das durch die Inverse transformierte Messrauschen bezeichnet, das hier in erster Näherung als gaußsch mit konstanter Varianz R angesetzt wird. Das im Datenstrom enthaltene `GMM`-Rauschen wird hier also durch gaußsches Messrauschen approximiert. Als Zustandsübergang wird ein Random-Walk-Modell $\theta[k+1] = \theta[k] + \zeta_{\mathrm{p}}[k]$ mit dem Prozessrauschen $\zeta_{\mathrm{p}}[k] \sim \mathcal{N}(0, Q)$ und dessen Kovarianzmatrix Q angesetzt. Die Initialisierung θ_0 erfolgt durch eine Least-Squares-Schätzung auf den transformierten drift-freien Trainingsdaten. Die Messrauschvarianz R wird als Varianz der zugehörigen Residuen bestimmt.

Der `EKF` schätzt demgegenüber die Gewichtungskoeffizienten der Linearkombination direkt. Mit dem Parametervektor $\theta = [\theta_1, \theta_2]^T$ lautet die nichtlineare Beobachtungsfunktion

$$y^{\mathrm{f}}[k] = h(\theta[k]) + \zeta_{\mathrm{m}}[k], \qquad h(\theta) = v[k] + \beta\, v[k]^3, \quad v[k] = \theta_1\, u_1^{\mathrm{f}}[k] + \theta_2\, u_2^{\mathrm{f}}[k]\ , \tag{3.13}$$

die gemäß Abschnitt 2.2.2 im jeweiligen Arbeitspunkt über die Jacobi-Matrix $H[k] =$

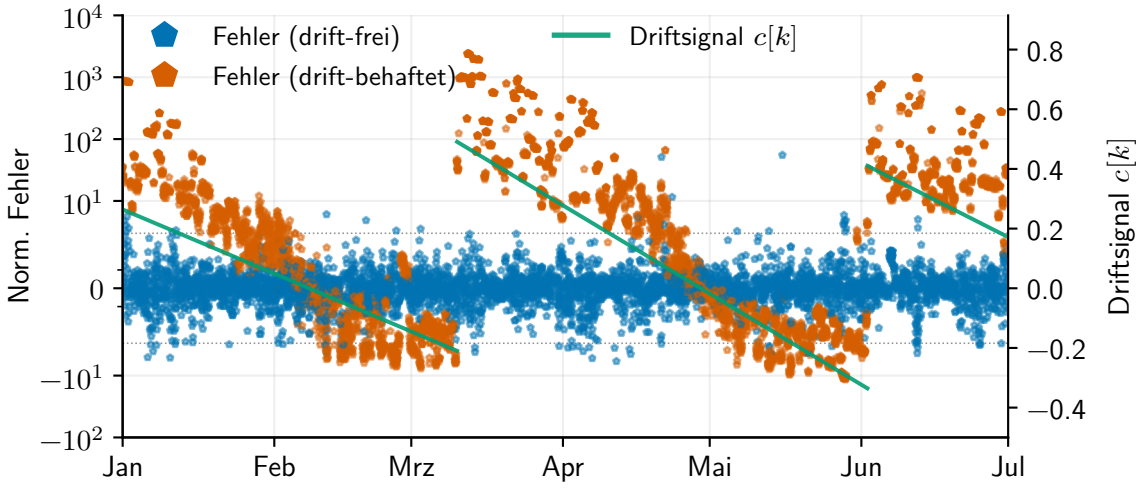
Tabelle 3.2: Initialisierung und Parametrierung der Filterverfahren

Parameter	`KF`	`EKF`
Initialschätzung θ_0	LS-Schätzung auf Trainingsdaten	$[\alpha_1, \alpha_2]^T$
Kovarianz P_0	$10^{-4} \cdot I_2$	$10^{-4} \cdot I_2$
Prozessrauschen Q	$10^{-10} \cdot I_2$	$10^{-10} \cdot I_2$
Messrauschen R	Varianz der Abweichung der LS-Schätzung	Varianz der Abweichung im Ausgangsraum

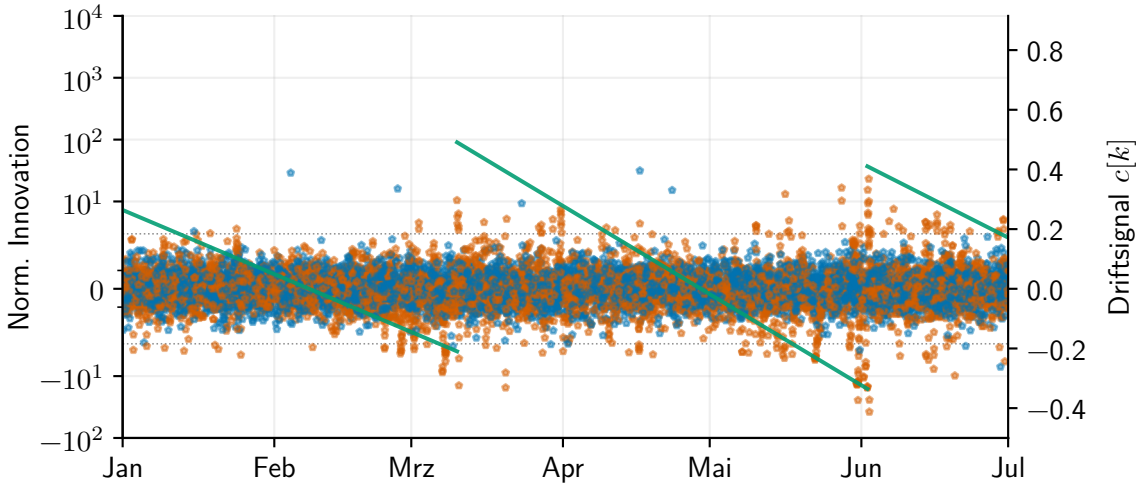
$\partial h / \partial \theta \,|_{\theta[k]}$ linearisiert wird. Der Zustandsübergang folgt analog zum `KF` einem Random-Walk-Modell. Initialisiert wird der `EKF` mit den nominalen Gewichtungskoeffizienten der Simulation $\theta_0 = [\alpha_1, \alpha_2]^T$ (vgl. Abschnitt 3.1.5). Die Messrauschvarianz R wird als Varianz der Abweichung zwischen gemessener Systemantwort und Ausgabe Modells auf den drift-freien Trainingsdaten bestimmt, da die Innovation des `EKF` — anders als die des `KF` — in der ursprünglichen Messgröße y^{f} vorliegt. Die vollständige Parametrierung beider Filterverfahren ist in Tabelle 3.2 zusammengefasst. Die Prozessrauschkovarianz Q wird dabei bewusst sehr klein gewählt, sodass die Parameter nach der Initialisierung nahezu konstant bleiben und beide Filter als quasi-statische Referenzmodelle agieren. Grundsätzlich üben beide Filter über ihren rekursiven Korrekturschritt bereits nativ eine kontinuierliche Adaption aus und besitzen damit neben dem expliziten Systemwissen einen zweiten strukturellen Vorteil gegenüber dem statischen `MLP`. Diese Anpassung wird hier durch die kleine Wahl von Q bewusst unterbunden, um die Filter als faire statische Vergleichsbasis einzusetzen.

Zur Evaluation werden alle drei Modelle auf einen Zeitraum der ersten sechs Monate des Datenstroms angewandt, sowohl auf den drift-freien als auch auf den drift-behafteten Signalen. Als Vergleichsgröße dient beim `MLP` der Simulationsfehler $y^{\mathrm{f}}[k] - \hat{y}[k]$, bei den Filterverfahren die Innovation, also die Abweichung zwischen Messung und Prädiktion vor dem Korrekturschritt. Beide Größen messen, wie gut das jeweilige Modell die beobachtete Systemantwort beschreibt. Um die drei Ansätze in einer gemeinsamen Darstellung gegenüberstellen zu können, werden Simulationsfehler und Innovationen je Modell mit Mittelwert und Standardabweichung des drift-freien Falls normiert. Die Innovation des `KF` liegt dabei im transformierten Messraum $z = g^{-1}(y)$. Durch diese Normierung bleiben die drei Ansätze dennoch vergleichbar.

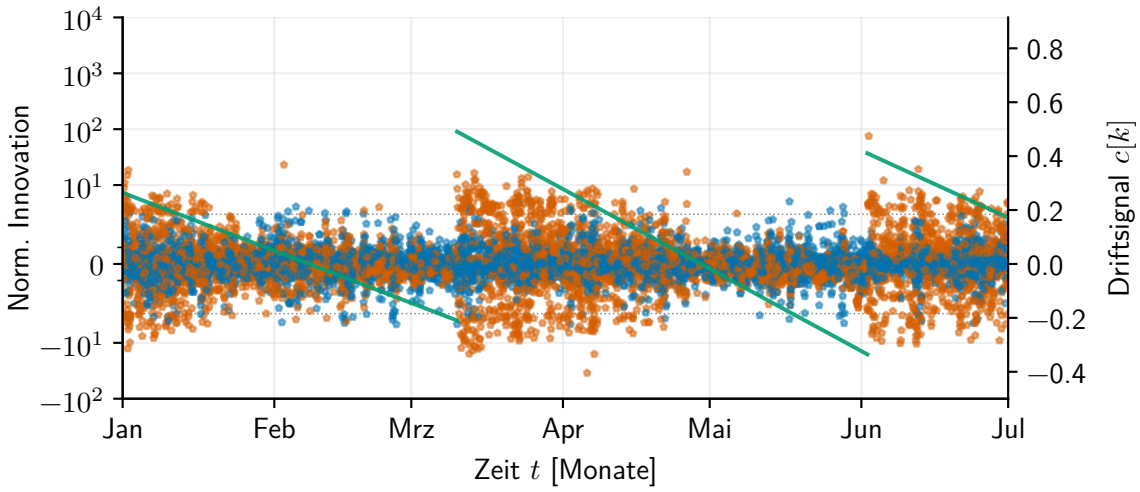
Abbildung 3.8 stellt für das `MLP` (a), den `KF` (b) und den `EKF` (c) den normierten Simulationsfehler bzw. die normierte Innovation auf dem drift-freien (blau) und dem drift-behafteten (orange) Teststrom dar, jeweils überlagert mit dem Driftsignal $c[k]$ (grün). Modellabgeleitete Größen wie Simulationsfehler und Innovation tragen dabei durchgängig fünfeckige Marker, während für die Signale des Prozesses weiterhin Kreise verwendet werden. Die Farbe kennzeichnet damit den Driftzustand und die Markerform die Art der dargestellten Größe. Auf dem drift-freien Datenstrom verbleiben die Fehler aller drei Modelle als mittelwertfreies, strukturloses Band um



(a) Statisches `MLP`



(b) Kalman-Filter



(c) Extended Kalman-Filter

Abbildung 3.8: Normierter Simulationsfehler bzw. normierte Innovationen der drei Modellansätze auf dem drift-freien und dem drift-behafteten Datenstrom im Vergleich zum Driftsignal $c[k]$

Tabelle 3.3: Kennzahlen der drift-bedingten Modellverschlechterung

Modell	RMSE		Streuung σ		Mittelwert $\bar{e}$ (drift-behaftet)
	drift-frei	drift-behaftet	drift-frei	drift-behaftet	
MLP	0,065	15,090	0,064	14,406	4,492
KF	0,009	0,015	0,009	0,015	−0,001
EKF	0,061	0,157	0,061	0,157	−0,001

Null. Dies bestätigt die korrekte, drift-freie Initialisierung. Auf dem drift-behafteten Datenstrom hingegen folgt der Fehlerverlauf bei allen drei Ansätzen der Sägezahnstruktur des Driftsignals. Der *Incremental Drift* innerhalb der Segmente äußert sich in einem mit dem Driftbetrag $|c[k]|$ systematisch anwachsenden Fehlerbetrag, während der *Sudden Drift* an der Segmentgrenze einen sprunghaften Wechsel des Fehlerniveaus verursacht. Dementsprechend korreliert der Betrag des Fehlers im drift-behafteten Fall mit dem Betrag des Driftsignals. Quantitativ bestätigt sich diese Beobachtung: Der Root Mean Squared Error (Wurzel des mittleren quadratischen Fehlers) (RMSE) des Fehlers bzw. der Innovation steigt bei allen drei Modellen vom drift-freien zum drift-behafteten Fall deutlich an (siehe Tabelle 3.3). Die Degradation betrifft damit alle drei Modellklassen. Dass der Anstieg beim MLP betragsmäßig am größten ausfällt, spiegelt dabei keine grundsätzlich schlechtere Modellgüte wider, sondern den Umstand, dass der Drift die Betriebspunkte weit über den drift-freien Trainingsbereich hinaus verschiebt. Das MLP als strukturfreies Black-Box-Modell kann diesen Bereich nicht mehr abbilden, während KF und EKF die bekannte Systemstruktur explizit ausnutzen und den Verlauf dadurch weiter fortschreiben.

Worin sich die drei Ansätze hingegen unterscheiden, ist die Zusammensetzung dieses Fehleranstiegs, die sich anhand der visuellen Darstellung sowie der Kennzahlen in Tabelle 3.3 erkennen lässt. Beim MLP überlagern sich zwei Effekte. Der Fehler im drift-behafteten Fall nimmt einen deutlich von Null verschiedenen Mittelwert $\bar{e}$ an, der dem Drift in einer Vorzugsrichtung folgt, den größten Beitrag zum Fehleranstieg liefert aber die anwachsende Streuung σ, sobald die Betriebspunkte den Trainingsbereich verlassen. Bei KF und EKF bleibt die Innovation demgegenüber nahezu mittelwertfrei. Die Verschlechterung äußert sich hier überwiegend in einer wachsenden, zeitlich aber ungerichteten Streuung σ. Beim MLP verschiebt sich der Fehler mit dem Drift und verbreitert sich zugleich, während bei den Filterverfahren nicht die Lage der Innovation, sondern nur deren Streuung mit $c[k]$ zunimmt. Die Schwere des Drifts schlägt sich bei den Filtern somit in der Breite und nicht in der Lage der Innovationsverteilung nieder.

Erwartungsgemäß können auch die Filterverfahren den Drift mit ihrer quasi-statischen Parametrierung nicht kompensieren. Der drift-bedingte Leistungsabfall des Modells ist somit kein spezifisches Defizit datengetriebener Black-Box-Modelle, sondern eine grundsätzliche Eigen-

schaft statischer Modelle unter CD. Daraus kann gefolgert werden, dass der synthetische Datenstrom die beabsichtigte Degradation reproduziert und sich als kontrollierte Testumgebung für die Evaluation von Detektions- und Adaptionsmechanismen eignet.

3.1.7 Drift-Erkennung

Zur Detektion des in Abschnitt 3.1.1 generierten CD werden die in Abschnitt 2.3.2 eingeführten Performance-basierten Drift-Detektoren herangezogen. Als Eingangssignal der Detektion dient der normierte Simulationsfehler des in Abschnitt 3.1.6 eingeführten MLP, dessen Architektur auch im ausgehenden Anwendungsfall der Mühle verwendet wird. Konkret kommen als Detektionsverfahren die beiden Fenster-basierten Verfahren KSWIN und ADWIN, die beiden auf statistischer Prozesskontrolle beruhenden Verfahren DDM und EDDM sowie ein Schwellwert-Detektor zum Einsatz. Letzterer bildet die Performance-basierte Erkennung in ihrer einfachsten Form ab, indem er anschlägt, sobald der über ein gleitendes Fenster berechnete RMSE des Simulationsfehlers einen Schwellwert überschreitet. Er kommt damit ohne statistischen Test und ohne Warnstufe aus und dient als Referenz, an der sich der Mehraufwand der übrigen vier Verfahren messen lassen muss.

Wie bereits in Abschnitt 2.3.2 beschrieben, verarbeiten die Fenster-basierten Verfahren eine kontinuierliche Größe und verwenden daher hier, ebenso wie der Schwellwert-Detektor, den Betrag des Simulationsfehlers $|e[k]|$. DDM und EDDM hingegen basieren auf Fehlerraten und sind ursprünglich auf binäre Fehlersignale ausgelegt, wie sie bei Klassifikationsaufgaben durch korrekte oder falsche Modellentscheidungen entstehen. Ein solches binäres Fehlersignal existiert für die vorliegende Regressionsaufgabe jedoch nicht, sodass der kontinuierliche Fehler vor der Auswertung binarisiert werden muss. Dazu wird ein Schwellenwert λ_{bin} eingeführt, der den Fehler in einen korrekten ($|e[k]| \leq \lambda_{\mathrm{bin}}$) und einen fehlerhaften Zustand ($|e[k]| > \lambda_{\mathrm{bin}}$) übersetzt. Da der Fehler in Abschnitt 3.1.6 mit der Standardabweichung des drift-freien Falls normiert wurde, entspricht der Schwellenwert einem Vielfachen dieser Standardabweichung.

Ein wesentlicher Unterschied zum Mühlen-Anwendungsfall besteht darin, dass der synthetische Datenstrom die Drift-Zeitpunkte als *Ground Truth* bereitstellt. Aus $c[k]$ werden dazu zwei Referenzgrößen abgeleitet: die Menge der *Sudden Drifts* als Indizes der sprungartigen Anstiege an den Segmentgrenzen sowie eine Unterscheidung zwischen drift-aktiven und drift-inaktiven Phasen. Obwohl sich $c[k]$ kontinuierlich verändert, gilt ein Zeitschritt als drift-inaktiv (bzw. *stabil*), wenn die Driftamplitude unter $10\,\%$ ihres Maximalwerts bleibt:

$$|c[k]| < 0{,}1 \cdot \hat{c}, \qquad \hat{c} = \max_k |c[k]| \,, \tag{3.14}$$

andernfalls gilt der Zeitschritt als drift-aktiv. Auf dieser Grundlage wird die Detektionsgüte über drei Kennzahlen erfasst:

- der *Recall* ρ der Sudden Drifts, also der Anteil der erkannten Sudden Drifts innerhalb

eines Toleranzfensters von $n_{\mathrm{tol}} = 1000$ Abtastschritten bzw. 16,7 Stunden

- der mittlere *Detektionsverzug* $\bar{d}$ zwischen Sprung und zugehöriger Detektion
- die Zahl N_{fa} der *Fehlalarme*
  Der weit überwiegende Teil des Stroms ist nach (3.14) drift-behaftet. Gezählt wird daher jede Detektion, die keinem Sudden Drift zugeordnet werden kann. Das Toleranzfenster umfasst dabei $0{,}92\,\%$ des mittleren Abstands zweier Sudden Drifts.

Diese drei Kennzahlen werden zu einer skalaren Zielfunktion

$$J_{\mathrm{d}} = \omega_\rho\,(1-\rho) + \omega_d\,\frac{\bar{d}}{n_{\mathrm{tol}}} + \omega_{\mathrm{fa}}\,\min\!\left(1,\ \frac{\log_{10}(1+N_{\mathrm{fa}}/N_{\mathrm{sd}})}{D}\right) \tag{3.15}$$

zusammengefasst, wobei N_{sd} die Zahl der Sudden Drifts bezeichnet. Die Gewichte werden mit $\omega_\rho = 3{,}0$, $\omega_d = 0{,}3$ und $\omega_{\mathrm{fa}} = 1{,}5$ so gewählt, dass eine vollständige Detektion der Sudden Drifts gegenüber Verzug und Fehlalarmen priorisiert wird. Der Fehlalarmterm ist logarithmisch gestaltet und über $D = 3$ so gestreckt, dass er zwischen einigen wenigen und einigen tausend Fehlalarmen den gesamten Wertebereich abdeckt.

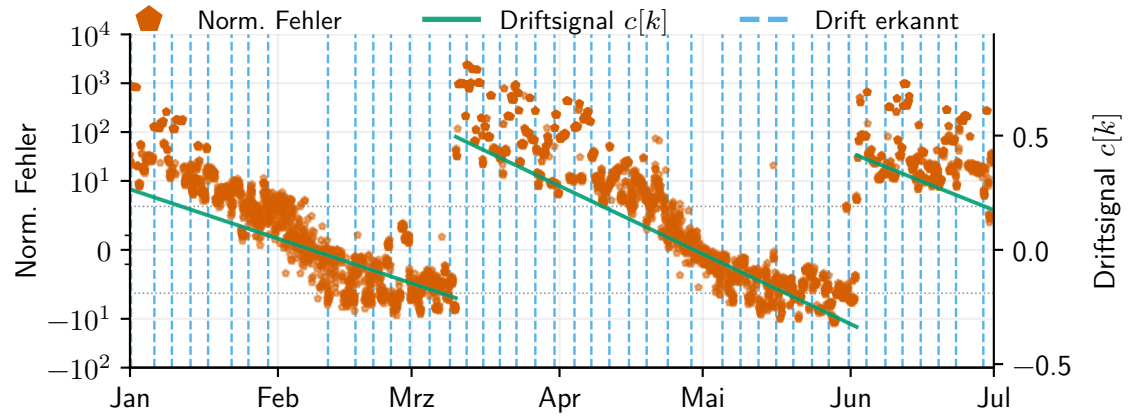
Der Recall allein ist als Gütemaß nicht aussagekräftig, solange die Zahl der Detektionen nicht mitbetrachtet wird. Ein Detektor, der hinreichend häufig meldet, erreicht bereits durch Zufall einen hohen Recall, ohne dass seine Meldungen mit Sprüngen zusammenhängen. Als Bezugsgröße dient deshalb das Zufallsniveau, also der Recall, den eine gleichverteilt zufällige Folge von N_{det} Detektionen auf einem Datenstrom der Länge N im Erwartungswert erreicht,

$$\rho_{\mathrm{zuf}} = 1 - \exp\!\left(-\frac{N_{\mathrm{det}}\,n_{\mathrm{tol}}}{N}\right)\ . \tag{3.16}$$

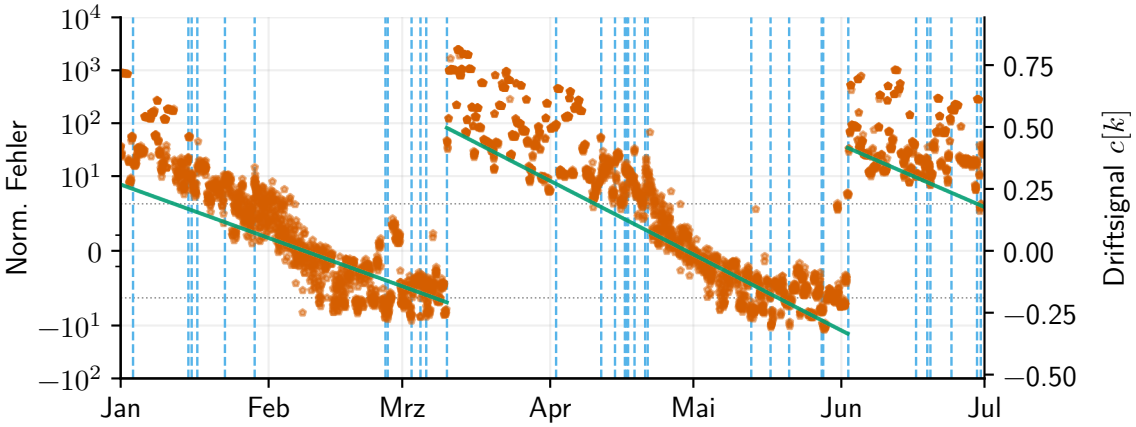
Der Abstand zwischen Recall und Zufallsniveau zeigt damit an, ob eine Detektion tatsächlich Informationen über den Zeitpunkt eines Sprungs trägt. Das Zufallsniveau geht nicht in die Zielfunktion ein, sondern dient als Einordnung der Ergebnisse.

Die Parameter der fünf Detektoren werden mit dem Optimierungsverfahren `Optuna` gegen J_{d} eingestellt. Um den Rechenaufwand zu begrenzen, wird der Datenstrom dafür um den Faktor 5 ausgedünnt, wobei das Toleranzfenster im gleichen Verhältnis verkürzt wird. Die Schwelle λ_{bin} von `DDM` und `EDDM` wird dabei logarithmisch abgetastet, da ihr Suchbereich mehrere Größenordnungen umfasst. Die so gefundenen Kandidaten sind somit auf einem verkürzten Datenstrom bewertet. Daher werden die 5 besten Kandidaten je Detektor anschließend auf dem vollständigen Datenstrom nachgerechnet. Diese Nachrechnung entscheidet über die endgültige Parametrierung.

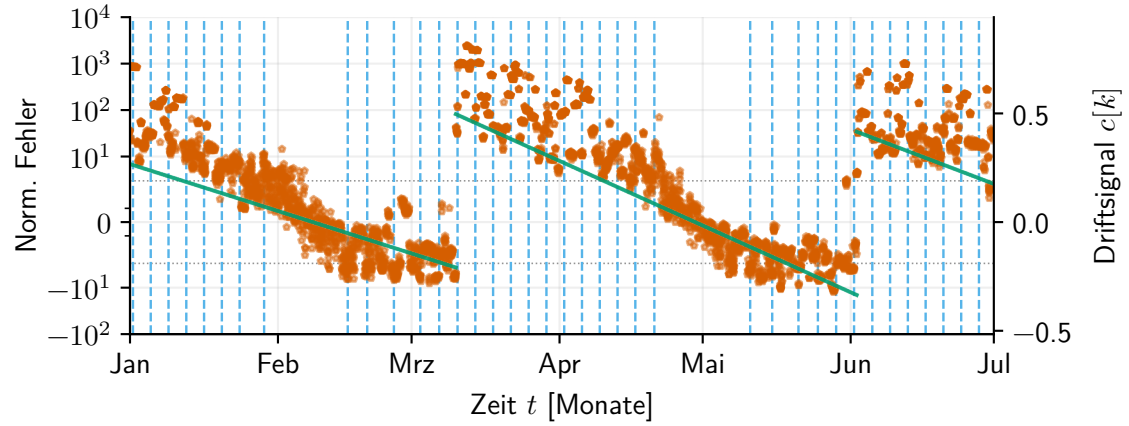
Abbildung 3.9 stellt die erkannten Drifts mit den Parameter-optimierten Detektoren dem Driftsignal $c[k]$ gegenüber. Zu sehen ist dabei `KSWIN` als Fenster-basiertes Verfahren (a), `DDM` als Fehlerraten-basiertes Verfahren (b) sowie der Schwellwert-Detektor (c). `ADWIN` und `EDDM` verhalten sich qualitativ wie ihre Pendants derselben Detektor-Klasse. Die zugehörigen Kennzahlen fasst Tabelle 3.4 zusammen. Ausgewertet wird der gesamte Datenstrom über ein Kalenderjahr,



(a) `KSWIN`



(b) `DDM`



(c) Schwellwert-Detektor

Abbildung 3.9: Erkannte Drifts ausgewählter Detektionsmethoden auf dem normierten Simulationsfehler im Vergleich zum Driftsignal $c[k]$

der an seinen Segmentgrenzen vier Sudden Drifts aufweist (vgl. Abbildung 3.2). Die Abbildung zeigt davon exemplarisch die ersten sechs Monate. Eine Eigenschaft der Fenster-basierten Verfahren und des Schwellwert-Detektors ist, dass sie den fortlaufenden *Incremental Drift* innerhalb eines Segments nicht als einmaliges Ereignis, sondern kontinuierlich als wiederholte Verteilungsänderung detektieren. Dies führt zu einer dichten Folge von Detektionen über die gesamte aktive Driftphase. Für die grafische Darstellung wird daher für diese drei Verfahren eine Refraktärzeit von 83 Stunden eingeführt, innerhalb derer nach einer Detektion weitere Detektionen unterdrückt werden. `DDM` und `EDDM` bleiben davon unberührt. Die in Tabelle 3.4 dargestellten Kennzahlen sind jedoch auf der vollständigen Detektionsfolge ohne Refraktärzeit berechnet.

Auswertend lässt sich festhalten, dass `KSWIN` und `ADWIN` sämtliche Segmentsprünge zuverlässig und mit geringem Verzug erkennen (vgl. Tabelle 3.4). Zugleich zeichnen sie den fortlaufenden Incremental Drift nach, was sich ohne Refraktärzeit in einer Vielzahl von Einzeldetektionen äußert. Mit Refraktärzeit reduziert sich diese auf wenige Detektionen, die die aktiven Drifts zuverlässig markieren. Diese hohe Sensitivität geht jedoch mit vergleichsweise vielen Fehlalarmen in stabilen Phasen gemäß (3.14) einher.

`DDM` und `EDDM` verhalten sich demgegenüber deutlich konservativer. Beide erfassen sämtliche Segmentsprünge, markieren den fortlaufenden Incremental Drift jedoch deutlich spärlicher und melden über den gesamten Strom um ein Vielfaches seltener. Dieses Verhalten ist eine unmittelbare Folge der Binarisierung. Da nur Fehlerbeträge oberhalb von λ_{bin} einfließen, reagieren beide Verfahren vorrangig auf ausgeprägte Fehlerverläufe und bleiben gegenüber schwächeren Änderungen vergleichsweise unempfindlich. Ihre sparsame Meldung geht im Vergleich zu den Fenster-basierten Verfahren mit einem höheren Detektionsverzug einher.

Der Schwellwert-Detektor ordnet sich zwischen beiden Klassen ein. Er reagiert wie die Fenster-basierten Verfahren unmittelbar auf den steigenden Fehlerbetrag und meldet daher über die aktiven Driftphasen hinweg wiederholt, bleibt dabei aber deutlich sparsamer, weil sein gleitendes Fenster erst bei anhaltend erhöhtem Fehler anschlägt. In den stabilen Phasen gemäß (3.14) meldet er, wie `DDM`, überhaupt nicht, `EDDM` nur vereinzelt. Entsprechend weist er den höchsten Detektionsverzug der fünf Verfahren auf.

Tabelle 3.4 stellt für jeden der fünf Detektoren den erreichten Recall, das zugehörige Zufallsniveau, den mittleren Verzug sowie die Zahl der Fehlalarme und der Detektionen insgesamt gegenüber. Alle fünf Verfahren erkennen nach der Parameteroptimierung sämtliche Sudden Drifts, sodass die Unterscheidung allein über die übrigen Kennzahlen erfolgt. Die Fenster-basierten Verfahren `KSWIN` und `ADWIN` melden dabei über den gesamten Datenstrom hinweg um ein Vielfaches häufiger als die Fehlerraten-basierten Verfahren, wodurch ihr Zufallsniveau nahe an den erreichten Recall heranrückt. Sie erreichen ihren vollständigen Recall damit im Wesentlichen durch die schiere Zahl ihrer Detektionen. `DDM` und `EDDM` liegen demgegenüber mit ihrem Zufallsniveau weit darunter. Der Schwellwert-Detektor liegt in seiner Meldungsdich-

Tabelle 3.4: Kennzahlen der fünf Drift-Detektoren auf dem normierten Simulationsfehler des drift-behafteten Stroms

Detektor	Recall ρ	Zufallsniveau ρ_{zuf}	Verzug $\bar{d}$ in h	Fehlalarme N_{fa}	Detektionen
`KSWIN`	1,00	0,70	2,5	614	626
`ADWIN`	1,00	0,88	1,0	1099	1115
`DDM`	1,00	0,12	3,4	61	66
`EDDM`	1,00	0,27	3,1	159	164
Schwellwert	1,00	0,46	4,4	319	324

te zwischen den beiden Klassen, und sein Abstand zum Zufallsniveau bleibt deutlich größer als der der Fenster-basierten Verfahren. Seine Meldungen kommen also seltener zufällig zustande, obwohl er ohne statistischen Test auskommt.

Welches Verhalten vorzuziehen ist, hängt von der angestrebten Adaptionsstrategie ab. Eine Adaption, die auf jede Meldung hin nachtrainiert, ist auf eine sparsame und zeitlich aussagekräftige Detektion angewiesen, da jede überflüssige Meldung Rechenzeit bindet und das Modell ohne Anlass verändert. Unter diesem Gesichtspunkt sind `DDM`, `EDDM` und der Schwellwert-Detektor vorzuziehen, deren Meldungen deutlich seltener zufällig zustande kommen. Wird die Detektion dagegen nur als ergänzendes Signal neben einem festen Nachtrainingsraster verwendet, fällt der höhere Verzug dieser drei Verfahren stärker ins Gewicht als deren Sparsamkeit.

3.1.8 Drift-Adaption

Nachdem der `CD` im vorigen Abschnitt detektiert werden konnte, besteht der abschließende Schritt der Verarbeitungskette darin, das in Abschnitt 3.1.6 eingeführte `MLP` an den drift-behafteten Prozess zu adaptieren und damit der Modellverschlechterung entgegenzuwirken. Methodisch werden drei Strategien dem drift-behafteten Modell gegenübergestellt, die sich darin unterscheiden, wann nachtrainiert wird.

- eine *passive* Adaption
- eine *aktive* Adaption
- eine *Kombination* aus passiver und aktiver Adaption

Unabhängig davon, wann eine Strategie nachtrainiert, lässt sich festlegen, mit welchen Daten dies geschieht. Ergänzend wird daher für alle Strategien ein *Recurrence-Selektor* untersucht, der das Trainingsfenster um passende zurückliegende Datenpunkte erweitert. Die Strategien und Selektor werden im Folgenden zunächst definiert und anschließend methodisch ausgestaltet.

Allen adaptiven Strategien liegt ein inkrementelles Nachtraining des `MLP` zugrunde. Anstatt das Modell vollständig neu zu trainieren, wird das bestehende Netz schrittweise an die veränderten Zusammenhänge angepasst. Welche Layer des Netzes dabei eingefroren werden, ist Teil der Hyperparameteroptimierung. Die übrigen Schichten werden mit kleiner Lernrate über wenige Epochen mit dem Adam-Optimierer und dem `MSE` als Verlustfunktion nachtrainiert. Ein Early-Stopping-Kriterium bricht das Training bei ausbleibender Verbesserung vorzeitig ab und begrenzt so das Risiko eines Overfitting an einen kurzen, lokalen Datenausschnitt. Zusätzlich wird jedes Nachtraining validiert: Senkt es den `MSE` auf dem eigenen Trainingsfenster nicht, wird das Update verworfen und der vorherige Gewichtsstand wiederhergestellt. Das Einfrieren von Layern, die kleine Lernrate sowie das Early-Stopping wirken dabei dem in Abschnitt 2.3.3 beschriebenen *Stability-Plasticity-Dilemma* entgegen, indem sie die Plastizität gezielt begrenzen. Im Gegensatz zum initialen Training auf dem drift-freien Referenzdatensatz (vgl. Abschnitt 3.1.6) werden für die Adaption die drift-behafteten Eingangssignale $\tilde{u}_i^{\mathrm{f}}[k]$ als Features und die ebenfalls drift-behaftete Systemantwort $\tilde{y}^{\mathrm{f}}[k]$ als Label verwendet. Die kontinuierliche Verfügbarkeit des Labels entspricht dabei dem Mühlen-Anwendungsfall, in dem die Zielgröße sensorisch fortlaufend erfasst wird.

Zur Adaption wird der Strom auf ein gröberes Abtastraster mit dem Adaptionsintervall $T_{\mathrm{a}} = 600\,\mathrm{s}$ reduziert, was $52\,560$ Datenpunkten entspricht, und anschließend chunk-weise verarbeitet. Ein *Chunk* bezeichnet in dem Zusammenhang einen zusammenhängenden Abschnitt des Datenstroms fester Länge – hier etwa einen Tag bzw. 144 Datenpunkte – und bildet die Einheit, auf der über eine Anpassung entschieden wird. Der Strom wird somit nicht punktweise, sondern blockweise in solchen Chunks durchlaufen. Innerhalb eines Chunks wird der Strom zunächst in einem Durchgang prädiziert und daraus der Simulationsfehler bestimmt. Anschließend werden die Strategie-spezifischen Bedingungen geprüft und nur bei Bedarf das Nachtraining ausgelöst. Die Verarbeitung folgt durchgängig dem prequentiellen Prinzip (*test-then-train*). Jeder Datenpunkt wird dabei zunächst prädiziert und fließt erst danach in ein etwaiges Nachtraining ein. Trainingsfenster greifen hier zu keinem Zeitpunkt auf zukünftige Datenpunkte zu.

Die *passive* Adaption setzt die in Abschnitt 2.3.3 eingeführte *blinde* Anpassung um. Sie ist rein proaktiv und benötigt weder Fehlerauswertung noch Detektor. In festen Abständen von n_{p} Punkten wird das Modell auf dem gleitenden Fenster der letzten n_{w} Punkte nachtrainiert und so eine kontinuierliche Grundadaption bereitgestellt, deren Eingriffsdichte allein über die Periode n_{p} gesteuert wird.

Die *aktive* Adaption koppelt das Nachtraining hingegen an die Drift-Detektion aus Abschnitt 3.1.7. Erst ein erkannter Drift löst eine Anpassung aus, die dadurch zielgerichtet, zugleich aber von der Detektionsqualität abhängig ist.

Die *kombinierte* Strategie soll beide Mechanismen zusammenführen, wobei die periodische passive Komponente als Rückfallebene (*Backstop*) wirkt. Sie löst nur aus, wenn im selben Zeitraum keine detektor-getriggerte Anpassung erfolgt ist. Verpasst der Detektor eine Veränderung, fängt

die periodische Komponente sie auf.

Losgelöst von diesen Strategien steht der *Recurrence-Selektor*. Er verändert nicht den Zeitpunkt eines Nachtrainings, sondern die Zusammensetzung des Trainingsfensters. Neben den Punkten des jeweiligen Fensters werden zurückliegende Punkte aus einem Rückblickhorizont n_{h} hinzugezogen, deren Fehler unter dem aktuellen Modell höchstens um λ_{rec} vom mittleren Fehler des Fensters abweicht, höchstens jedoch N_{rec} Stück. Der Selektor greift damit den in Abschnitt 2.3.1 beschriebenen *Recurring Drift* auf, indem er einem wiederkehrenden Betriebszustand die bereits gesehenen Daten desselben Zustands zurückgibt, statt sie mit dem gleitenden Fenster zu verwerfen. Die Parameter des Selektors werden für jede Strategie eigens abgestimmt, während deren übrige Parameter unverändert bleiben. Ein Unterschied zur jeweiligen Strategie ohne Selektor geht damit allein auf die Auswahl der Trainingsdaten zurück.

Da die aktive Strategie von der Wahl des Detektors abhängt, wird dieser nicht manuell festgelegt, sondern anhand eines Screenings bestimmt. Als Trigger stehen die fünf in Abschnitt 3.1.7 untersuchten Detektoren zur Auswahl. Für jeden von ihnen werden die Hyperparameter mit reduziertem Budget abgestimmt und die gefundene Konfiguration anschließend auf dem voll aufgelösten Strom über mehrere Zufalls-Seeds nachgerechnet, da die Abstände zwischen den Triggern kleiner ausfallen als die Streuung eines einzelnen Triggers. Dabei gelten alle Trigger als gleichwertig, deren Abstand zum besten in einem gepaarten t-Test nicht signifikant ist und höchstens $5\,\%$ beträgt. Bevorzugt wird dann das Verfahren mit den wenigsten Annahmen, also der geringsten Komplexität. Der so bestimmte Trigger wird mit vollem Budget nachgetunt und als aktive Komponente auch der kombinierten Strategie übernommen. Im vorliegenden Fall führt der Schwellwert-Detektor das Screening an, ohne dass ein anderes Verfahren als gleichwertig gilt, und kommt damit als Trigger zum Einsatz. Die Anforderungen an den Detektor unterscheiden sich dabei von der reinen Detektions-Evaluation aus Abschnitt 3.1.7, denn im geschlossenen Kreis arbeitet er auf dem Simulationsfehler $e[k]$ des sich fortlaufend ändernden Modells, dessen Niveau sich mit jeder Anpassung verschiebt. Fehlerraten-basierte Verfahren wie `DDM` oder `EDDM` setzen dazu den Betrag des Simulationsfehlers $|e[k]|$ über die in Abschnitt 3.1.7 eingeführte Schwelle λ_{bin} in ein binäres Richtig/Falsch-Signal um. Diese Schwelle ist am Fehlerniveau des statischen Modells festgelegt. Sobald die laufende Adaption dieses Niveau absenkt, passt sie nicht mehr, und der Detektor arbeitet außerhalb seines vorgesehenen Betriebspunkts. Dasselbe gilt für den Schwellwert-Detektor, dessen Schwelle unmittelbar auf dem Fehlerniveau liegt. Die Fenster-basierten Verfahren `KSWIN` und `ADWIN` vergleichen dagegen ein aktuelles Detektionsfenster mit einem mitlaufenden Referenzfenster und kommen ohne fest kalibrierten Schwellwert aus, sodass sie vom absoluten Fehlerniveau kaum betroffen sind. Um für alle Verfahren einen passenden Betriebspunkt zu erhalten, werden die Detektorparameter nicht aus der reinen Detektion übernommen, sondern im geschlossenen Adaptionskreis neu abgestimmt.

Wird bei der aktiven Strategie zum Zeitschritt k_{d} ein Drift erkannt, erfolgt unmittelbar ein Nachtraining auf dem zurückliegenden Fenster $[\,k_{\mathrm{d}} - n_{_},\, k_{\mathrm{d}}\,]$. Würde bis zum Nachtraining

zunächst n_+ Punkte abgewartet um dem Nachtraining mehr Daten zur Verfügung zu stellen, entstünde eine Totzeit, in der der bereits detektierte Drift unberücksichtigt bliebe. Stattdessen wird das Modell während einer Nachführphase der Länge n_+ nachgeführt, das heißt am Ende jedes Chunks durch ein kurzes inkrementelles Nachtraining auf den zuletzt eingetroffenen Datenpunkten an die fortschreitende Veränderung angepasst. Detektionen innerhalb einer laufenden Phase bleiben ohne Wirkung und lösen insbesondere kein weiteres Sofort-Training aus. Der Eingriffsaufwand ist damit von der Detektionsdichte entkoppelt. Auch ein fenster-basiertes Verfahren, das den Incremental Drift als dichte Folge von Einzeldetektionen nachzeichnet (vgl. Abschnitt 3.1.7), wirkt so als Ereignis-Trigger.

Zwischen zwei Sofort-Trainings wird zudem eine Refraktärzeit als Vielfaches der mittleren Dauer stationärer Betriebsphasen erzwungen, die den Eingriffsaufwand begrenzt.

Die Bewertung der Strategien erfolgt über den Simulationsfehler auf dem drift-behafteten Strom über das gesamte Kalenderjahr. Als Kennzahlen dienen der `RMSE`, der Mittelwert $\bar{e}$ und die Streuung σ des Fehlers, die Korrelation des Fehlerbetrags $|e|$ mit dem Driftbetrag $|c[k]|$ sowie die Anzahl der ausgelösten Nachtrainings.

Letztere quantifiziert den Eingriffsaufwand der jeweiligen Strategie. Da das inkrementelle Nachtraining seine Trainingsdaten in zufälliger Reihenfolge durchläuft, endet es auch bei identischen Daten auf leicht unterschiedlichen Gewichten. Die Auswertung wird daher über 50 unabhängige Läufe mit unterschiedlichen Zufalls-Seeds wiederholt und jede Kennzahl als Mittelwert samt Streuung über diese Läufe zusammengefasst. Die Streuung über die Seeds dient dabei als Maß für die Reproduzierbarkeit einer Strategie.

Die Wirksamkeit der adaptiven Strategien hängt maßgeblich von ihren Hyperparametern ab. Diese werden daher analog zur Optimierung der Detektoren in Abschnitt 3.1.7 je Strategie mit dem Optimierungsverfahren `Optuna` eingestellt. Im Unterschied zur Detektion erfordert jeder Versuch hier einen vollständigen Adaptionslauf mit wiederholtem Nachtraining und ist damit rechenintensiver. Das Tuning erfolgt dabei auf einem gröber abgetasteten Strom mit $T_{\mathrm{a}} = 3600\,\mathrm{s}$. Anders als bei der Detektion wird keine *Ground Truth* der Drifts benötigt, da die gemessene Systemantwort als Label vorliegt und der `RMSE` direkt bewertet werden kann. Die Zielfunktion kombiniert den auf den Fall ohne Adaption normierten `RMSE` mit einem Strafterm für die Zahl der Nachtrainings, um den Eingriffsaufwand zu begrenzen:

$$J_{\mathrm{a}} = \omega_{\mathrm{r}} \frac{\mathrm{RMSE}}{\mathrm{RMSE}_{\mathrm{base}}} + \omega_{\mathrm{t}} \frac{N_{\mathrm{train}}}{N_{\mathrm{chunk}}} \,, \tag{3.17}$$

mit den Gewichten $\omega_{\mathrm{r}} = 1{,}0$ und $\omega_{\mathrm{t}} = 0{,}1$, wobei $\mathrm{RMSE}_{\mathrm{base}}$ den `RMSE` der Baseline, N_{train} die Zahl der Nachtrainings und N_{chunk} die Zahl der Chunks bezeichnet.

Optimiert werden die Lernrate, die Epochenzahl, das Freeze-Muster, das Reset-Verhalten, die strategie-spezifischen Fenster- und Schwellenparameter sowie die Refraktärzeit. Drei Maßnahmen sichern die Übertragbarkeit der auf dem gröber abgetasteten Tuning-Strom gefundenen

Tabelle 3.5: Kennzahlen der fünf Adaptionsstrategien auf dem drift-behafteten Strom, als Mittelwert ± Standardabweichung über 50 Seeds

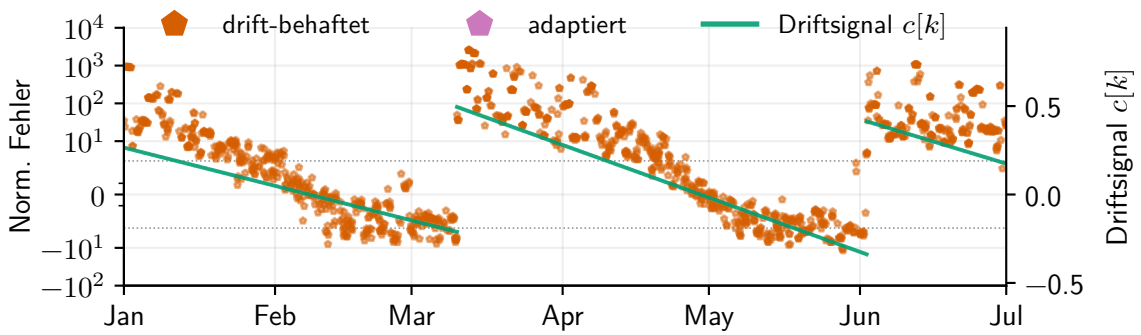
Strategie	RMSE	Mittelwert $\bar{e}$	Streuung σ	Korr. $\lvert e\rvert, \lvert c\rvert$	Nachtrainings
Baseline	$1{,}663 \pm 0{,}000$	$0{,}401 \pm 0{,}000$	$1{,}614 \pm 0{,}000$	$0{,}37 \pm 0{,}00$	$0{,}0 \pm 0{,}0$
Passiv	$0{,}514 \pm 0{,}054$	$0{,}006 \pm 0{,}006$	$0{,}514 \pm 0{,}054$	$0{,}17 \pm 0{,}01$	$365{,}0 \pm 0{,}0$
Passiv + Rec	$0{,}451 \pm 0{,}018$	$0{,}006 \pm 0{,}003$	$0{,}451 \pm 0{,}018$	$0{,}16 \pm 0{,}01$	$365{,}0 \pm 0{,}0$
Aktiv	$0{,}601 \pm 0{,}012$	$0{,}013 \pm 0{,}007$	$0{,}601 \pm 0{,}012$	$0{,}17 \pm 0{,}01$	$127{,}2 \pm 4{,}4$
Kombiniert	$0{,}604 \pm 0{,}008$	$0{,}019 \pm 0{,}005$	$0{,}604 \pm 0{,}008$	$0{,}18 \pm 0{,}01$	$198{,}3 \pm 3{,}8$

Konfigurationen auf die Zielauflösung ab:

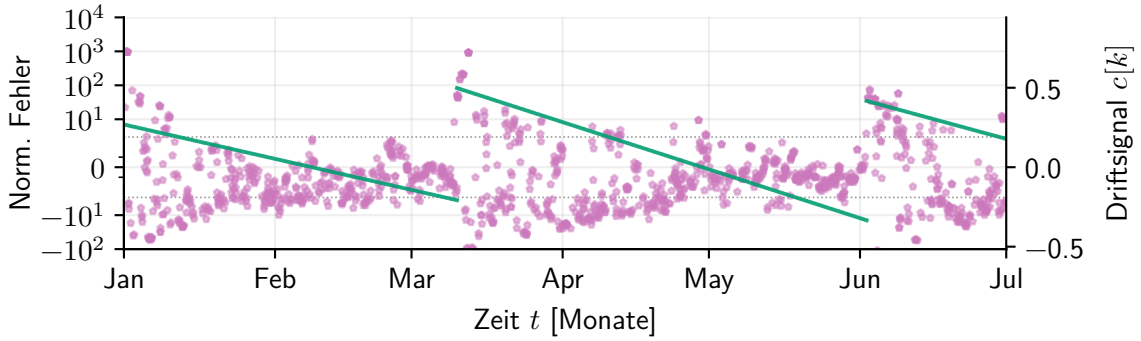
- Die Refraktärzeit wird nicht als absolute Punktzahl, sondern als dimensionsloses Vielfaches der mittleren Dauer stationärer Betriebsphasen optimiert, sodass derselbe Wert im Tuning und in der Anwendung dieselbe physikalische Dauer bezeichnet und ohne Umskalierung übertragbar ist.
- Die kombinierte Strategie erhält die Optima der beiden Einzelstrategien als Startkandidaten der Optimierung. Die Varianten mit Recurrence-Selektor starten von den Selektorparametern der passiven Strategie.
- Die besten Konfigurationen aller vier Strategien werden abschließend auf dem voll aufgelösten Strom über dieselben Seeds wie das Screening gegengeprüft. Übernommen wird je Strategie die beste Konfiguration, unter gleichwertigen nach derselben Regel diejenige mit den wenigsten Nachtrainings.

Das Freeze-Muster, das ausschließlich das Output-Layer freigibt, ist für die Detektor-gestützten Strategien vom Suchraum ausgeschlossen.

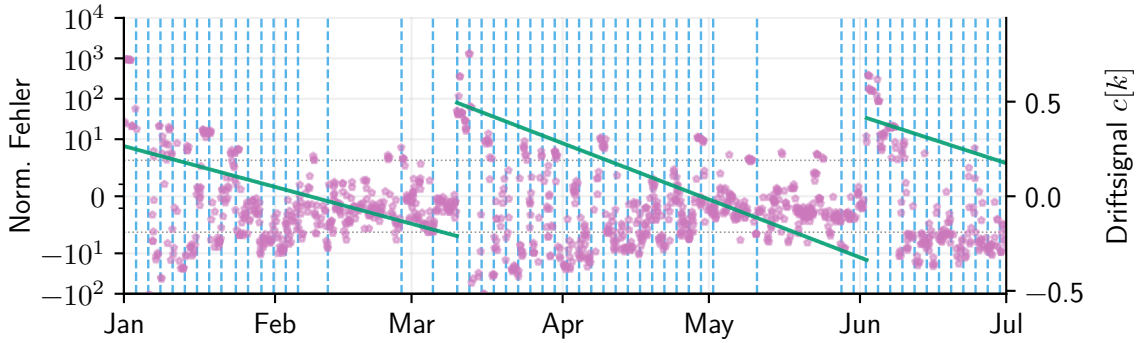
Abbildung 3.10 stellt den normierten Simulationsfehler dem Driftsignal $c[k]$ gegenüber. Die einzelnen Teilabbildungen zeigen für die Baseline ohne Adaption (a), die passive Adaption mit Recurrence-Selektor (b), die aktive (c) und die kombinierte Strategie (d) jeweils den Fehlerverlauf über sechs Monate, wobei in den Detektor-gestützten Varianten zusätzlich die erkannten Drifts markiert sind. Da eine solche Zeitreihe stets nur einen einzelnen der 50 Läufe abbilden kann, wird der Medoid-Seed dargestellt, also der Lauf, dessen Kennzahlen dem Mittel über alle Läufe am nächsten liegen. Der Medoid ist der zentralste tatsächlich aufgetretene Vertreter einer Menge und damit, anders als das rechnerische Mittel, ein real beobachteter Lauf. Die zugehörigen Kennzahlen fasst Tabelle 3.5 zusammen. Ohne Adaption reproduziert das Modell erwartungsgemäß die in Abschnitt 3.1.6 beschriebene Degradation, der Fehlerverlauf folgt sichtbar der Sägezahnstruktur des Driftsignals.



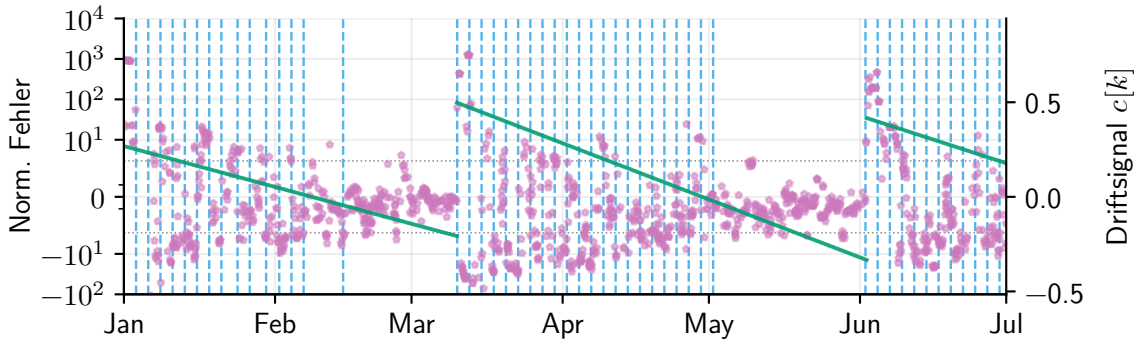
(a) Baseline (keine Adaption)



(b) Passive Adaption mit Recurrence-Selektor



(c) Aktive Adaption



(d) Kombinierte Adaption

Abbildung 3.10: Normierter Simulationsfehler ausgewählter Adaptionsstrategien auf dem driftbehafteten Datenstrom im Vergleich zum Driftsignal $c[k]$; erkannte Drifts sind in (c) und (d) markiert

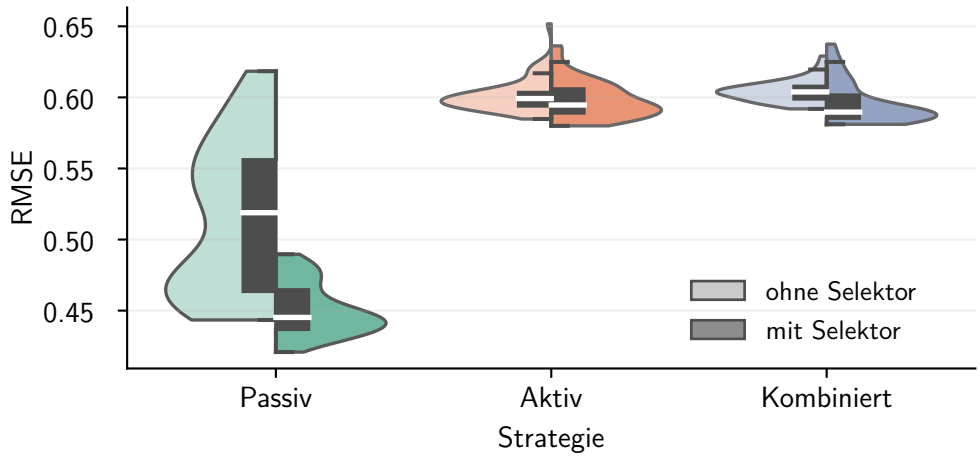


Abbildung 3.11: Verteilung des RMSE je Strategie ohne und mit Recurrence-Selektor über 50 Seeds, dargestellt als Violinen mit überlagerten Boxplots

Mit den getunten Hyperparametern senken alle vier adaptiven Strategien den Simulationsfehler auf deutlich weniger als die Hälfte des Werts ohne Adaption, wobei sich eine klare Rangfolge ergibt (vgl. Tabelle 3.5). Den geringsten Fehler erzielt die passive Adaption mit Recurrence-Selektor. Es folgen die aktive und die kombinierte Strategie nahezu gleichauf, obwohl die kombinierte deutlich häufiger nachtrainiert. Am weitesten zurück bleibt die rein periodische passive Adaption. Sie kommt mit derselben, allein über die Periode n_p festgelegten Zahl an Nachtrainings aus wie die Variante mit Recurrence-Selektor und mit etwa ebenso vielen wie die aktive Strategie. Bei vergleichbarem Eingriffsaufwand zahlen sich damit sowohl die gezielte Auslösung als auch die Auswahl der Trainingsdaten aus, wobei die Auswahl der Trainingsdaten den größeren Beitrag liefert. Die periodische Komponente der kombinierten Strategie fügt dem Modell dagegen zusätzliche Eingriffe hinzu, ohne den Fehler weiter zu senken.

Im Gegensatz zum Fall ohne Adaption, wo der Fehler von einem gerichteten, mit dem Drift korrelierten Anteil dominiert wird, sinkt durch alle Adaptionsstrategien sowohl der Bias als auch die Streuung deutlich. Der mittlere Fehleranteil $\bar{e}$ fällt dabei überall auf eine vergleichbare Größenordnung, sodass die Rangfolge im RMSE nahezu vollständig über die Streuung σ entsteht. Am kleinsten fällt diese unter der passiven Adaption mit Recurrence-Selektor aus, und entsprechend sinkt auch die Korrelation des Fehlerbetrags $|e|$ mit dem Driftbetrag $|c[k]|$ hier am stärksten. Der CD wird durch die Adaption somit weitgehend kompensiert.

Neben den Mittelwerten gibt die Streuung der Läufe über 50 Seeds Aufschluss über die Reproduzierbarkeit der Strategien und über den Beitrag des Recurrence-Selektors. Abbildung 3.11 zeigt die Verteilung des RMSE für die passive, die aktive und die kombinierte Strategie, jeweils ohne und mit Selektor als gegenüberliegende Hälften einer Violine. Die Violinen zeichnen die Dichte der Läufe nach. Ohne Selektor streut die rein periodische passive Adaption am breitesten und liegt deutlich über den beiden Detektor-gestützten Strategien, die sich untereinander kaum unterscheiden. Mit Selektor verschiebt sich die Verteilung der passiven Strategie geschlossen nach unten und wird zugleich schmaler, sodass sie unterhalb aller übrigen Verteilungen liegt.

Bei der aktiven und der kombinierten Strategie fällt die Verschiebung dagegen gering aus, und die beiden Hälften überlappen weitgehend.

Der Selektor wirkt damit je nach Strategie sehr unterschiedlich. Die passive Strategie verbessert er in jedem einzelnen Lauf deutlich. Bei der kombinierten Strategie fällt die Verbesserung klein aus, zeigt sich im gepaarten Vergleich über dieselben Seeds aber durchgängig. Bei der aktiven Strategie lässt sich dagegen kein gesicherter Unterschied nachweisen. Daran ändert auch die Abstimmung der Selektorparameter für diese Strategien wenig. Eine naheliegende Erklärung liegt im Zeitpunkt des Nachtrainings. Die aktive Strategie trainiert gezielt nach einer erkannten Veränderung auf Daten, die den neuen Zustand bereits abbilden, sodass die zusätzliche Auswahl dort kaum etwas zu ergänzen findet. Das fest getaktete Nachtraining der passiven Strategie greift dagegen unabhängig vom Prozesszustand auf das jeweils letzte Fenster zu und profitiert entsprechend stärker von zurückgeholten Daten desselben Zustands. Die kombinierte Strategie liegt mit ihrer periodischen Komponente dazwischen. An der Rangfolge der Strategien ändert der Selektor nichts.

Insgesamt zeigt sich, dass die Adaption den synthetisch erzeugten CD wirksam ausgleichen kann, sofern ihre Hyperparameter geeignet gewählt werden. Als wirksamste Strategie erweist sich hier das passive periodische Nachtraining, ergänzt um den Recurrence-Selektor. Es erreicht den niedrigsten Gesamtfehler bei zugleich geringstem Eingriffsaufwand. Die Detektion verbessert das rein periodische Nachtraining zwar spürbar, bleibt aber hinter der gezielten Auswahl der Trainingsdaten zurück.

3.2 Der Tennessee-Eastman-Process

Beim TEP handelt es sich um eine nichtlineare Simulation einer realen industriellen Chemieanlage, die primär als Benchmark-Prozess für die Entwicklung, Untersuchung und Bewertung von Prozesssteuerungs- und Überwachungstechnologien dient. 1993 veröffentlicht von Downs und Vogel, wurde das Simulationsmodell von der Eastman Chemical Company entwickelt, um eine realistische industrielle Grundlage für die Forschung zur Erprobung von Steuerungsstrategien bereitzustellen [DV93]. Um vertrauliche Eigenschaften des ursprünglichen Verfahrens zu wahren, wurden jedoch spezifische Komponenten wie die Kinetik und die Betriebsbedingungen verändert.
Bei dem Prozess entstehen zwei flüssige Produkte G und H aus vier gasförmigen Reaktanten A, C, D und E. Zusätzlich tritt eine inerte Komponente B auf und es fällt ein Nebenprodukt F an. Das chemische System umfasst dabei zwei gleichzeitig ablaufende primäre exotherme

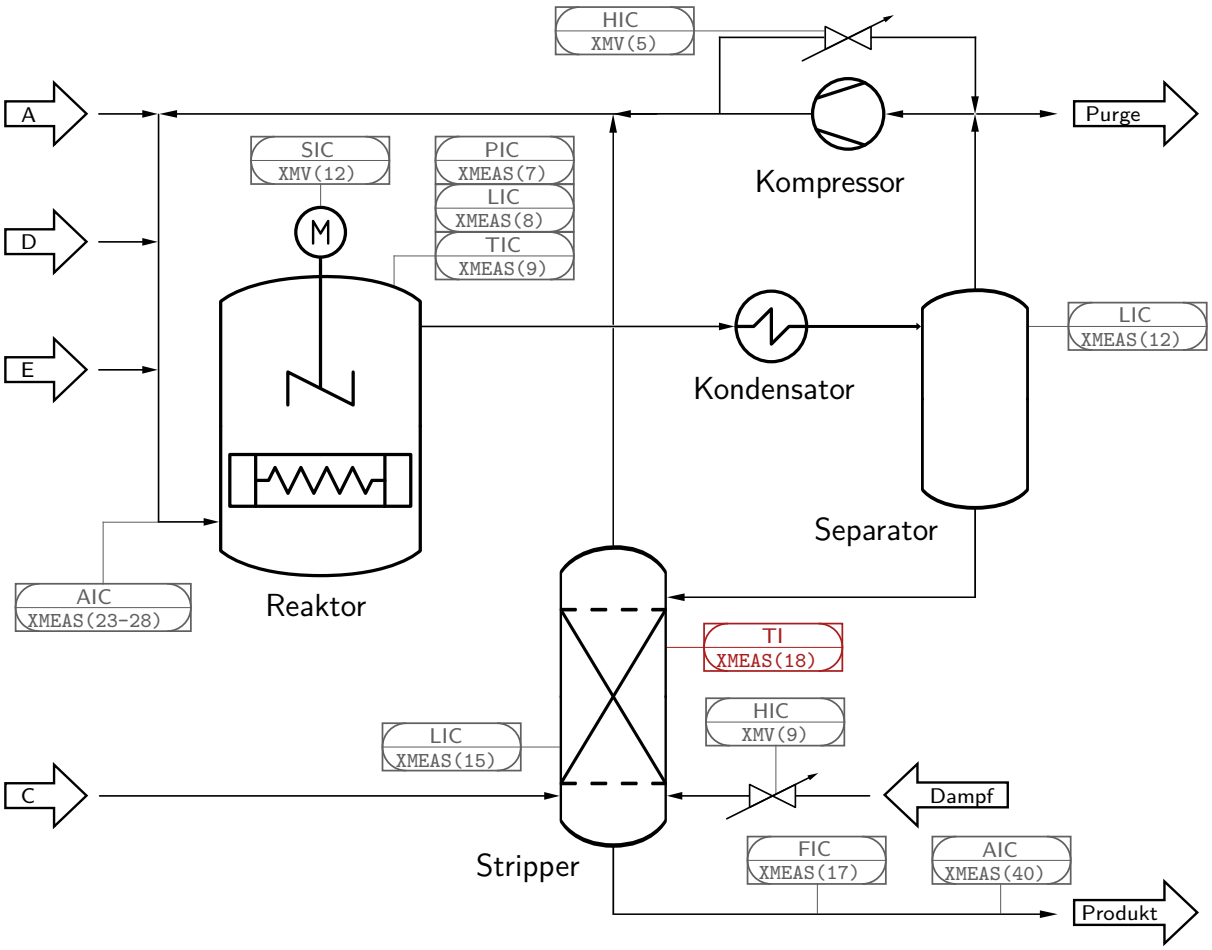


Abbildung 3.12: Anlagenschema des TEP mit seinen fünf Hauptkomponenten, nach [DV93]; in Grau: Setpoints aus Tabelle A.2, in Rot: Strippertemperatur XMEAS(18)

Gas-Flüssigkeits-Reaktionen und zwei zusätzliche Nebenproduktreaktionen:

$$\mathrm{A}_{(\mathrm{g})} + \mathrm{C}_{(\mathrm{g})} + \mathrm{D}_{(\mathrm{g})} \rightarrow \mathrm{G}_{(\mathrm{liq})} \qquad \text{Produkt 1} \qquad (3.18)$$

$$\mathrm{A}_{(\mathrm{g})} + \mathrm{C}_{(\mathrm{g})} + \mathrm{E}_{(\mathrm{g})} \rightarrow \mathrm{H}_{(\mathrm{liq})} \qquad \text{Produkt 2} \qquad (3.19)$$

$$\mathrm{A}_{(\mathrm{g})} + \mathrm{E}_{(\mathrm{g})} \rightarrow \mathrm{F}_{(\mathrm{liq})} \qquad \text{Nebenprodukt} \qquad (3.20)$$

$$3\mathrm{D}_{(\mathrm{g})} \rightarrow 2\mathrm{F}_{(\mathrm{liq})} \qquad \text{Nebenprodukt} \qquad (3.21)$$

Das TEP besteht aus fünf systemischen Hauptkomponenten, die sequenziell miteinander verbunden sind und sich nach der ersten Reaktionsphase in zwei Stränge verzweigen: dem vorwärtsgerichteten Produktstrang und dem rückführenden Recyclingkreislauf. Der Prozessfluss wird im Folgenden beschrieben und kann anschaulich anhand von Abbildung 3.12 nachvollzogen werden.

- **Schritt 1**
  Zunächst werden die Reaktanten A, D und E dem System zugeführt. Zusammen mit dem sich bereits im System befindlichen Reaktanten C und dem Inertgas B, das als Verunreinigung zu verstehen ist, werden die Gase in den **Reaktor** geleitet. Im Reaktor finden die vier irreversiblen und exothermen chemischen Reaktionen statt (siehe (3.18)-(3.21)). Die

Reaktionen laufen an einem in der flüssigen Phase gelösten, nicht-flüchtigen Katalysator ab, wobei die Produkte G und H sowie das Nebenprodukt F entstehen. Die flüssigen Produkte verdampfen und verlassen den Reaktor zusammen mit den nicht umgesetzten Edukten, während der Katalysator im Reaktor verbleibt. Da die exothermen Reaktionen erhebliche Wärme erzeugen, ist der Reaktor mit einem internen Kühlbündel ausgestattet, um die Reaktionswärme abzuführen und eine stabile Temperatur aufrechtzuerhalten.

- **Schritt 2**
  Das resultierende Gemisch aus Dämpfen und nicht umgesetzten Gasen, wird anschließend durch den **Kondensator** geleitet und gekühlt, sodass die Produkte kondensieren.
- **Schritt 3**
  Das Gemisch gelangt dann in den **Dampf-Flüssigkeits-Separator**, der die flüssigen Produkte von Gasen trennt und den Prozess so in zwei Stränge teilt: den vorwärtsgerichteten Produktstrang und den rückführenden Recyclingkreislauf.
- **Schritt 4**
  Die Flüssigkeit aus dem Separator gelangt zum **Produktstripper**, wo unten das Reaktionsgas C zugeführt wird. Die Flüssigkeit wird so durch das aufsteigende Gas von Resten der verbleibenden Reaktanten gereinigt. Die Kopfgase gelangen dann über den Recyclingkreislauf wieder in den Reaktor.
- **Recyclingkreislauf**
  Ein Teil der nicht kondensierten Dämpfe aus dem Separator wird abgesaugt, um zu verhindern, dass sich inerte Stoffe und Nebenprodukte im Prozess ansammeln. Die restlichen Dämpfe des Separators werden durch den **Kompressor** geleitet, der sie im Recyclingkreislauf zurück in den Reaktor führt, wo sich ihnen die Kopfgase des Strippers zumischen. Durch die Rückführung der nicht umgesetzten Stoffe wird eine Steigerung der Gesamteffizienz erreicht.

Der `TEP` verfügt über 41 Messgrößen `XMEAS(1)` bis `XMEAS(41)` und 12 Stellgrößen `XMV(1)` bis `XMV(12)`, wobei der Großteil der Stellgrößen der Ventilsteuerung dient. In der ursprünglichen Form von Downs und Vogel ist der Prozess ungeregelt und im offenen Betrieb instabil, sodass er bereits nach kurzer Zeit in eine Sicherheitsabschaltung läuft. Downs und Vogel geben bewusst keine Regelstrategie vor, sondern stellen deren Entwurf als eigentliche Aufgabe des Benchmarks [DV93]. Ebenso geben sie zwar sechs primäre Betriebsmodi vor, die über das Massenverhältnis der Produkte G und H sowie über die Produktionsrate definiert sind, legen damit jedoch keine Betriebspunkte fest. Ein Modus ist als Anforderung aus der Nachfrage zu verstehen, die der Prozess auf ganz unterschiedlichen stationären Zuständen erfüllen kann.

Sowohl die nicht festgelegten Betriebspunkte als auch die fehlende Regelstrategie adressiert Ricker. Er bestimmt zunächst zu den sechs Modi die optimalen Betriebspunkte, also diejenigen stationären Zustände, welche die jeweilige Vorgabe zu den geringsten Betriebskosten erfüllen

[Ri95], und entwirft darauf aufbauend ein dezentrales Regelkonzept, das den Prozess in einem solchen Punkt hält und Störungen ausregelt [Ri96]. Das in dieser Arbeit verwendete Simulationsmodell beruht auf diesem Konzept. Die Stellgrößen werden dadurch von den Reglern gestellt und nur noch aufgezeichnet, während der Betrieb über Setpoints geführt wird. Diese entsprechen in ihrer Rolle den Setpoints der Mühle aus Abschnitt 2.1.2, werden also exogen vorgegeben und anschließend ausgeregelt, und ihre Kombination bildet den Betriebspunkt. In Abbildung 3.12 sind die zugehörigen Messstellen grau hinterlegt; Tabelle A.2 im Anhang führt ihre Werte für die sechs Betriebsmodi auf.

Ursprünglich wurde das Modell des TEP von [DV93] als FORTRAN-Quellcode veröffentlicht. Später wurde dieser Quellcode in C-Code konvertiert, um ihn als S-Funktion in MATLAB®/Simulink® verwenden zu können. Dieser Code ist im *Tennessee Eastman Challenge Archive* zu finden, welches einige weitere Simulator-Versionen enthält, darunter das zuvor genannte dezentrale Regelkonzept sowie Ansätze zu Model Predictive Control [Ri05]. Die aktuelle Version des C-Codes ist das Ergebnis einer Überarbeitung von Bathelt et al., die notwendig war, da die Simulations-Engine von Simulink und die ursprüngliche Struktur des Modells nicht mehr zusammenpassten [BRJ15]. Im Zuge dieser Überarbeitung wurden die ursprünglich 20 Störungsszenarien um acht zusätzliche Störungen erweitert [BRJ15]. Tabelle A.1 im Anhang listet alle Prozessstörungen auf. Zur Simulation in Simulink existiert außerdem das Python-Paket *pyTEP*, das eine Open-Source-Schnittstelle zwischen Simulink und Python bereitstellt und so die Steuerung der Simulation über Python ermöglicht, bei der Prozesseingaben verändert und komplexe Szenarien konfiguriert werden können [RE22]. Die Daten lassen sich so unter anderem als Pandas DataFrames speichern. Dass der Prozess in dieser Form als Quellcode vorliegt, eröffnet zugleich die Möglichkeit, gezielt in die Prozessdynamik einzugreifen.

In seiner ursprünglichen Konzeption ist der TEP nicht als Grundlage für die Untersuchung von CD vorgesehen. Downs und Vogel definieren einen festen Katalog vordefinierter Störungen (IDV), die typische Fehlerfälle wie sprunghafte Änderungen der Zulaufzusammensetzung oder Ventilblockaden abbilden, einen kontinuierlichen, schleichenden CD der hier betrachteten Art jedoch nicht umfassen [DV93]. Dem TEP liegt jedoch ein deterministisches dynamisches Prozessmodell zugrunde. Dessen stochastische Anteile lassen sich kontrolliert parametrieren, und als Simulation eines realen industriellen Prozesses bleibt er zugleich anwendungsnah. Der TEP wird daher über seine ursprüngliche Bestimmung hinaus genutzt und um einen drift-behafteten Betrieb erweitert. Die dafür entwickelte Injektion eines CD-Einflusses unmittelbar im C-Code ergänzt den TEP um einen in den etablierten Störungskatalogen nicht enthaltenen, schleichenden Fehlerfall. Damit erfüllt er alle im Eingangsabschnitt von Kapitel 3 beschriebenen Voraussetzungen und eignet sich somit als Datengrundlage für die in dieser Arbeit behandelte Problemstellung.

Der so erweiterte Prozess bildet die Grundlage der Verarbeitungskette aus Generierung, Modellierung, Detektion und Adaption analog zu Abschnitt 3.1. Zunächst wird in Abschnitt 3.2.1 die Modellierung des CD im C-Code beschrieben, die den drift-behafteten Fehlerfall realisiert.

Abschnitt 3.2.2 erläutert die darauf aufbauende Generierung des Datenstroms über die pyTEP-Schnittstelle. Der erzeugte Datenstrom wird anschließend in Abschnitt 3.2.3 modelliert, bevor Abschnitt 3.2.4 die Detektion des CD und Abschnitt 3.2.5 abschließend die Adaption des Modells an den drift-behafteten Prozess behandelt.

3.2.1 Concept Drift

Unter den definierten Störfällen kommt IDV(13) der Zielsetzung dieser Arbeit auf den ersten Blick nah, da er in der Literatur als *slow drift* der Reaktionskinetik bezeichnet wird [DV93]. Bei Aktivierung multipliziert das Modell dabei die Raten der Primärreaktionen (3.18) und (3.19) mit den zeitvarianten Störfaktoren r1f bzw. r2f. Diese ziehen in zufälligen Zeitabständen gleichverteilte Zielwerte aus dem Intervall $[0{,}75; 1{,}25]$ (Funktion tesub5) und interpolieren dazwischen kubisch (Funktion tesub8), sodass ein glatter, strikt begrenzter Verlauf entsteht. Bei deaktiviertem Fehler bleiben die Faktoren konstant 1 und die Reaktionskinetik unverändert.

Durch IDV(13) bleibt der Prozess somit stationär und ohne Trend, da sein Gedächtnis nur wenige Stunden weit reicht und mit dem nächsten Zielwert endet. Die induzierte Fluktuation verbreitert lediglich die Verteilung der betroffenen Prozessgrößen einmalig bei Aktivierung, verschiebt sie anschließend aber nicht weiter. Da CD gemäß (2.9) eine Veränderung der gemeinsamen Verteilung $P(X, y)$ voraussetzt, liegt entgegen der Bezeichnung *slow drift* gemäß den formellen Definitionen in Tabelle 2.3 kein CD vor, sondern allenfalls eine abrupte Störung mit stationärem Charakter.

Für die Zielsetzung dieser Arbeit folgt daraus, dass der TEP in seiner ursprünglichen Form keinen Fehler bereitstellt, der diese Eigenschaften erfüllt. Um den TEP dennoch als Benchmark für die Drift-Erkennung und -Adaption nutzen zu können, wird das Simulationsmodell um eine Störgröße erweitert, die einen strukturierten, anhaltenden Drift nach dem Vorbild des synthetischen Datenstroms aus Abschnitt 3.1 erzeugt.

Realisiert wird diese Störgröße durch einen Eingriff im C-Code als IDV(29), in Fortführung der etablierten Störungen (siehe Tabelle A.1). Aktivierung und Amplitude sind über dieselbe Schnittstelle einstellbar wie bei den übrigen Störungen. Offen bleibt, an welcher Stelle im Prozess der Drift angreifen soll. Naheliegend wäre dazu ein Angriffspunkt in der Anlagenphysik, etwa ein Eingriff in die Reaktionskinetik wie durch IDV(13), da sich dort eine tatsächliche Veränderung des Prozessverhaltens abbildet. Der TEP ist jedoch vollständig geregelt. Diese Regelung fängt jede anhaltende Störung ab und überführt sie in eine bleibende Veränderung der Stellgrößen, die gemeinsam mit den Messgrößen aufgezeichnet werden. Anhand ebendieses Messvektors werden die Betriebskosten als Zielgröße durch den Simulator deterministisch erzeugt. Eine Veränderung der Anlagenphysik ist in den aufgezeichneten Daten also vollständig sichtbar und der Zusammenhang zwischen ihnen und den Kosten bleibt erhalten. Das gilt auch für wenige ausgewählte Features, da diese nach ihrer Erklärungskraft für die Kosten bestimmt

werden und die Störung folglich nur um den Preis ihrer eigenen Prädiktivität ausblenden könnten. Verschoben wird lediglich die Lage der Daten im Feature-Raum, was gemäß (2.7) einem *Covariate Shift* entspricht und die gelernte Abbildung von Features auf Kosten gültig lässt.

Außerhalb dieses Wirkungskreises verbleibt jedoch die Sensorik, welche die Messwerte bereitstellt. Messstellen unterliegen in der industriellen Praxis Einflüssen wie Verschmutzung und Alterung, während Wartungs- und Kalibrierintervalle begrenzt sind. Es wird daher angenommen, dass ein Thermoelement über Wochen hinweg einen zunehmend verfälschten Wert liefert, ohne dass sich am Prozess selbst etwas ändert. Genau diesen Sensorverschleiß bildet `IDV(29)` an der Strippertemperatur (`XMEAS(18)`, in Abbildung 3.12 rot hervorgehoben) nach. Sie wird im `TEP` aufgezeichnet, ist kein Teil eines Reglers und tritt in der Kostenformel des Simulators nicht auf. Verfälscht wird deshalb allein der ausgegebene Messwert, während der Prozesswert und mit ihm das Anlagenverhalten unberührt bleiben.

Damit folgen die Betriebskosten weiterhin dem unveränderten Prozess, während der aufgezeichnete Messwert zunehmend von ihm abweicht. Ein und derselbe beobachtete Wert der Strippertemperatur gehört im Verlauf des Drifts zu immer anderen Betriebskosten, sodass sich die bedingte Verteilung $P(y \mid X)$ verschiebt und nach der Systematik gemäß (2.10) *Real Concept Drift* vorliegt. Der aufgezeichnete Messwert wird dabei mit dem Faktor $1 + \kappa_m \cdot c(t)$ skaliert, worin κ_m den Driftkoeffizienten und $c(t)$ das im Folgenden beschriebene Driftsignal bezeichnet. Da dieser Faktor multiplikativ wirkt, ist der absolute Fehler Betriebspunkt-abhängig. Das Driftsignal ist dabei strukturgleich zum synthetischen Signal $c[k]$ aus Abschnitt 3.1.1, wird jedoch als $c(t)$ zeitkontinuierlich formuliert. Das liegt daran, dass das Simulink-Modell den Solver `ode45` mit veränderlicher Schrittweite verwendet und die Aufrufzeitpunkte damit auf keinem festen Raster liegen. Innerhalb eines Segments fällt es gemäß Gleichung (3.1) jedoch weiterhin linear von einem Niveau eines oberen Bandes auf ein Niveau eines unteren Bandes ab. An der Segmentgrenze springt es auf das Startniveau des folgenden Segments, sodass beide Drifttypen aus Abbildung 2.5 auch im `TEP` abgebildet sind. Die Segmentgrenzen folgen demselben Raster der nominellen Segmentdauer T_{seg} mit unveränderter zufälliger Verschiebung. Da der `TEP` über einen frei gewählten Zeitraum statt über ein Kalenderjahr simuliert wird, ergibt sich hier lediglich die Segmentzahl aus der Simulationsdauer. Alle Zufallsniveaus werden deterministisch aus einem Seed und dem Segmentindex abgeleitet, womit das Signal zustandslos und unabhängig von der Schrittweite des Solvers reproduzierbar ist.

Die Segmentdauer T_{seg} wird aus dem synthetischen Fall übernommen. Nach unten ist sie dadurch begrenzt, dass sich der Kalibrierfehler während des Anfahrens und Haltens eines Betriebspunkts nur unwesentlich ändern darf, damit jeder Betriebspunkt einen einzelnen Driftpegel abbildet. Der Driftkoeffizient κ_m ist so bemessen, dass die Verschiebung des Simulationsfehlers deutlich über dem Rauschen des Modells innerhalb eines Betriebspunkts liegt, der Kalibrierfehler in der Spitze aber als schleichende Dekalibrierung eines Thermoelements plausibel bleibt.

Der Eingriff in den Quelltext des Simulators beschränkt sich auf wenige Zeilen in

```
xmeas[17] = tcc;                        /* Prozesswert von XMEAS(18) */
if (idv[29] > 0.) {
    if (drift_tonset < 0.) drift_tonset = time;
    driftf = 1. + idv[29] * drift_signal(time - drift_tonset,
                                DRIFT_PERIOD, DRIFT_GAMMA, DRIFT_SEED);
} else {
    drift_tonset = -1.;
    driftf = 1.;
}
xmeas[DRIFT_SENSOR_INDEX] *= driftf;
```

Codeausschnitt 3.1: Anwendung des Sensordrifts in `temexd_mod.c`, gekürzt und vereinfacht

`temexd_mod.c`, die in Codeausschnitt 3.1 dargestellt sind. Das Driftsignal selbst ist in der separaten Datei `drift_function.c` umgesetzt, welche die S-Funktion als Quelltext einbindet und die sich zugleich als dynamische Bibliothek übersetzen lässt, sodass dasselbe Signal auch aus Python ausgewertet werden kann. Im Codeausschnitt bezeichnet `tcc` die vom Prozessmodell berechnete Strippertemperatur, die zunächst dem null-indizierten Messvektor `xmeas` an der Stelle von `XMEAS(18)` zugewiesen wird. Der Eintrag `idv[29]` enthält den über die Schnittstelle eingestellten Wert der Störung und damit den Driftkoeffizienten κ_m. Beim ersten Aufruf nach der Aktivierung wird die Simulationszeit `time` in `drift_tonset` gespeichert. Beim Abschalten der Störung wird dieser Wert auf einen negativen Wert zurückgesetzt, sodass eine erneute Aktivierung das Muster neu beginnt. Die Funktion `drift_signal` liefert $c(t)$ für die seit der Aktivierung verstrichene Zeit und wird über die Konstanten `DRIFT_PERIOD`, `DRIFT_GAMMA` und `DRIFT_SEED` parametriert, die T_{seg}, γ_τ und dem Seed der Zufallsniveaus entsprechen. Der resultierende Faktor `driftf` skaliert den Eintrag an `DRIFT_SENSOR_INDEX`, der auf `XMEAS(18)` verweist, erst nach der Zuweisung des Prozesswerts. Er wirkt damit ausschließlich auf den ausgegebenen Messwert, während der simulierte Prozess in seinem Verhalten identisch zum ungestörten Fall bleibt.

3.2.2 Generierung des Datenstroms

Auf Basis des drift-behafteten Prozessmodells wird über die *pyTEP*-Schnittstelle ein Datenstrom erzeugt, der als Grundlage der anschließenden Modellierung, Detektion und Adaption dient. Das Paket kapselt das Simulink-Modell hinter einer Python-Schnittstelle [RE22], sodass die eigentliche Simulation weiterhin in MATLAB®/Simulink® abläuft und Python allein die Ablaufsteuerung übernimmt. Über die Schnittstelle lässt sich der Simulator in seinen Ausgangszustand zurücksetzen, es lassen sich einzelne Setpoints über eine wählbare Dauer rampen, Störungen aktivieren und die Simulation über ein vorgegebenes Zeitintervall vorantreiben. Nach jedem dieser Intervalle stehen Messgrößen, Stellgrößen, Setpoints, Betriebskosten und Störungszustände als DataFrames bereit und können so zu einem fortlaufenden Datensatz zusammengeführt werden.

Der in Abschnitt 3.2.1 beschriebene Eingriff liegt im C-Quelltext des Prozessmodells und ist damit Teil der übersetzten S-Funktion. Für die Werkzeugkette folgt daraus, dass `IDV(29)` über

denselben Aufruf angesprochen wird wie die etablierten Störungen und sich der drift-behaftete Lauf vom nominalen allein durch das gesetzte Störungsargument unterscheidet. Eine Änderung am Drift erfordert eine Neuübersetzung des Modells, nicht aber eine Anpassung der Ablaufsteuerung. Die Anbindung erfordert daneben kleinere Eingriffe im Paket selbst. Ergänzt werden `IDV(29)` in der auf achtundzwanzig Einträge ausgelegten Störungsliste samt zugehöriger Initialisierung sowie die mitzuübersetzenden Quelldateien des Drifts. Hinzu kommt eine heraufgesetzte maximale Simulationsdauer, da die Voreinstellung unterhalb der benötigten Laufzeit liegt.

Anders als beim synthetischen Datenstrom aus Abschnitt 3.1, dessen Erzeugung vollständig im Skript formuliert ist, wird hier lediglich die Anregung des Prozesses vorgegeben. Prozessantwort und Zielgröße entstehen im Simulator. Wie der `TEP` jedoch unter Sicherstellung des Betriebs angeregt werden soll, um einen geeigneten Datensatz für die Zwecke dieser Arbeit zu erzeugen, ist Kern dieses Abschnitts. Den Rahmen der Simulation bilden dabei die sechs Betriebsmodi des `TEP`, die Downs und Vogel über das Massenverhältnis der beiden Produkte und die Produktionsrate festlegen (siehe Tabelle 3.6) [DV93]. Die ersten drei Modi geben feste Raten vor, während die übrigen drei die jeweils höchste erreichbare Rate fahren. Da der Prozess vollständig geregelt ist, wird ein Modus nicht unmittelbar über die zwölf Stellgrößen des Prozessmodells gesetzt, sondern über zwölf Setpoints, die die Schnittstelle durch `pyTEP` freigibt. Ihre Kombination definiert einen Betriebspunkt. Der Simulator startet im stationären Optimum des ersten Modus.

Tabelle 3.6 nennt die Kenngrößen, über die die sechs Modi definiert sind. Der zugehörige Satz aus zwölf Setpoints gehört zum dezentralen Regelkonzept von Ricker [Ri96], das auch dem hier verwendeten Simulationsmodell zugrunde liegt, und beschreibt den jeweils optimalen stationären Betriebspunkt [Ri95]. Übernommen sind die Werte in der Zusammenstellung von Reinartz et al. [RKR21] und in Tabelle A.2 im Anhang aufgeführt. Über alle Modi gleich bleiben dabei Füllstände, Reaktordruck, Dampfventil und Rührerdrehzahl, sodass die Modusidentität im Wesentlichen von Produktionsrate, Produktzusammensetzung, Reaktortemperatur und Zulaufzusammensetzung getragen wird, in den Modi 3 und 6 zusätzlich von der Stellung des Recycle-Ventils.

In die Simulation gehen alle sechs Modi ein. Die Abdeckung unterschiedlicher, über die Setpoints gezielt erzeugbarer Betriebspunkte zählt zu den in Kapitel 3 formulierten Voraussetzungen an die Datengrundlage und war für die Wahl des `TEP` als Benchmark-Prozess maßgeblich. Ein einzelner Modus deckt nur einen schmalen Ausschnitt des Betriebsbereichs ab, während der Datenstrom die Bandbreite abbilden soll, in der sich ein Modell im Betrieb bewähren muss. Hinzu kommt, dass der Kalibrierfehler aus Abschnitt 3.2.1 multiplikativ wirkt und sein absoluter Beitrag damit vom Betriebspunkt abhängt.

Die sechs Modi aus Tabelle 3.6 allein genügen der erforderlichen Betriebspunktabdeckung nicht. Daher wird jeder Modus zu einer Schar benachbarter Betriebspunkte ausgedehnt. Als Rahmen

	Massenverhältnis G/H	Produktionsrate in $\mathrm{kg\,h^{-1}}$
Modus 1	50/50	14076
Modus 2	10/90	14076
Modus 3	90/10	11111
Modus 4	50/50	maximal
Modus 5	10/90	maximal
Modus 6	90/10	maximal

Tabelle 3.6: Kenngrößen der sechs Betriebsmodi des TEP nach [DV93]

dient das Fenster von $\pm 15\%$ je Setpoint, das Reinartz et al. in Schritten von 5% abtasten [RKR21]. Dieser Rahmen wird übernommen, das Raster der Abtastung dagegen ersetzt. Die Grenzen sind dabei als Prozentsatz des Modusnennwerts festgelegt und nicht als fester Absolutbetrag, sodass die Auslenkung über die Modi hinweg vergleichbar bleibt, obwohl die Setpoints in unterschiedlichen Größenordnungen liegen. Zulässig ist jede Kombination, die in allen zwölf Größen innerhalb dieser Grenzen liegt, also ein eng begrenzter Ausschnitt des Setpoint-Raums um den Nennwert des Modus. Das entspricht dem Stellgrößenraum aus Abschnitt 2.1.2, hier jedoch lokal um einen bekannten Betriebspunkt statt über den gesamten Stellbereich.

Zwei Einschränkungen ergeben sich jedoch aus dem Prozess. Die Rührerdrehzahl, deren Nennwert bei voller Leistung und damit bereits am Rand des Stellbereichs liegt, wird nur nach unten variiert. Zudem bleibt das Fenster bewusst eng, da der TEP über Sicherheitsabschaltungen verfügt und eine Kombination weit ab vom optimalen Betriebspunkt zum Abbruch der Simulation führen kann. Das Fenster wird nach dem Latin-Hypercube-Ansatz abgetastet, wie er im Mühlen-Anwendungsfall in Abschnitt 2.2.3 und beim synthetischen Datenstrom in Abschnitt 3.1.6 zum Einsatz kommt. Das Verfahren unterteilt jede der zwölf Achsen in gleich große Intervalle und belegt jedes davon genau einmal, sodass jeder Setpoint seinen Bereich unabhängig von der Zahl der Punkte gleichmäßig abdeckt.

Die gleichmäßige Belegung der einzelnen Achsen schließt jedoch nicht aus, dass die Punkte im Raum ungünstig liegen können und sich einzelne Paare unnötig nahe sind. Daher wird nicht eine einzelne Ziehung verwendet, sondern aus einer größeren Zahl unabhängiger Ziehungen diejenige genutzt, deren kleinster Punktabstand am größten ausfällt. Dieses Maximin-Kriterium bevorzugt Versuchspläne, die das Fenster gleichmäßig ausfüllen, und vermeidet Häufungen, die einen Teil des Betriebsbereichs unbelegt ließen.

Jeder Modus erhält einen eigenen Versuchsplan in seinem Fenster, ergänzt um den unveränderten Nennwert als Bezugspunkt. Daraus ergeben sich 50 Punkte je Modus und insgesamt 300 Kandidaten. Alle zwölf Setpoints werden gemeinsam über eine Rampendauer von $48\,\mathrm{h}$ auf ihre Zielwerte geführt, da ein Sprung die Regelung an ihre Grenzen bringt und eine Sicherheitsab-

schaltung auslösen kann. Anschließend bleibt der Betriebspunkt stehen, bis die Ausgleichsvorgänge abgeklungen sind und der Prozess einen quasi-stationären Zustand erreicht hat. Zwischen zwei Punkten kehren die Setpoints über eine weitere Rampe in den Ausgangszustand zurück. Jeder Betriebspunkt wird damit aus demselben Zustand angefahren und weite Sprünge zwischen entfernt liegenden Punkten bleiben aus. Ein Punkt belegt so einen Block von $144\,\mathrm{h}$. Dieser Block ist vergleichsweise kurz gegenüber der Segmentdauer des Drifts aus Abschnitt 3.2.1, sodass jeder Betriebspunkt näherungsweise einem einzelnen Driftpegel zugeordnet bleibt.

Ein Punkt aus dem Versuchsplan ist nicht zwangsläufig fahrbar. Das Simulationsmodell in Simulink überwacht Reaktordruck, Reaktortemperatur sowie die Füllstände in Reaktor, Separator und Stripper und bricht die Simulation ab, sobald eine dieser Größen ihre Schranke verlässt. Die Schranken sind im Quelltext des Modells hinterlegt und werden von dort übernommen, sodass zur Prüfung dieselben Grenzen herangezogen werden wie es der Simulator tut. Ein Abbruch mitten im fortlaufenden Datenstrom ist dabei besonders ungünstig, da der Strom an dieser Stelle endet und mit dem Neustart des Simulators auch der Driftverlauf zurückgesetzt wird. Jeder Kandidat wird deshalb in einem Vorlauf einzeln geprüft. Als fahrbar gilt ein Punkt dann, wenn der Block ohne Abschaltung zu Ende läuft.

Der Vorlauf ist zugleich der Kampagnenlauf, der die Trainingsdaten für das später folgende Modell bereitstellt. Jeder fahrbare Block wird aufgezeichnet, sodass ein Bestand einzeln angefahrener Betriebspunkte entsteht, die alle aus demselben Ausgangszustand hervorgehen. Anders als im fortlaufenden Betrieb entfällt dabei die Rückkehr in den Ausgangszustand, da der Simulator nach jedem Kandidaten zurückgesetzt wird. Ein Block des Vorlaufs umfasst deshalb allein die Anfahrrampe und den stationären Bereich. Von den 300 Kandidaten erweisen sich 136 Punkte als fahrbar. Die Ausbeute fällt je Modus dabei unterschiedlich aus. Modus 1 ist der Auslegungspunkt des Prozesses und zugleich der Ausgangszustand, aus dem jeder Block angefahren wird, und liegt deutlich über den übrigen, deren stärker verschobene Setpoints häufiger in eine Abschaltung führen. Da ein Betriebspunkt nur als Ganzes verwertbar ist, geht diese Ausbeute unmittelbar in den Umfang des späteren Datenstroms ein. Die fahrbaren Punkte werden anschließend zu einem fortlaufenden Datenstrom aneinandergereiht, der einen Produktionsbetrieb über zwei Jahre nachbildet. Ihre Reihenfolge wird zufällig gezogen. Dieser Lauf findet ohne den Einfluss eines Drifts statt und dient so als drift-freie Referenz für den Fehlerfall.

Im fortlaufenden Betrieb hat sich jedoch gezeigt, dass ein Teil der Punkte trotz des Vorlaufs ausfällt, da die Betriebspunkte hier nicht aus dem Ausgangszustand, sondern aus dem Zustand des vorangegangenen Blocks angefahren werden. Der Lauf wird deshalb ein zweites Mal ohne die ausgefallenen Punkte gefahren und liefert so einen durchgehenden Strom aus 122 Betriebspunkten über rund zwei Jahre. Die Simulationszeit wird dabei auf Zeitstempel abgebildet, wie in Abschnitt 3.1.1 beschrieben.

Die gemeinsame Variation aller zwölf Setpoints geht über das hinaus, was für einen Prozess dieser Art üblich ist, und ist in der Literatur zum TEP in dieser Form nicht zu finden. Sie ist hier

jedoch nötig, damit die Datengrundlage der Betriebspunktvielfalt des Mühlen-Anwendungsfalls entspricht und die angestrebte Generalisierung trägt.

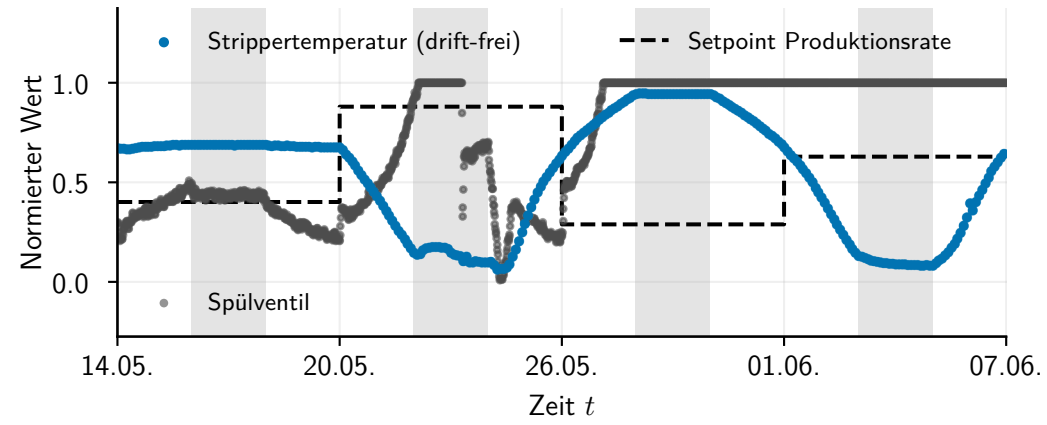
Der Fehlerlauf wiederholt dieselbe Folge von Betriebspunkten mit aktivierter Störung `IDV(29)`. Der Simulator wird dafür neu aufgesetzt, sodass beide Läufe aus demselben Zustand starten. Da der Kalibrierdrift den Prozess nicht berührt, sind beide Läufe prozessseitig identisch. Die Amplitude des Drifts liegt zwischen zwei Schranken. Nach unten muss die von ihm verursachte Verschiebung des Simulationsfehlers deutlich über dem Rauschen des Modells innerhalb eines Betriebspunkts liegen, damit der Drift überhaupt erkennbar wird. Nach oben muss der Kalibrierfehler als schleichende Dekalibrierung eines Thermoelements plausibel bleiben. Mit $\frac{3}{10}$ erreicht der Kalibrierfehler einen Spitzenwert von $14{,}7\,\%$ beziehungsweise $9{,}2\,\mathrm{K}$.

Damit liegen drei Datensätze vor. Die Kampagne aus einzeln angefahrenen Betriebspunkten bildet den Bestand, aus dem die Trainingsdaten gezogen werden, der nominale Lauf dient als drift-freie Referenz und der Fehlerlauf als drift-behafteter Strom, der über seine Laufzeit rund zehn Driftperioden durchläuft. Abbildung 3.13 zeigt einen Ausschnitt über vier Betriebspunkte. (a) stellt den Setpoint der Produktionsrate zwei Messgrößen gegenüber: die Strippertemperatur und das Spülventil. Erkennbar sind das Anfahren über die Rampe, das hinterlegte Haltefenster mit dem quasi-stationären Verlauf und die Rückkehr in den Ausgangszustand. (b) zeigt die Strippertemperatur im Drift-freien sowie im Drift-behafteten Fall zusammen mit dem Driftsignal. Die Abweichung zwischen beiden Läufen folgt dem Verlauf des Driftsignals einschließlich seines Sprungs an der Segmentgrenze.

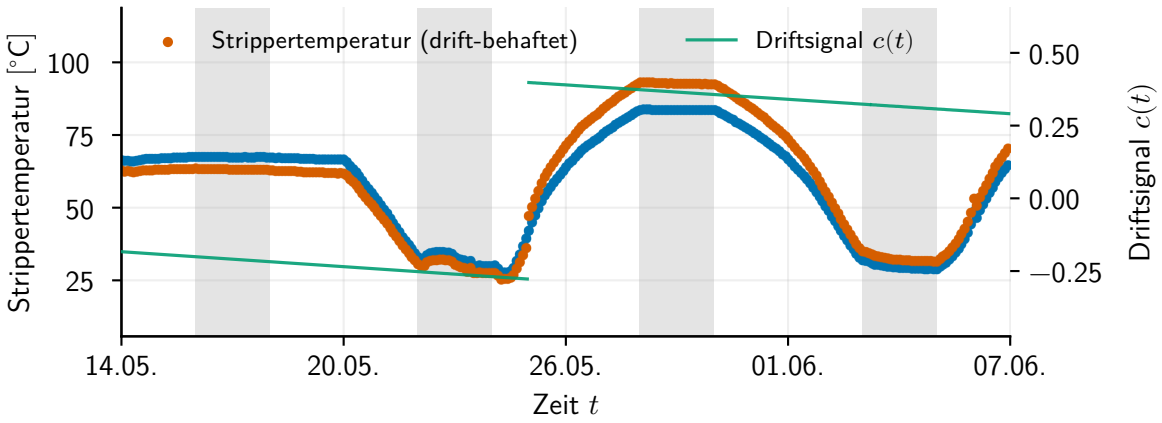
3.2.3 Modellierung

Nachdem der `TEP` in Abschnitt 3.2.1 um einen Kalibrierdrift erweitert und in Abschnitt 3.2.2 ein Datenstrom erzeugt wurde, soll der so erzeugte Prozess nun modelliert werden. Trainiert wird das Modell auf dem Datensatz der Kampagne, während die beiden Produktionsläufe der Bewertung dienen. Für die Bewertung ist entscheidend, dass beide Läufe dieselbe Folge von Betriebspunkten durchlaufen haben und prozessseitig identisch sind. Sie unterscheiden sich allein im aufgezeichneten Messwert der Strippertemperatur, sodass jede Abweichung zwischen ihren Fehlerverläufen auf den Kalibrierdrift zurückgeht. Der Vergleich der beiden Verläufe macht damit die drift-bedingte Modellverschlechterung sichtbar.

Das Vorgehen entspricht grundsätzlich dem des synthetischen Datenstroms aus Abschnitt 3.1.6: Ein Modell wird auf einem drift-freien Referenzdatensatz trainiert, auf den Datenstrom angewandt und über seinen Simulationsfehler bewertet. Der `TEP` stellt daran zwei Anforderungen, die im synthetischen Fall nicht bestanden. Die Zielgröße entsteht im Simulator, während die Systemantwort des synthetischen Prozesses in Abschnitt 3.1.5 durch eine bekannte Gleichung gegeben ist. Zudem zeichnet der `TEP` eine Vielzahl von Mess- und Stellgrößen auf, aus denen die Features des Modells zu wählen sind, wohingegen der synthetische Prozess seine beiden



(a) Betriebspunkte



(b) Drift

Abbildung 3.13: Ausschnitt des erzeugten Datenstroms über vier Betriebspunkte.

Eingangssignale inhärent vorgibt.

Als Label dient die Zielgröße der Betriebskosten, die der Simulator nach der von Downs und Vogel angegebenen Rechnung deterministisch aus dem Messvektor bildet [DV93]. Sie ist damit über den aufgezeichneten Messvektor eindeutig bestimmt, ohne dass der Zusammenhang zwischen Betriebspunkt und Kosten in geschlossener Form vorläge. Aus derselben Rechnung werden auch die Features abgeleitet. Den größten Anteil an den Kosten tragen die Verluste über den Spülstrom. Innerhalb des Spülstroms folgen allein die Anteile der Komponenten D und E den Kosten über die Betriebspunkte hinweg. Daraus ergibt sich die Wirkkette, die die Auswahl der Features begründet. Das Spülventil stellt die Rate des Spülstroms ein. Die Zufuhr der Komponente E zum Reaktor ist deren Quelle, und die Separatortemperatur entscheidet darüber, welcher Anteil in den Spülstrom gelangt statt zu kondensieren. Hinzu kommt das geregelte Massenverhältnis der beiden Produkte, das die sechs Betriebsmodi voneinander trennt. Größen, die unmittelbar in die Kostenrechnung eingehen, bleiben ausgeschlossen, damit das Modell den Prozesszusammenhang lernt und nicht die Rechnung des Simulators nachvollzieht.

Als weiteres Feature kommt die Strippertemperatur hinzu. Sie wird durch eine Vorwärtsselektion gestützt, die die Größen nacheinander nach ihrem Zugewinn an Modellgüte aufnimmt. Zugleich ist sie der Wirkort des Kalibrierdrifts, der über sie in das Modell gelangt. Dass sie diese zweite Rolle übernehmen kann, beruht auf den drei in Abschnitt 3.2.1 genannten Eigenschaften. Sie wird aufgezeichnet, sie gehört keinem Regelkreis an und sie tritt in der Kostenrechnung nicht auf. Prozess und Kosten bleiben deshalb unberührt, sodass die Differenz zwischen den Fehlerverläufen beider Läufe allein aus dem Modell stammt. Unter den Features reagiert sie überdies am empfindlichsten auf eine Verfälschung ihres Messwerts. Mit der Separatortemperatur wäre eine weitere Größe als Wirkort in Frage gekommen, die über die Betriebspunkte nahezu denselben Verlauf nimmt, auf eine Verfälschung ihres Messwerts jedoch schwächer reagiert und als Wirkort daher ausscheidet. Die Auswahl ruht damit auf der Kostenrechnung und dem Aufbau des Prozesses und bleibt vom Umfang der Kampagne unabhängig.

Trainiert wird auf 60 ausgewählten Betriebspunkten und damit nicht auf dem gesamten Bestand der 136 fahrbaren Punkte. Diese Teilmenge bildet den Umfang ab, den ein Versuchsprogramm in der Praxis zeitlich hergibt, und hält damit den Bezug zum Mühlen-Anwendungsfall, in dem der Versuchsplan ebenfalls nur einen Teil des später gefahrenen Betriebsbereichs abdeckt. Die übrigen Punkte tragen die Bewertung auf Betriebszuständen, die das Modell im Training nicht gesehen hat. Mit jedem weiteren Kampagnenpunkt sinkt der nominale Fehler des Modells, zugleich schrumpft jedoch die Zahl der Punkte außerhalb der Kampagne, die diese Bewertung tragen. Die gewählte Größe hält beide Anteile in einem tragfähigen Verhältnis.

Innerhalb dieses Umfangs bleibt zu bestimmen, welche Punkte die Kampagne bilden. Gesetzt sind zunächst die sechs Modusnennwerte, da sie die dokumentierten Betriebspunkte des Prozesses sind und ein Versuchsprogramm von ihnen ausgehen würde. Die übrigen Plätze werden nach demselben Maximin-Kriterium aufgefüllt, das bereits dem Versuchsplan der Generierung

zugrunde liegt, hier jedoch im skalierten Setpoint-Raum über den Bestand der fahrbaren Punkte. Aufgenommen wird also jeweils derjenige Punkt, der am weitesten von allen bereits gewählten entfernt liegt. Die Kampagne ist damit ein Versuchsplan über die sechs Betriebsmodi aus Tabelle 3.6 und deren Umgebung, während die verbleibenden Punkte in den Lücken dazwischen liegen und dort einen Abstand zur Kampagne wahren, der eine Bewertung außerhalb des Trainingsgebiets trägt. Sie entspricht darin dem Referenzdatensatz, der im synthetischen Fall nach dem Latin-Hypercube-Ansatz erzeugt wird, und der Datenselektion in Abschnitt 2.2.3.

Die Vorverarbeitung folgt der des synthetischen Falls in Abschnitt 3.1.6. Die Signale werden mittels Min-Max-Normierung skaliert und anschließend über ein Fenster von 10 Schritten gleitend gemittelt. Die Skalierung wird auf dem Bestand der einzeln angefahrenen Betriebspunkte bestimmt und auf beide Produktionsläufe unverändert angewandt, sodass alle drei Datensätze in derselben Bezugsskala liegen. Als Trainingsdaten dienen die stationären Bereiche der Kampagne, in denen die Betriebspunkte eingeschwungen sind. Ihre Lage ergibt sich unmittelbar aus der Blockstruktur aus Rampe und stationärem Bereich, die alle drei Datensätze teilen, sodass sich dieser Bereich fest vorgeben lässt. Auf einen Ausreißerfilter wird verzichtet, da die Streuung über die Betriebspunkte hier das Nutzsignal trägt und die extremen Punkte gerade den Rand des abgedeckten Betriebsbereichs abbilden.

Als Modell dient wie im synthetischen Fall ein `MLP`, womit die in Abschnitt 2.2.4 getroffene Modellwahl fortgeführt wird. Der `KF` und der `EKF` bleiben demgegenüber auf den synthetischen Datenstrom beschränkt, da sie ein explizit formuliertes Beobachtungsmodell voraussetzen, das beim `TEP` mit der im Simulator hinterlegten Kostenrechnung nicht vorliegt. Der Aufbau des Netzes folgt dem des synthetischen Falls aus Abschnitt 3.1.6. Es besteht aus zwei Hidden-Layern mit je 12 Neuronen und Tanh-Aktivierung sowie einem linearen Ausgabeneuron. Trainiert wird mit dem Adam-Optimierer und dem `MSE` als Verlustfunktion über maximal 200 Epochen bei einer Batchgröße von 200. Die Schichten fallen dabei sogar schmaler aus als im synthetischen Fall, obwohl der zugrunde liegende Prozess ungleich umfangreicher ist. Denn die Modellierung zur Betriebspunktoptimierung bleibt eine Regression weniger Features auf eine skalare Zielgröße, wobei lediglich eingeschwungene Zustände berücksichtigt werden. Die Prozessdynamik bleibt außen vor (vgl. Abschnitt 2.2.4).

Validiert wird auf den Anfahrrampen der Kampagnenpunkte. Da die Zahl der Betriebspunkte gering ist, bleiben auf diese Weise alle Punkte für das Training erhalten. Die Rampen liegen außerhalb der stationären Bereiche und stellen damit Betriebszustände dar, die dem Modell im Training nicht begegnen. Als eigenständige Validierungsdaten sind sie dennoch geeignet, da sie zwischen dem Ausgangszustand und dem jeweiligen Betriebspunkt verlaufen und damit denselben Zusammenhang unter Zwischenzuständen abbilden. Über den Validierungsfehler bricht ein Early Stopping das Training ab und greift auf die Gewichte der besten Epoche zurück.

Als Bewertungsgröße dient der Simulationsfehler, also die Abweichung der tatsächlichen Betriebskosten des Simulators von der Prädiktion des Modells. Er wird auf dem nominalen Lauf

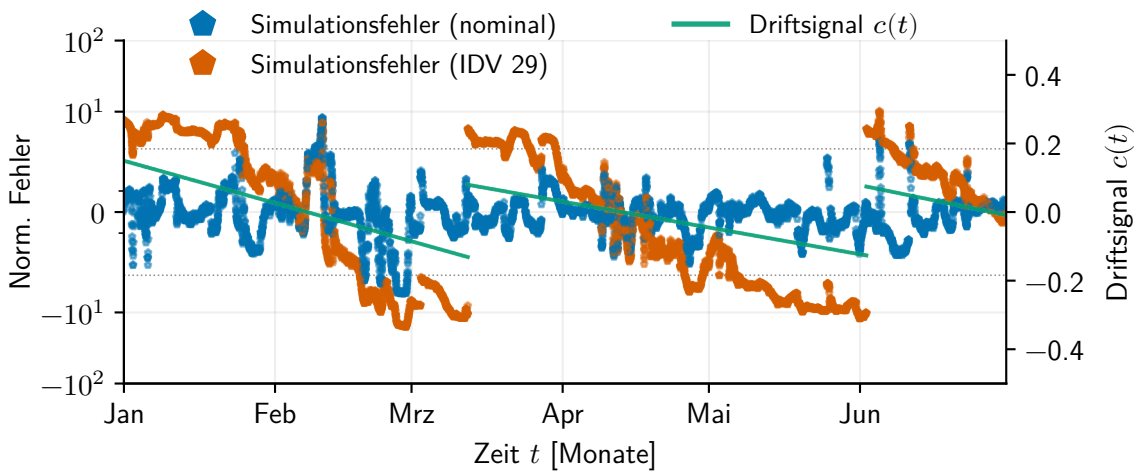


Abbildung 3.14: Normierter Simulationsfehler auf dem nominalen und dem drift-behafteten Datenstrom im Vergleich zum Driftsignal $c(t)$

und auf dem Fehlerlauf über den gesamten Datenstrom ausgewertet. Beide Verläufe werden mit Mittelwert und Streuung des nominalen Fehlers normiert. Als Bezugsgröße tritt das Driftsignal $c(t)$ hinzu. Da der Sensordrift den eigentlichen Prozess nicht berührt, entspricht das Verhältnis der aufgezeichneten Strippertemperatur beider Läufe dem Kalibrierfaktor.

Abbildung 3.14 stellt den normierten Simulationsfehler des nominalen und des drift-behafteten Laufs über einem Ausschnitt des Datenstroms dar, überlagert mit dem Driftsignal $c(t)$. Farbe und Markerform folgen dabei weiter Farb- und Marker-Konvention. Auf dem nominalen Lauf ist zu erkennen, dass der Fehler um Null schwankt, wobei sich seine Streuung abhängig vom Betriebspunkt verändert, die Blockstruktur des Datenstroms jedoch erkennbar bleibt. Die Rampen kommen in den Trainingsdaten nicht vor. Sie tragen dennoch weniger zum Fehler bei, als ihr Anteil am Datenstrom erwarten ließe, denn sie verlaufen zwischen dem Ausgangszustand und dem Betriebspunkt und durchqueren damit bekanntes Gebiet. Auf dem drift-behafteten Lauf treten hingegen Abschnitte mit deutlich größeren Ausschlägen auf, die dem Verlauf des Driftsignals folgen. Die Sägezahnstruktur zeichnet sich dabei nicht unmittelbar im Fehlerverlauf ab, da die Streuung zwischen den Betriebspunkten den Drifteinfluss überlagert. Der systematische Zusammenhang tritt jedoch in der segmentweisen Auswertung hervor, in der die Differenz beider Fehlerverläufe dem Driftsignal über alle vollständigen Segmente hinweg folgt.

Tabelle 3.7 fasst die Kennzahlen des Simulationsfehlers zusammen, aufgeteilt nach den Betriebspunkten der Kampagne und den übrigen Punkten des Datenstroms. Ein Teil der Kampagnenpunkte ist im Datenstrom dabei nicht enthalten, da er dort im zweiten Durchgang ausgefallen ist, und geht deshalb allein in das Training ein, nicht aber in die Bewertung. Der Aufbau entspricht jedoch dem des synthetischen Falls in Tabelle 3.3, ergänzt um das Fehlerverhältnis aus drift-behaftetem und nominalem `RMSE`. Die Kennzahlen sind dabei in Anteilen der Kostenspanne der Kampagne angegeben und nicht in der Normierung der Abbildung. Über alle Betriebspunkte hinweg steigt der `RMSE` vom nominalen zum drift-behafteten Lauf deutlich

Tabelle 3.7: Kennzahlen der drift-bedingten Modellverschlechterung

Betriebspunkte	RMSE		Streuung σ		Mittelwert $\bar{e}$ drift-behaftet	Fehler-verhältnis
	drift-frei	drift-behaftet	drift-frei	drift-behaftet		
Alle	$0{,}013$	$0{,}057$	$0{,}013$	$0{,}056$	$-0{,}009$	$4{,}337$
Kampagne	$0{,}010$	$0{,}060$	$0{,}010$	$0{,}060$	$0{,}003$	$6{,}217$
Übrige	$0{,}015$	$0{,}054$	$0{,}015$	$0{,}051$	$-0{,}018$	$3{,}563$

an. Getragen wird der Anstieg nahezu vollständig von der Streuung, während der Mittelwert des Fehlers zwar von Null abweicht, dazu aber nur wenig beiträgt. Damit wiederholt sich das Bild, das das MLP bereits auf dem synthetischen Datenstrom gezeigt hat.

Die Aufteilung nach der Zugehörigkeit zur Kampagne trennt davon einen zweiten Beitrag ab. Auf den Punkten der Kampagne liegt das nominale Fehlerniveau niedriger als auf den übrigen, da das Modell diese Betriebspunkte im Training bereits gesehen hat. Da der Drift das Fehlerniveau beider Gruppen anhebt, fällt das Fehlerverhältnis auf den Kampagnenpunkten mit ihrem niedrigeren Ausgangsniveau am größten aus. Wie weit der Drift im Simulationsfehler sichtbar bleibt, hängt damit davon ab, wie weit ein Betriebspunkt von der Kampagne entfernt liegt.

Bestimmt man im Merkmalsraum den Abstand jedes Betriebspunkts zum nächstgelegenen Kampagnenpunkt, so misst dieser, wie weit der Punkt im Eingangsraum des Modells von den Trainingsdaten entfernt liegt. Über den abgedeckten Abstandsbereich hinweg wächst der nominale Fehler um Faktor 2,4. Die drift-bedingte Verschiebung wächst dagegen nicht gleichmäßig mit, sondern schwankt zwischen den Abständen. Sie übersteigt den nominalen Fehler jedoch auf nahen wie auf fernen Betriebspunkten, sodass der Drift über den gesamten abgedeckten Bereich sichtbar bleibt, mit zunehmendem Abstand aber weniger deutlich vor dem Fehlerniveau des Modells steht. Über die Sichtbarkeit des Drifts entscheidet somit nicht seine eigene Größe, sondern das Fehlerniveau, vor dem er sich abheben muss.

Der TEP-Datenstrom reproduziert damit die Degradation, die sich zuvor bereits am synthetischen Prozess gezeigt hat. Ausgelöst wird sie hier jedoch nicht durch einen Eingriff in die Systemgleichung, sondern durch einen Sensordrift, der allein den aufgezeichneten Messwert verfälscht. Dass das Modell auch dann an Güte verliert, wenn die Anlage selbst unverändert weiterläuft, entspricht der industriellen Praxis, in der Messstellen verschmutzen und altern. Mit dem TEP liegt damit ein Benchmark unter realistischen Prozessbedingungen vor, an dem sich der Umgang mit CD erproben lässt.

3.2.4 Drift-Erkennung

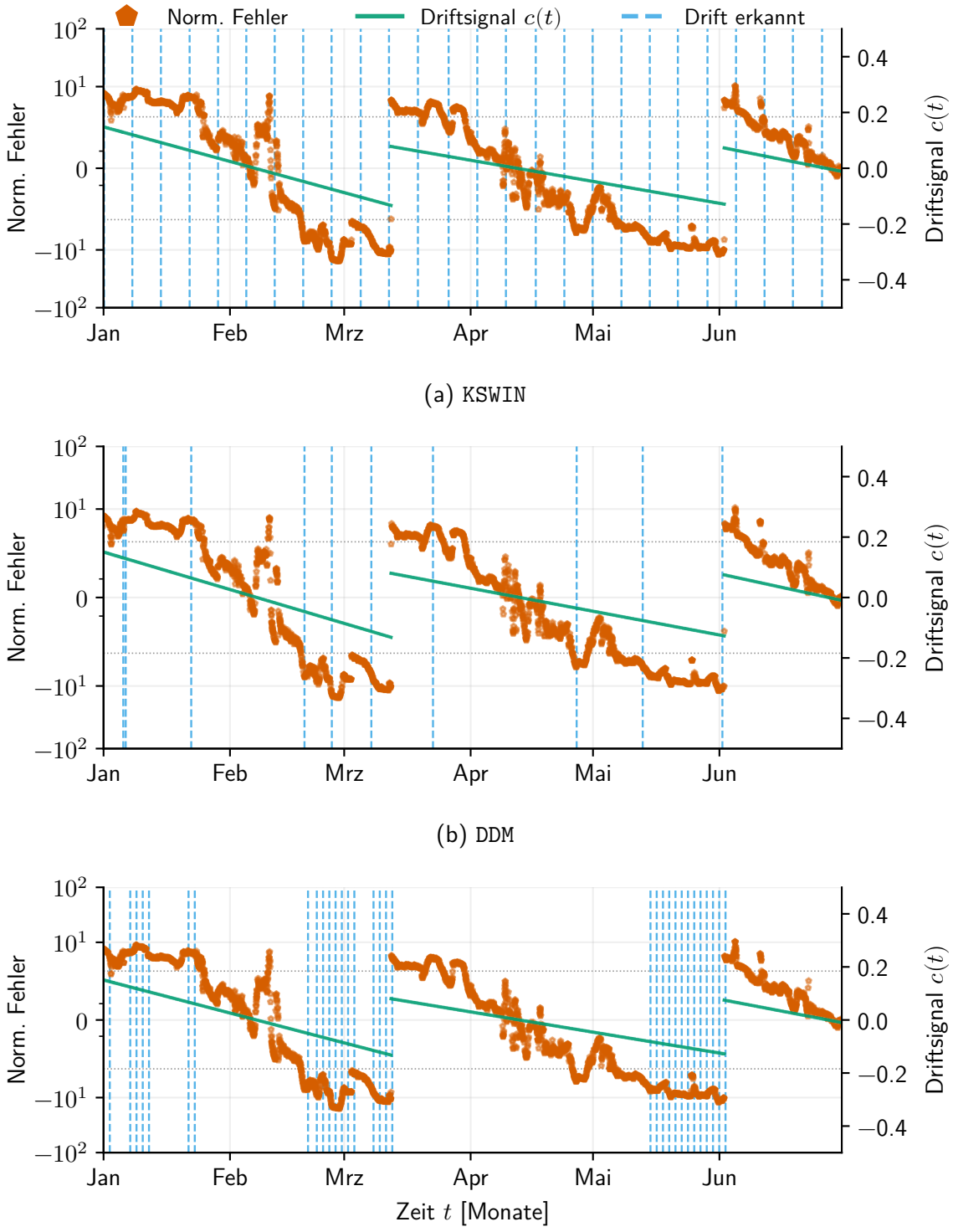
Die Detektion des in Abschnitt 3.2.1 eingeführten CD folgt dem Vorgehen aus Abschnitt 3.1.7. Zum Einsatz kommen dieselben fünf Performance-basierten Detektoren, also die Fenster-basierten Verfahren KSWIN und ADWIN, die Fehlerraten-basierten Verfahren DDM und EDDM sowie der Schwellwert-Detektor. Überwacht wird der Simulationsfehler des in Abschnitt 3.2.3 trainierten MLP auf dem Fehlerlauf, der wie dort mit Mittelwert und Streuung des nominalen Laufs normiert wird. DDM und EDDM verarbeiten ihn wiederum in binarisierter Form über die Schwelle λ_{bin}. Methodisch unterscheidet sich die Detektion damit nicht vom synthetischen Fall, sodass Unterschiede in den Ergebnissen allein auf den Datenstrom zurückgehen.

Die Bewertung der Detektoren stützt sich wie im synthetischen Fall auf die bekannte Ground Truth des Driftsignals. Als Sudden Drifts gelten die sprungartigen Übergänge von $c(t)$ an den Segmentgrenzen (vgl. Abschnitt 3.2.1). Nicht gezählt wird der Beginn der Aufzeichnung, da ihm kein Sprung vorausgeht. Über den Fehlerlauf ergeben sich so 10 Sudden Drifts. Die Unterscheidung zwischen drift-aktiven und stabilen Phasen erfolgt nach Gleichung (3.14), wobei der Anteil drift-aktiver Zeitschritte ähnlich hoch ausfällt wie im synthetischen Datenstrom. Recall, Detektionsverzug und Fehlalarme sowie die daraus gebildete Zielfunktion J_{d} gemäß Gleichung (3.15) werden samt ihrer Gewichtung unverändert übernommen, ebenso das Zufallsniveau gemäß Gleichung (3.16) als Bezugsgröße für den Recall.

Das Toleranzfenster n_{tol} wird wie im synthetischen Fall als Anteil am mittleren Abstand zweier Sudden Drifts festgelegt. Dadurch, dass der TEP-Datenstrom gröber abgetastet ist als der synthetische, resultiert daraus jedoch eine andere Zahl an Abtastschritten. Da beide Driftsignale derselben nominellen Segmentdauer folgen, ergibt sich mit 15 Stunden ein nahezu gleich langes Fenster wie im synthetischen Fall, sodass die Detektionsgüte beider Datenströme auf derselben zeitlichen Grundlage bewertet wird. Die Parametrierung der Detektoren folgt mit der Optimierung auf dem ausgedünnten Strom und der anschließenden Nachrechnung der besten Kandidaten auf dem vollständigen Datenstrom dem in Abschnitt 3.1.7 beschriebenen Vorgehen.

Der wesentliche Unterschied zum synthetischen Fall liegt im Aufbau des Datenstroms. Er setzt sich aus Blöcken einzeln angefahrener Betriebspunkte zusammen (vgl. Abschnitt 3.2.2), und das Fehlerniveau des Modells wechselt mit jedem Block. Hinzu kommt, dass der Kalibrierfehler multiplikativ wirkt und sein Beitrag zum Simulationsfehler damit selbst vom Betriebspunkt abhängt. Wie in Abschnitt 3.2.3 gezeigt, zeichnet sich die Sägezahnstruktur des Driftsignals deshalb nicht unmittelbar im Fehlerverlauf ab, sondern erst in der segmentweisen Auswertung. Der drift-bedingte Fehler liegt im TEP somit in derselben Größenordnung wie die Streuung zwischen den Betriebspunkten. Ein Sudden Drift muss sich für einen Detektor hier also gegen diese Streuung abheben. Zu erwarten ist daher, dass die Detektoren im TEP deutlich schwerer auf die Sudden Drifts ansprechen als im synthetischen Fall.

Abbildung 3.15 stellt die erkannten Drifts der Parameter-optimierten Detektoren dem Driftsi-



(a) `KSWIN`

(b) `DDM`

(c) Schwellwert-Detektor

Abbildung 3.15: Erkannte Drifts ausgewählter Detektionsmethoden auf dem normierten Simulationsfehler im Vergleich zum Driftsignal $c(t)$

gnal $c(t)$ gegenüber. Gezeigt ist KSWIN als Fenster-basiertes Verfahren (a), DDM als Fehlerraten-basiertes Verfahren (b) sowie der Schwellwert-Detektor (c). ADWIN verhält sich qualitativ wie KSWIN, während EDDM noch seltener meldet als DDM. Ausgewertet wird der gesamte Fehlerlauf über rund zwei Jahre, von dem die Abbildung exemplarisch die ersten sechs Monate mit den ersten beiden Sudden Drifts zeigt. Für die Darstellung der Fenster-basierten Verfahren wird eine Refraktärzeit von 165 Stunden genutzt. Der Schwellwert-Detektor meldet im TEP so selten, dass er ungefiltert dargestellt wird. Die Kennzahlen in Tabelle 3.8 sind weiterhin auf der vollständigen Detektionsfolge ohne Refraktärzeit berechnet.

KSWIN und ADWIN erkennen sämtliche Sudden Drifts innerhalb des Toleranzfensters und mit geringem Verzug. Anders als im synthetischen Fall lässt sich daraus jedoch keine Aussage über den Drift ableiten. Beide Verfahren melden über den gesamten Datenstrom hinweg nahezu ununterbrochen, in stabilen ebenso wie in drift-aktiven Phasen, auf den Rampen ebenso wie in den stationären Bereichen der Betriebspunkte. Auch mit Refraktärzeit melden sie jeweils unmittelbar nach deren Ablauf erneut, sodass die Markierungen in Abbildung 3.15a ein gleichmäßiges Raster bilden und keinen Bezug zum Verlauf von $c(t)$ erkennen lassen. Die Fenster-basierten Verfahren reagieren also auf die Schwankungen des Fehlers, die den Drift überlagern. Ihr vollständiger Recall ergibt sich damit allein aus der Dichte ihrer Meldungen.

DDM und EDDM melden dagegen nur selten und in stabilen Phasen überhaupt nicht. DDM erfasst dabei einen kleinen Teil der Sudden Drifts, zwar häufiger als eine zufällige Folge gleich vieler Meldungen, aber bei weitem nicht vollständig. EDDM erkennt keinen der Sudden Drifts. Ursache ist wie im synthetischen Fall die Binarisierung über λ_{bin} (vgl. Abschnitt 3.1.7), die im TEP jedoch zwei gegensätzlichen Anforderungen genügen muss. Liegt die Schwelle niedrig, überschreitet bereits die Streuung zwischen den Betriebspunkten die Schwelle, und die Fehlerrate schwankt ohne Bezug zum Drift. Liegt sie hoch, hebt ein Sudden Drift den Fehler nur an wenigen Betriebspunkten über die Schwelle. Die Optimierung löst diesen Zielkonflikt zugunsten weniger Fehlalarme und nimmt dafür einen geringen Recall in Kauf.

Der Schwellwert-Detektor ist das einzige Verfahren, dessen Meldungen deutlich über dem Zufallsniveau mit den Sudden Drifts zusammenfallen. Er erkennt etwa die Hälfte von ihnen, und zwar überwiegend jene, nach denen der Drift den Simulationsfehler besonders stark anhebt. Anders als DDM und EDDM wertet er nicht aus, wie häufig einzelne Fehler eine Schwelle überschreiten, sondern vergleicht den über ein gleitendes Fenster gemittelten Fehlerbetrag mit einem festen Schwellwert. Kurzzeitige Schwankungen des Fehlers werden dadurch geglättet, während eine anhaltende Anhebung unabhängig vom bisherigen Verlauf zu einer Meldung führt. In stabilen Phasen meldet er daher nicht. Seine übrigen Meldungen fallen in Abschnitte, in denen der Betrag des Driftsignals groß ist und der Fehler über längere Zeit erhöht bleibt. Da der Detektor nach jeder Meldung zurückgesetzt wird und erst mit neu gefülltem Fenster wieder anschlagen kann, folgen die Detektionen dort im Abstand der Fensterlänge aufeinander. Die Mittelung über das Fenster bedingt zugleich den höchsten Verzug der Verfahren, der aber innerhalb des

Tabelle 3.8: Kennzahlen der fünf Drift-Detektoren auf dem normierten Simulationsfehler des Fehlerlaufs

Detektor	Recall ρ	Zufallsniveau ρ_{zuf}	Verzug $\bar{d}$ in h	Fehlalarme N_{fa}	Detektionen
KSWIN	1,00	0,95	1,1	3442	3473
ADWIN	1,00	0,76	1,4	1647	1663
DDM	0,20	0,03	2,5	31	33
EDDM	0,00	0,00	–	4	4
Schwellwert	0,50	0,07	6,1	80	85

Toleranzfensters bleibt.

Tabelle 3.8 fasst für die fünf Detektoren Recall, Zufallsniveau und mittleren Verzug sowie die Zahl der Fehlalarme und der Detektionen über den gesamten Fehlerlauf zusammen. Anders als im synthetischen Fall, in dem alle Verfahren sämtliche Sudden Drifts erkannten, unterscheiden sich die Verfahren hier bereits im Recall. Den vollständigen Recall erreichen allein die Fensterbasierten Verfahren, deren Zufallsniveau jedoch nahe daran heranreicht. Bei DDM bleibt der Recall trotz seines Abstands zum Zufallsniveau gering, bei EDDM fehlt er ganz. Einzig der Schwellwert-Detektor verbindet einen nennenswerten Recall mit einem Zufallsniveau, das weit darunter liegt. Damit trägt im TEP allein der Schwellwert-Detektor eine Information über den Zeitpunkt der Sudden Drifts. Der Verzug spielt für diese Einordnung keine Rolle, da er bei allen Verfahren mit Treffern innerhalb des Toleranzfensters bleibt.

Insgesamt bestätigt sich die Erwartung, dass die Detektoren im TEP deutlich schwerer auf die Sudden Drifts ansprechen als im synthetischen Fall. Ursache ist nicht die Größe des Drifts, sondern wie in Abschnitt 3.2.3 gezeigt das Fehlerniveau, vor dem er sich abheben muss. In einem Datenstrom aus vielen Betriebspunkten überlagert die Streuung zwischen ihnen den Drifteinfluss so weit, dass Verfahren, die auf Veränderungen der Fehlerverteilung oder der Fehlerrate reagieren, ihre Aussagekraft über den Zeitpunkt eines Sprungs weitgehend verlieren. Robuster erweist sich die einfachste Form der Performance-basierten Erkennung, die den gemittelten Fehlerbetrag unmittelbar mit einem festen Schwellwert vergleicht. Ihre Meldungen außerhalb der Toleranzfenster zählen dabei zwar als Fehlalarme, markieren aber Abschnitte anhaltend erhöhten Fehlers und damit gerade jene Stellen, an denen ein Nachtraining des Modells ansetzen würde.

3.2.5 Drift-Adaption

Die Adaption des in Abschnitt 3.2.3 trainierten `MLP` an den Kalibrierdrift folgt dem Vorgehen aus Abschnitt 3.1.8. Gegenübergestellt werden dieselben drei Strategien, also die passive, die aktive und die kombinierte Adaption, jeweils ohne und mit Recurrence-Selektor und im Vergleich zum Modell ohne Adaption. Allen liegt dasselbe inkrementelle Nachtraining zugrunde, dessen Updates verworfen werden, sobald sie den Fehler auf dem eigenen Trainingsfenster nicht senken. Der Fehlerlauf wird chunk-weise verarbeitet, wobei jeder Datenpunkt zunächst prädiziert wird und erst danach in ein Nachtraining einfließen kann. Als Features dienen die Größen aus Abschnitt 3.2.3, nun jedoch aus dem Fehlerlauf einschließlich der drift-behafteten Strippertemperatur. Als Label dienen weiterhin die Betriebskosten. Sie bleiben vom Kalibrierfehler unberührt und lassen sich als Rechengröße aus dem Messvektor fortlaufend bilden. Den Trigger der aktiven und der kombinierten Strategie bestimmt ein Screening der Detektoren. Die Hyperparameter aller Strategien werden gegen die Zielfunktion nach Gleichung (3.17) optimiert, die Parameter des Selektors wie im synthetischen Fall je Strategie bei festgehaltenen übrigen Parametern. Bewertet werden alle Varianten auf dem voll aufgelösten Strom über 50 Seeds.

Der Datenstrom des `TEP` erstreckt sich über rund zwei Jahre und wird für die Adaption auf das Adaptionsintervall $T_{\mathrm{a}} = 1800\,\mathrm{s}$ reduziert. Das ergibt $35\,137$ Datenpunkte, also einen Umfang in ähnlicher Größenordnung wie im synthetischen Fall. Ein Chunk umfasst einen Tag und damit 48 Datenpunkte. Das Tuning läuft auf einem ausgedünnten Strom mit $T_{\mathrm{a}} = 10\,800\,\mathrm{s}$, auf dem ein Chunk ebenfalls einen Tag abdeckt. Die Refraktärzeit bemisst sich an der mittleren Dauer stationärer Betriebsphasen. Im `TEP` entspricht diese einem Block, in dem ein Betriebspunkt angefahren und gehalten wird, und umfasst damit mehrere Tage. Da die Refraktärzeit als dimensionsloses Vielfaches dieser Dauer optimiert wird, überträgt sie sich unverändert vom Tuning- auf den Zielstrom. Schon ein kleines Vielfaches sperrt ein weiteres Sofort-Training damit über mehrere Betriebspunkte hinweg.

Den Trigger der aktiven und der kombinierten Strategie bestimmt ein Screening der fünf Detektoren über fünf Seeds, deren Parameter im geschlossenen Adaptionskreis abgestimmt werden. Den geringsten Fehler erreicht dabei `KSWIN`. `ADWIN` folgt knapp dahinter, überschreitet den zulässigen Abstand von 5 % jedoch und gilt damit nicht als gleichwertig. `KSWIN` kommt somit als Trigger für die aktive und die kombinierte Strategie zum Einsatz. Die Fehlerraten-basierten Verfahren `DDM` und `EDDM` bleiben deutlich zurück. Der Schwellwert-Detektor versagt dagegen im geschlossenen Kreis, obwohl er in der reinen Detektion als einziges Verfahren Information über die Zeitpunkte der Sudden Drifts trägt (vgl. Abschnitt 3.2.4). Das Tuning findet für ihn nur Konfigurationen mit sehr wenigen Nachtrainings, die je nach Seed wirkungslos bleiben oder das Modell deutlich verschlechtern. Eine naheliegende Erklärung liegt in seiner festen Schwelle, die nur einem Fehlerniveau entsprechen kann (vgl. Abschnitt 3.1.8). Liegt sie auf dem Niveau vor der Adaption, spricht er nach dem ersten wirksamen Nachtraining kaum noch an.

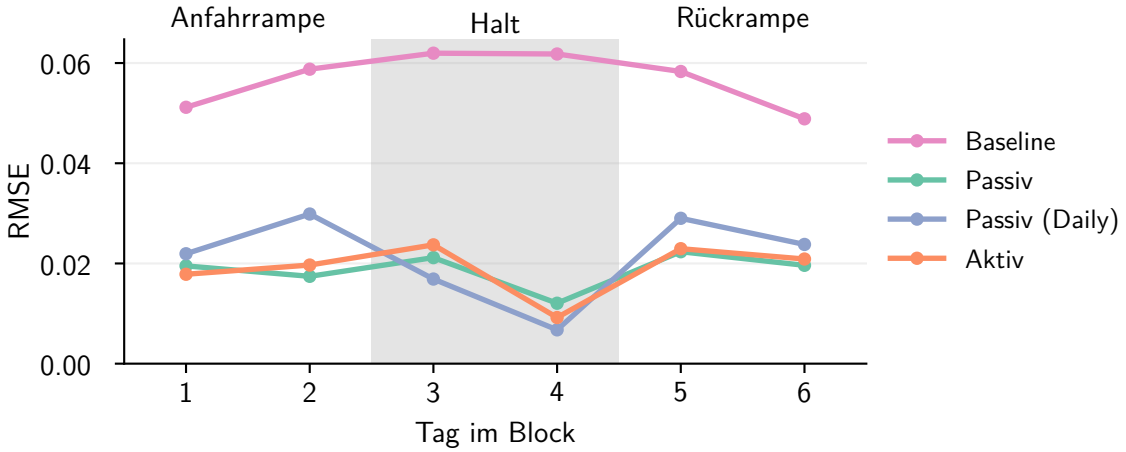


Abbildung 3.16: Mittlerer RMSE je Tag eines Betriebspunkt-Blocks über 50 Seeds

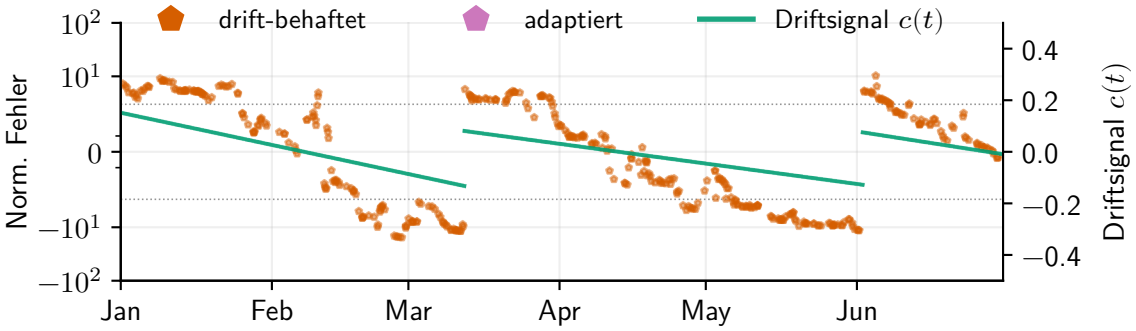
Welche Rolle der Detektor dabei spielt, erschließt sich aus dem Nachführschema der aktiven Strategie (vgl. Abschnitt 3.1.8). Ein erkannter Drift löst dort ein Sofort-Training auf einem zurückliegenden Fenster aus und eröffnet zugleich eine Nachführphase, in der das Modell am Ende jedes Chunks auf den soeben eingetroffenen Daten nachtrainiert wird. Meldungen innerhalb einer laufenden Phase bleiben ohne Wirkung. Im TEP wählt das Tuning eine Nachführphase von mehreren Wochen. KSWIN meldet zudem, wie in Abschnitt 3.2.4 gezeigt, ohne erkennbaren Bezug zum Verlauf von $c(t)$ und so häufig, dass nach dem Ende einer Phase in aller Regel sofort die nächste beginnt. Die Nachführphasen reihen sich damit nahezu lückenlos aneinander, und das Modell wird fast an jedem Tag nachtrainiert. Der Detektor bestimmt also kaum, wann angepasst wird, sondern hält eine dichte Folge von Nachtrainings aufrecht. Die Eigenschaft, die die Fenster-basierten Verfahren in der reinen Detektion als Schwäche auszeichnet, wird im geschlossenen Kreis so zur Grundlage ihres Abschneidens. Das zeigt sich auch darin, dass ADWIN im Screening nur knapp hinter KSWIN liegt.

Weshalb die aktive Strategie die passive so deutlich übertrifft, obwohl sie im TEP ebenfalls nahezu periodisch nachtrainiert, lässt sich vor allem auf die Eingriffsdichte zurückführen. Einen Hinweis darauf gibt Abbildung 3.16. Sie zeigt den Fehler je Tag eines Betriebspunkt-Blocks, gemittelt über alle Blöcke und Seeds. Ergänzt ist eine passive Variante, die mit den Parametern der passiven Strategie an jedem Tag nachtrainiert. Mit dem Anfahren eines neuen Betriebspunkts steigt der Fehler der passiven Strategie deutlich an und bleibt über den gesamten Halt auf diesem Niveau. Die täglich nachtrainierte Variante steigt dagegen nur leicht an und baut den Fehler im Laufe des Halteintervalls wieder ab. Auf den Rampen liegt die täglich nachtrainierte Strategie dagegen über der aktiven Strategie, vermutlich weil sie sich zu stark an den zuletzt gehaltenen Betriebspunkt anpasst. Die aktive Strategie verbindet das tägliche Nachtraining mit einer kleinen Lernrate und gelegentlich längeren Trainingsfenstern und verläuft über den ganzen Block gleichmäßig niedrig. Hier zeigt sich das Stabilitäts-Plastizitäts-Dilemma aus Abschnitt 2.3.3 im Verlauf eines Betriebspunkts. Offen bleibt, weshalb die Optimierung der passiven Strategie keine vergleichbar dichte Einstellung wählt, obwohl bereits die täglich nach-

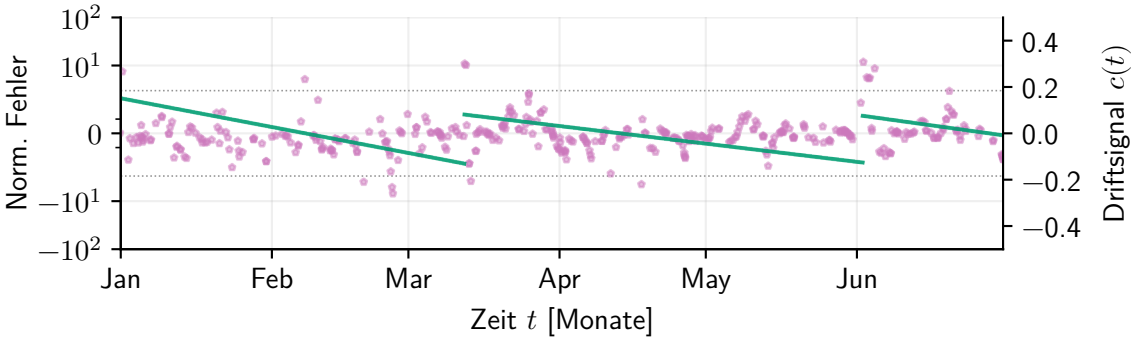
Tabelle 3.9: Kennzahlen der Adaptionsstrategien auf dem drift-behafteten Strom, als Mittelwert $\pm$ Standardabweichung über 50 Seeds

Strategie	RMSE	Mittelwert $\bar{e}$	Streuung σ	Korr. $\lvert e\rvert, \lvert c\rvert$	Nachtrainings
Baseline	$0{,}057 \pm 0{,}000$	$-0{,}009 \pm 0{,}000$	$0{,}056 \pm 0{,}000$	$0{,}82 \pm 0{,}00$	$0{,}0 \pm 0{,}0$
Passiv	$0{,}019 \pm 0{,}000$	$0{,}002 \pm 0{,}000$	$0{,}019 \pm 0{,}000$	$0{,}05 \pm 0{,}01$	$732{,}0 \pm 0{,}0$
Passiv + Rec	$0{,}017 \pm 0{,}000$	$0{,}000 \pm 0{,}000$	$0{,}017 \pm 0{,}000$	$0{,}06 \pm 0{,}01$	$732{,}0 \pm 0{,}0$
Aktiv	$0{,}020 \pm 0{,}001$	$0{,}001 \pm 0{,}000$	$0{,}020 \pm 0{,}001$	$0{,}10 \pm 0{,}02$	$716{,}7 \pm 3{,}5$
Kombiniert	$0{,}019 \pm 0{,}001$	$0{,}001 \pm 0{,}000$	$0{,}019 \pm 0{,}001$	$0{,}06 \pm 0{,}02$	$720{,}2 \pm 3{,}1$

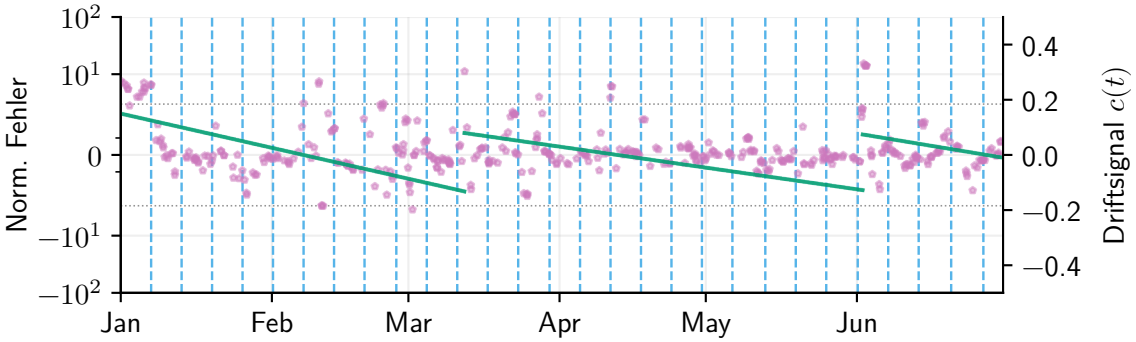
trainierte Variante einen deutlich geringeren Fehler erreicht.



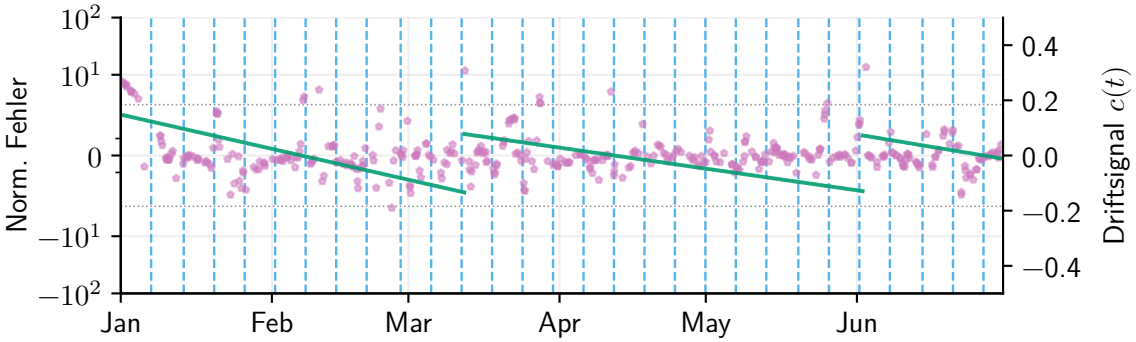
(a) Baseline (keine Adaption)



(b) Passive Adaption mit Recurrence-Selektor



(c) Aktive Adaption



(d) Kombinierte Adaption

Abbildung 3.17: Normierter Simulationsfehler ausgewählter Adaptionsstrategien auf dem driftbehafteten Datenstrom im Vergleich zum Driftsignal $c(t)$

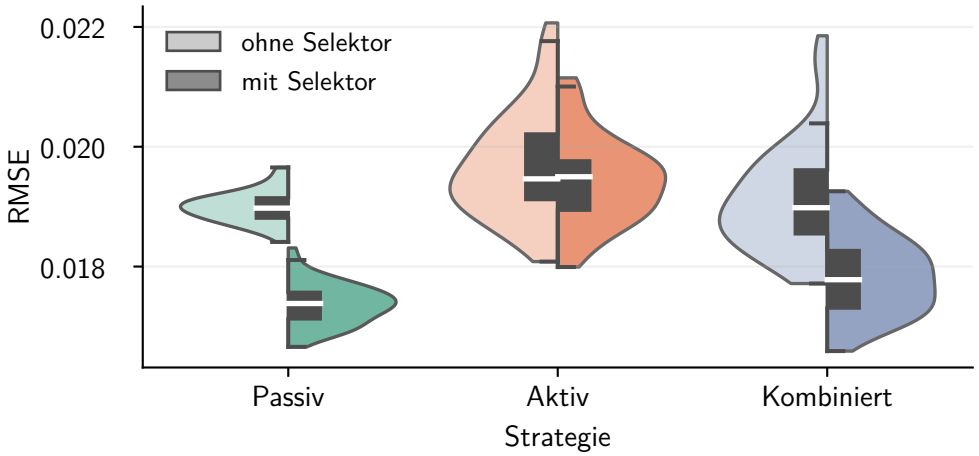


Abbildung 3.18: Verteilung des RMSE der adaptiven Strategien über 50 Läufe mit unterschiedlichen Seeds

4 Anwendung

Ausgangspunkt dieser Arbeit ist die Beobachtung, dass der Modellfehler der spezifischen Energie an einer VRM im Produktionsbetrieb nicht konstant bleibt, sondern über die Zeit schwankt, wie bereits in Abbildung 1.2 dargestellt. Ein Modell, dessen Güte sich im Betrieb unbemerkt verändert, eignet sich jedoch nur begrenzt als Grundlage einer Betriebspunktoptimierung. Aus dieser Beobachtung sind die Leitfragen in Abschnitt 1.2 hervorgegangen. Dieses Kapitel kehrt an genau diese Anlage und zu genau diesem Modell zurück. Es geht zunächst der zweiten Leitfrage nach, welche Form von CD an der Anlage vorliegt, und untersucht im Sinne der dritten Leitfrage anschließend, welche messbaren Vorteile die Adaption des Modells im realen Betrieb bringt.

Die in Kapitel 3 erprobte Verarbeitungskette ist bewusst nicht auf die Mühle zugeschnitten, sondern als allgemeine Lösung eingeführt, die sich nun auch am Anwendungsfall bewähren soll. Aufbau und Verfahren der Drift-Erkennung und Drift-Adaption bleiben daher erhalten. Gegenstand sei dabei jetzt die in Abschnitt 2.1.1 beschriebene VRM, deren Betriebspunkte sich nach Abschnitt 2.1.2 über die von den Operatoren vorgegebenen Setpoints definieren. Das verwendete Modell wurde entsprechend der Datenselektion aus Abschnitt 2.2.3 auf den quasi-stationären Betriebspunkten eines Feldexperiments trainiert. Dieses Feldexperiment hält den Zustand der Anlage zu einem festen Zeitpunkt fest und bildet damit den Bezug, gegenüber dem sich jede spätere Veränderung des Prozessverhaltens abhebt.

An die Stelle der Generierung eines Datenstroms tritt an der realen Anlage dessen Erhebung aus historischen Betriebsdaten. Der so entstehende Strom erstreckt sich über mehrere Jahre und wird durch betriebsbedingte Stillstände vereinzelt unterbrochen. Zudem ist der CD anders als in den generalisierten Fällen weder vorgegeben noch unmittelbar gemessen. Eine Ground Truth, an der sich Erkennung und Adaption direkt bewerten ließen, liegt daher nicht vor. An ihre Stelle treten zwei Referenzen. Die Wartungen der Mühle, die sich über das Kalibriersignal der Walzen erschließen lassen, dienen als Ereignisreferenz für die Drift-Erkennung. Ein zweites Feldexperiment, deutlich später als das erste durchgeführt, dient als Generalisierungsreferenz für die Drift-Adaption. Es zeigt, ob ein adaptiertes Modell auch über den im Betrieb gefahrenen Bereich hinaus gültig bleibt. Beide Feldexperimente dienten bereits in [Ig24] als Grundlage einer eingehenden Untersuchung des CD an dieser Anlage, auf der aufbauend [Ma25] Form des Drifts und Nutzen der Adaption betrachtet. An die Befunde beider Arbeiten knüpft dieses Kapitel an.

Das Kapitel führt schrittweise vom Prozess und seinen Daten hin zur Erkennung und zur

Adaption des CD. Zunächst beschreibt Abschnitt 4.1 die möglichen Ursachen von CD an der Anlage und welche davon messtechnisch erfasst sind. Abschnitt 4.2 behandelt die Erhebung des Datenstroms und seine Verdichtung zu Betriebspunkten. In Abschnitt 4.3 wird das Modell auf die erhobenen Daten angewandt und sein Fehler über den Betrachtungszeitraum bewertet. Die Kontextanalyse in Abschnitt 4.4 erschließt Form und Ursachen des CD aus Betriebs- und Wartungsdaten und leitet daraus die Ereignisreferenz ab. Darauf bauen die Drift-Erkennung in Abschnitt 4.5 und die Drift-Adaption in Abschnitt 4.6 auf.

4.1 Concept Drift

Als Ursachen für CD kommen an der VRM mehrere Einflüsse in Frage, die sich in ihrer Wirkung auf den Mahlprozess und in ihrem zeitlichen Verlauf unterscheiden. Naheliegend ist der Verschleiß der in Abschnitt 2.1.1 beschriebenen Verschleißteile. Mit fortschreitendem Abtrag an Walzen und Mahlteller verändert sich die Geometrie der Mahlzone und damit die Bedingungen, unter denen das Mahlgut zerkleinert wird. Diesem stetigen Verlauf stehen Wartungen gegenüber, die den Zustand der Anlage sprunghaft verändern können. Im Zuge einer Wartung werden zahlreiche Arbeiten an der Anlage durchgeführt, die anhand der Daten nicht nachzuvollziehen sind. Dazu zählt der Tausch verschlissener Walzenmäntel oder deren Wiederaufbau durch Aufschweißen. Besonders weitreichend war der Umbau des Sichters im September 2024, bei dem unter anderem das Sichtergetriebe ersetzt wurde. Da sich die Anlage danach grundlegend von der des Feldexperiments unterscheidet, endet der betrachtete Datenstrom vor diesem Umbau.

Neben dem Zustand der Anlage kann sich auch das Aufgabematerial über die Zeit verändern. Anders als viele Zementhersteller, die auf derselben Mühle mehrere Produkte vermahlen, stellt der Betreiber der betrachteten Anlage ausschließlich gemahlene Hochofenschlacke her. Die Schlacke stammt jedoch aus mehreren Herkunftsländern und wird per Schiff angeliefert, sodass sich die Lieferungen in Eigenschaften wie Feuchte und Feinheit unterscheiden können. Hinzu kommt die Lagerung vor Ort, die teils überdacht und teils unter freiem Himmel erfolgt und damit insbesondere die Feuchte des Materials zusätzlich beeinflusst. Schließlich können auch die Fahrweise der Operatoren und äußere Bedingungen das Prozessverhalten über die Zeit verändern.

Nur ein Teil dieser Einflüsse ist messtechnisch erfasst. Die Lage der vier Walzen über dem Mahlteller wird fortlaufend gemessen und gibt im Betrieb die Höhe des Mahlbetts wieder. Mit zunehmendem Abtrag an Walzen und Mahlteller verschiebt sich jedoch der Nullpunkt dieser Messung. Neu gesetzt wird er bei stehender Mühle mit aufliegenden Walzen, was nach jeder Wartung, aber auch bei kürzeren Eingriffen geschieht. Da eine Wartung stets einen längeren Stillstand erfordert, lässt sie sich über die Kalibrierung und die Dauer des vorausgehenden Stillstands im gesamten Datenstrom erkennen. Für einen Teil des Betrachtungszeitraums sind die größeren Wartungen zusätzlich in einer Liste dokumentiert.

Über die Durchsatzmessung lässt sich zudem die seit der letzten Wartung vermahlene Menge bestimmen. Vom Aufgabematerial wird daneben allein die Feuchte erfasst, die einerseits zu einem ruhigen Mahlbetrieb beitragen kann, andererseits aber durch Heißgas wieder entzogen werden muss, um eine möglichst geringe Restfeuchte des Endprodukts zu gewährleisten. Reicht die Feuchte des Materials für einen ruhigen Lauf nicht aus, wird als Gegenmaßnahme Wasser eingedüst, dessen Menge ebenfalls gemessen wird. Zusammensetzung und Feinheit des Materials sowie Herkunft und Lagerung einzelner Lieferungen liegen dagegen nicht vor. Sie wirken damit als Hidden Context nach Abschnitt 2.3.1, der den Prozess beeinflusst, ohne im Modell erfasst zu sein. Die Fahrweise der Operatoren ist über die Setpoints vollständig erfasst.

Bei unveränderten Setpoints verändert sich mit dem Zustand von Walzen und Mahlteller der Zusammenhang zwischen Betriebspunkt und spezifischer Energie, also $P_t(y \mid X)$. Nach Tabelle 2.3 ist durch den Verschleiß daher ein Real Concept Drift zu erwarten. Phänomenologisch entspricht der fortschreitende Abtrag einem Incremental Drift nach Abbildung 2.5, während Wartungen einen Sudden Drift hervorrufen. Auch Veränderungen des Aufgabematerials wirken auf diesen Zusammenhang. Da Wartungen die Anlage in einen ähnlichen Ausgangszustand zurückversetzen, können sich frühere Verschleißzustände wiederholen, was einem Recurring Drift entspricht. Streng genommen kehrt ein Concept jedoch nur dann wieder, wenn neben dem Verschleißzustand auch die übrigen Einflüsse wie das Aufgabematerial übereinstimmen, sodass eine Wiederkehr allenfalls näherungsweise zu erwarten ist. Die Fahrweise der Operatoren verschiebt dagegen die Verteilung der Betriebspunkte und damit $P_t(X)$, was einem Covariate Shift entspricht. Dieser lässt den Zusammenhang selbst unberührt, kann den Modellfehler aber dennoch erhöhen, wenn Bereiche angefahren werden, die im Feldexperiment kaum abgedeckt waren.

4.2 Datenerhebung

Grundlage des Datenstroms sind die Sensordaten der Anlage, die im laufenden Betrieb fortlaufend aufgezeichnet werden. Wie in Abschnitt 2.2.3 beschrieben, werden die Messgrößen mit einer Abtastrate von $1\,\mathrm{Hz}$ erfasst. In derselben Auflösung liegen auch die Daten der Feldexperimente vor, auf denen das Modell trainiert wurde. Der betrachtete Zeitraum beginnt mit dem ersten Feldexperiment im November 2021. Frühere Daten sind nicht verwertbar, da bis dahin Sensoren teils fehlerhaft konfiguriert waren. Er endet vor dem Umbau des Sichters im September 2024 und umfasst damit 1032 Tage. Der Strom schließt so unmittelbar an den Bezugszustand an, in dem das Modell trainiert wurde.

Um den Strom auf die stationären Zustände zu beschränken, auf denen das Modell arbeitet, wird er zu Betriebspunkten verdichtet. Dabei wird die Regel aus Abschnitt 2.2.3 übernommen, nach der ein Zustand als stationär gilt, wenn nach der letzten Änderung einer Stellgröße alle Regelgrößen für mindestens 10 Minuten unverändert bleiben. Zu Beginn jedes Betriebspunkts

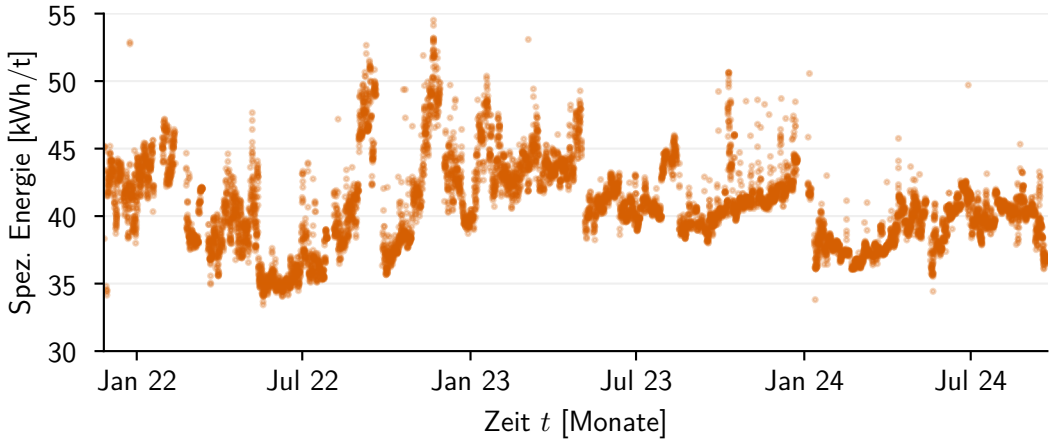


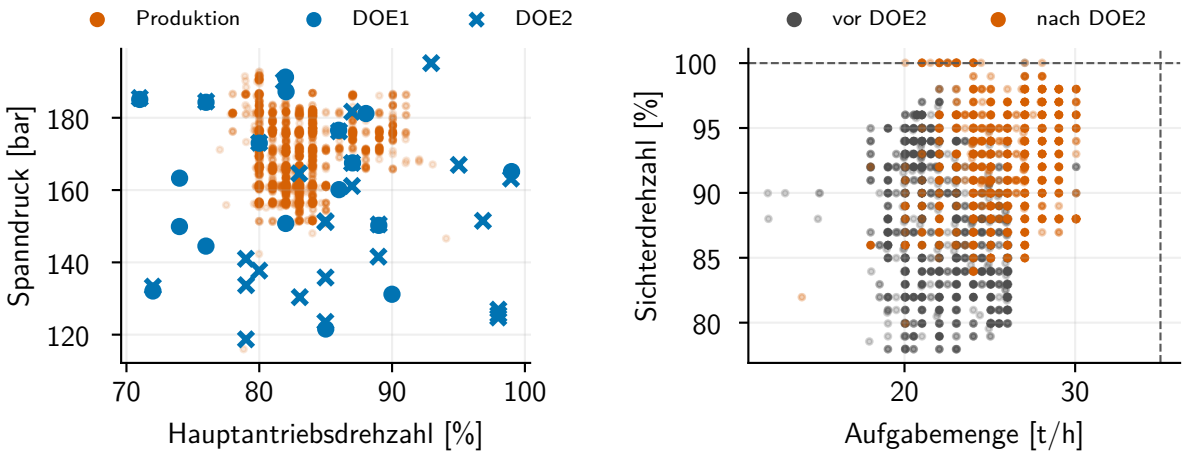
Abbildung 4.1: Spezifische Energie der Betriebspunkte im Produktionsbetrieb

wird zusätzlich eine Einschwingzeit von 5 Minuten verworfen. Länger gehaltene Betriebspunkte werden nach spätestens 120 Minuten geteilt, damit sich Veränderungen innerhalb dieser Zeit nicht in einem einzigen Mittelwert verlieren. Berücksichtigt werden außerdem nur Zeiten, in denen die Mühle läuft, da sich die Setpoints bei einem Stillstand nicht ändern. Die Verdichtung reduziert die große Zahl an Abtastwerten dabei bewusst auf eine überschaubare Folge von Betriebspunkten, deren Mittelwerte weniger Rauschen tragen als die einzelnen Messungen.

Aus der Verdichtung geht ein Strom von 11124 Betriebspunkten hervor. Die Betriebspunkte folgen dabei nicht in festen Abständen aufeinander, sondern dem Verlauf der Produktion. Größere Abstände entstehen bei Stillständen und in Phasen häufiger Setpoint-Änderungen, in denen kaum ein Zustand lange genug gehalten wird, um als stationär zu gelten. Vereinzelt produziert die Mühle über mehrere Tage nicht, sodass auch längere Lücken im Strom auftreten. Ein Betriebspunkt entspricht damit keinem festen Zeitschritt, weshalb zeitliche Abstände im Weiteren in Tagen und nicht in Betriebspunkten bemessen werden.

Abbildung 4.1 zeigt die spezifische Energie der Betriebspunkte über den gesamten Betrachtungszeitraum. Anders als der normierte Modellfehler in Abbildung 1.2 ist hier die gemessene Zielgröße selbst dargestellt. Auffällig ist zunächst, dass sich ihr Niveau über Wochen und Monate deutlich verschiebt. Im Verlauf des Jahres 2022 steigt sie von den niedrigsten bis zu den höchsten Werten des gesamten Zeitraums an. Ab Mitte 2023 verläuft sie ruhiger, fällt zu Beginn des Jahres 2024 sprunghaft ab und nimmt anschließend wieder allmählich zu. Einzelne Abschnitte zeigen damit das Muster aus allmählichem Anstieg und plötzlichem Abfall, das von Verschleiß und Wartung zu erwarten ist. Aus der Zielgröße allein lässt sich ein solcher Drift jedoch nicht ablesen, da sie ebenso von den gefahrenen Betriebspunkten abhängt. Diese Trennung leistet erst der Modellfehler, in dem der Einfluss der Betriebspunkte durch das Modell berücksichtigt ist.

Neben dem Produktionsstrom stehen die beiden Feldexperimente zur Verfügung, die als Referenzen dieses Kapitels dienen. Das erste Feldexperiment im November 2021 liefert mit seinen



(a) Feldexperimente (b) Verschiebung des Betriebs

Abbildung 4.2: Abdeckung des Stellgrößenraums

18 Betriebspunkten die Trainingsdaten des Modells. Rund anderthalb Jahre später, im März 2023, wurde ein zweites Feldexperiment mit 28 Betriebspunkten durchgeführt. Es liegt damit mitten im Betrachtungszeitraum und zeigt die Anlage in einem Zustand, der sich durch die zwischenzeitliche Nutzung vom Bezugszustand des ersten Feldexperiments unterscheidet. Beide Feldexperimente liegen sowohl als verdichtete Betriebspunkte als auch in der vollen Auflösung der Abtastwerte vor. Für das Training werden, wie in Abschnitt 2.2.3 beschrieben, die Abtastwerte der stationären Phasen genutzt, während die Bewertung auf den Betriebspunkten erfolgt.

Abbildung 4.2 zeigt, welche Bereiche des Stellgrößenraums vom Produktionsbetrieb und von den beiden Feldexperimenten abgedeckt werden. Dargestellt sind jeweils zwei Stellgrößen, über denen die Betriebspunkte eingetragen sind. In Abbildung 4.2a sind Hauptantriebsdrehzahl und Spanndruck gegenübergestellt. Der Produktionsbetrieb konzentriert sich auf einen schmalen Bereich mittlerer Drehzahlen und hoher Spanndrücke. Beide Feldexperimente (DOE1, DOE2) erweitern diesen Raum in alle Richtungen deutlich, das zweite reicht dabei zu niedrigen Spanndrücken hin noch etwas weiter als das erste.

Abbildung 4.2b zeigt dagegen nur den Produktionsbetrieb, aufgeteilt in die Zeit vor und nach dem zweiten Feldexperiment (DOE2), über Aufgabemenge und Sichterdrehzahl. Die gestrichelten Linien markieren die Nenngrenzen der Anlage. Nach dem zweiten Feldexperiment verschiebt sich der Betrieb deutlich zu höherem Durchsatz und höheren Sichterdrehzahlen, die teils bis an die Nenngrenze reichen. Damit werden im späteren Teil des Stroms zunehmend Bereiche angefahren, die am Rand des bisherigen Betriebs und der Feldexperimente liegen. Eine solche Verschiebung der Betriebspunkte entspricht dem in Abschnitt 4.1 beschriebenen Covariate Shift.

Tabelle 4.1: Kennzahlen des Simulationsfehlers je Datensatz

Datensatz	RMSE [kWh/t]	Mittelwert $\bar{e}$ [kWh/t]	Streuung σ [kWh/t]	R^2
DOE1	2,23	0,55	2,16	0,87
DOE2	1,99	0,30	1,97	0,58
Produktion	2,26	−0,69	2,15	0,48

4.3 Modellierung

Als Modell dient das statische MLP aus Abschnitt 2.2.4, das auf den Abtastwerten der stationären Phasen von DOE1 neu trainiert wird. Die Vorverarbeitung folgt dem dort beschriebenen Vorgehen mit zwei Abweichungen, die beide der Struktur der Betriebspunkte geschuldet sind. Der Butterworth-Filter wird je Betriebspunkt angewandt, damit Setpoint-Änderungen nicht zu Rampen verändert werden. Zudem werden Trainings- und Validierungsdaten nach Betriebspunkten statt nach Abtastwerten getrennt, da benachbarte Abtastwerte nahezu identisch sind und sonst nur die Interpolation innerhalb eines Betriebspunkts geprüft würde. Architektur und Hyperparameter sind in Anhang B zusammengefasst. Das Modell $f(X)$ schätzt diesen Erwartungswert anhand der Trainingsdaten und damit für den Zeitpunkt t_0 des ersten Feldexperiments.

Bewertet wird das Modell auf drei Datensätzen, deren Kennzahlen Tabelle 4.1 zusammenfasst. DOE1 dient dabei als Bezug, da das Modell auf ihm trainiert wurde. Mittelwert und Streuung des Simulationsfehlers auf diesem Datensatz bilden zugleich die Normierung, mit der der Fehler im Weiteren dargestellt wird. DOE2 prüft die Übertragung auf einem späteren Zustand der Anlage, der Produktionsstrom den laufenden Betrieb. Über alle drei Datensätze bleibt der RMSE auf ähnlichem Niveau. Das Bestimmtheitsmaß fällt auf DOE2 und im Produktionsbetrieb dagegen deutlich ab, sodass das Modell die Unterschiede zwischen den Betriebspunkten dort weniger gut erklärt. Auffällig ist zudem das Vorzeichen des mittleren Fehlers, der auf beiden Feldexperimenten positiv, im Produktionsbetrieb aber negativ ist. Im Betrieb überschätzt das Modell die spezifische Energie also im Mittel. Als Mittel über den gesamten Zeitraum verdecken diese Kennzahlen jedoch, wie sich der Fehler über die Zeit entwickelt.

Wie sich der Fehler über die Zeit entwickelt, zeigt der normierte Simulationsfehler in Abbildung 1.2, die bereits in Kapitel 1 als Ausgangspunkt der Arbeit eingeführt wurde. Anders als die Zielgröße in Abbildung 4.1 folgt er einem deutlich geordneteren Verlauf, da der Einfluss der Betriebspunkte durch das Modell weitgehend berücksichtigt ist. Über den gesamten Zeitraum wiederholt sich dabei ein Muster aus allmählichem Anstieg und plötzlichem Abfall. Innerhalb eines Abschnitts steigt der Fehler über Wochen bis Monate stetig an, die gemessene spezifische Energie liegt also zunehmend über der Vorhersage. An den Grenzen der Abschnitte fällt er da-

gegen sprunghaft auf ein niedrigeres Niveau zurück. Dieses sägezahnartige Muster entspricht dem Zusammenspiel aus Verschleiß und Wartung, wie es in Abschnitt 4.1 beschrieben wurde. Die Ausschläge reichen dabei deutlich über die Streuung auf dem ersten Feldexperiment hinaus. Anders als in Kapitel 3 fehlt jedoch ein ursächliches Driftsignal, an dem sich dieser Verlauf überprüfen ließe, da die Ursache des Drifts an der Anlage nicht gemessen ist.

Über den Betrachtungszeitraum nimmt der Fehler dabei nicht stetig zu. In der zweiten Hälfte des Stroms fällt der RMSE sogar geringer aus als in der ersten, in der die größten Ausschläge liegen. Der Drift äußert sich damit nicht als fortschreitende Verschlechterung des Modells, sondern als wiederkehrende Abweichung, die mit jedem Abfall zumindest teilweise zurückgesetzt wird. Bemerkenswert ist dies, da sich der Betrieb in der zweiten Hälfte zu höherem Durchsatz verschiebt und damit Bereiche anfährt, die am Rand der Feldexperimente liegen. Eine solche Verschiebung ist für sich genommen kein Drift, könnte sich im Fehler aber ähnlich äußern. Dass der Fehler in dieser Phase dennoch kleiner ausfällt, spricht dafür, dass sein Verlauf weniger von der Lage der Betriebspunkte als vom Zustand der Anlage bestimmt wird.

4.4 Kontextanalyse

Der Verlauf des Simulationsfehlers legt nahe, dass Verschleiß und Wartung das Verhalten der Anlage verändern. Da die Ursache des Drifts nicht gemessen ist, lässt sich diese Vermutung am Fehler allein jedoch nicht belegen. Die Kontextanalyse soll Informationen zum Drift daher aus den Betriebs- und Wartungsdaten erschließen, die neben dem Modell zur Verfügung stehen. Sie folgt dabei drei Fragen. Zunächst ist zu klären, ob sich der Zusammenhang zwischen Betriebspunkt und spezifischer Energie tatsächlich verändert oder lediglich die Verteilung der Betriebspunkte verschiebt. Daran schließt die Frage an, welche Ereignisse und Kontextgrößen mit dem Drift einhergehen und damit als Ursache in Betracht kommen. Schließlich wird untersucht, wie der Drift beschaffen ist, ob er also sprunghaft oder allmählich verläuft und die Anlage als Ganzes oder nur einzelne Betriebspunkte betrifft. Als Ereignisse dienen dabei die Wartungen, erkannt an einer Kalibrierung nach mindestens 48 Stunden Stillstand. Für den Zeitraum der Wartungsliste stimmen beide vollständig überein. Die Wartungen lassen sich damit über den gesamten Datenstrom bestimmen.

4.4.1 Nachweis

Eine naheliegende Erklärung für eine schwankende Modellgüte wäre ein unruhiger Betrieb, in dem häufig wechselnde Betriebspunkte den Stellgrößenraum ungleichmäßig abdecken. Die Streuung der Betriebspunkte in einem gleitenden Fenster steht jedoch in keinem Zusammenhang mit dem Betrag des Simulationsfehlers ($\rho = -0{,}02$), sodass sich seine Schwankungen nicht auf einen unruhigen Betrieb zurückführen lassen. Dass sich die Verteilung der Betriebs-

Tabelle 4.2: Kolmogorov-Smirnov-Statistik D je Größe und Datensatzpaar

Größe	DOE1–DOE2	DOE1–Prod.	DOE2–Prod.	vor/nach DOE2
Aufgabemenge	0,20	0,49	0,44	0,53
Sichtergeschwindigkeit	0,16	0,24	0,17	0,45
Spanndruck	0,26	0,39	0,57	0,53
Ventilatorgeschwindigkeit	0,26	0,25	0,29	0,49
Mahltellergeschwindigkeit	0,26	0,37	0,52	0,35
Rezirkulationsklappe	0,72	0,78	0,31	0,40
Kaminklappe	1,00	0,86	0,56	0,31
Last des Heißgaserzeugers	0,83	0,21	0,69	0,20
Spezifische Energie	0,28	0,42	0,42	0,22

punkte dennoch verändert, zeigt ein Vergleich der Datensätze mit dem Kolmogorov-Smirnov-Test aus Abschnitt 2.3.2, der prüft, ob zwei Stichproben derselben Verteilung entstammen. Gegenübergestellt werden die beiden Feldexperimente untereinander, jedes von ihnen mit dem Produktionsstrom sowie der Produktionsstrom vor und nach dem zweiten Feldexperiment. Tabelle 4.2 gibt dafür die Teststatistik D an, also den größten Abstand der empirischen Verteilungsfunktionen. Bei der großen Zahl an zeitlich abhängigen Betriebspunkten weist der Test nahezu jeden Unterschied als signifikant aus. Aussagekräftiger ist deshalb die Größe von D selbst, die angibt, wie stark sich zwei Verteilungen unterscheiden.

Die beiden Feldexperimente unterscheiden sich in den meisten Stellgrößen nur wenig, deutlich dagegen in den Klappenstellungen und der Last des Heißgaserzeugers. Der Produktionsbetrieb weicht von beiden vor allem in Aufgabemenge, Spanndruck und Mahltellergeschwindigkeit ab, da er nur einen Ausschnitt des Stellgrößenraums nutzt. Vor und nach DOE2 verschieben sich diese Größen ebenfalls deutlich, was die Verlagerung des Betriebs aus Abbildung 4.2b bestätigt. Neben diesem Covariate Shift verändert sich auch die Verteilung der spezifischen Energie selbst, sodass auf einen Prior Probability Shift geschlossen werden kann.

Damit verschieben sich sowohl die Eingangsgrößen gemäß (2.7) als auch die Ausgangsgrößen gemäß (2.8). Ein Actual Drift gemäß (2.11), der eine unveränderte Verteilung der Eingangsgrößen voraussetzt, lässt sich damit ausschließen. Offen bleibt dagegen, ob sich auch der Zusammenhang $P_t(y \mid X)$ zwischen Betriebspunkt und spezifischer Energie verändert. Da sich die Verteilung der Zielgröße über

$$P_t(y) = \int P_t(y \mid X)\, P_t(X)\, \mathrm{d}X \tag{4.1}$$

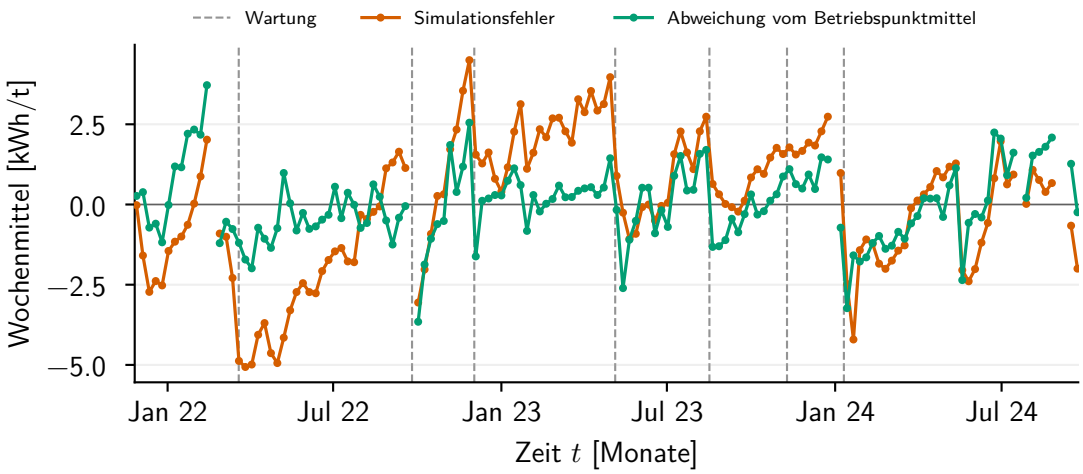


Abbildung 4.3: Wochenmittel von Driftsignal und Simulationsfehler

aus diesem Zusammenhang und der Verteilung der Eingangsgrößen ergibt, lässt sich ihre Verschiebung auf zwei Weisen erklären. Entweder folgt sie allein aus dem Covariate Shift, während der Zusammenhang unverändert bleibt, was einem Virtual Drift gemäß (2.12) entspricht. Oder der Zusammenhang verändert sich zusätzlich selbst, sodass ein Real Concept Drift gemäß (2.10) vorliegt. Unterscheiden lassen sich beide nur, wenn die Zielgröße bei gleichen Betriebspunkten über die Zeit verglichen wird.

Ein direkter Nachweis gelingt über Betriebspunkte, die im Betrachtungszeitraum wiederholt gefahren werden. Kehrt derselbe Betriebspunkt nach Wochen oder Monaten wieder und weicht die spezifische Energie dann von ihrem früheren Niveau ab, hat sich der Zusammenhang bei festgehaltenen Eingangsgrößen verändert. Gebildet werden die Betriebspunkte dafür über die Setpoints und nicht über die gemessenen Stellgrößen. Die Setpoints geben die Entscheidung des Operators wieder, während die Messwerte bereits die Antwort der Anlage und damit selbst vom Drift betroffen sind. Da exakt gleiche Kombinationen der Setpoints kaum wiederkehren, werden ähnliche Kombinationen per k-Means zu 40 Betriebspunkten zusammengefasst, innerhalb derer die Setpoints nur wenig streuen. Nahezu alle Punkte des Stroms entfallen dabei auf Betriebspunkte, die wiederholt gefahren werden.

Mittelt man die spezifische Energie eines Betriebspunkts über den gesamten Betrachtungszeitraum, erhält man sein typisches Energieniveau. Die Abweichung davon beantwortet für jeden Zeitpunkt die Frage, ob die Mühle bei denselben Setpoints mehr oder weniger Energie benötigt als üblich. Da der Einfluss des Betriebspunkts damit herausgerechnet ist und kein Modell eingeht, bleibt allein die Veränderung der Anlage. Diese Abweichung wird daher, obwohl es keine Ground Truth ist, im Folgenden als *"Driftsignal"* bezeichnet.

Abbildung 4.3 zeigt in Grün den Verlauf des Driftsignals als Wochenmittel über die wiederholt gefahrenen Betriebspunkte, zentriert um sein Gesamtmittel. Obwohl die Setpoints ähnlicher Betriebspunkte nahezu gleich bleiben, schwankt die spezifische Energie über die Zeit um mehrere kWh/t. Das Driftsignal steigt zwischen den Wartungen tendenziell an und fällt an den

meisten von ihnen sprunghaft ab. Die Anlage verändert damit ihr Verhalten bei unveränderten Setpoints.

Zum Vergleich ist in Orange der Simulationsfehler $e_t = y - f(X)$ auf denselben Betriebspunkten eingetragen, ebenfalls um sein Gesamtmittel zentriert. Anders als das Driftsignal vergleicht er die Messung mit der Vorhersage des Modells und nicht mit dem Mittel des Betriebspunkts. Zum einen beinhaltet er dadurch wie das Driftsignal die Veränderung der Anlage. Darüber hinaus enthält er aber einen Fehleranteil, der von der Lage des jeweiligen Betriebspunkts im Stellgrößenraum und damit von dessen Abdeckung durch die Trainingsdaten abhängt. Getrennt werden beide Komponenten durch den Zustand der Anlage zum Zeitpunkt t_0 der Trainingsdatenerfassung, den das Modell abzubilden versucht. Formal lässt sich die spezifische Energie dazu zu jedem Zeitpunkt t mit $y = \mathrm{E}_t[y \mid X] + \varepsilon$ als Erwartungswert bei gegebenen Eingangsgrößen und einer zufälligen Abweichung ε darstellen. Mit $\mathrm{E}_{t_0}[y \mid X]$ als Zustand der Anlage zum Zeitpunkt t_0 setzt sich die Abweichung zwischen Messung und Modell aus der Veränderung der Anlage seit t_0 und der Abweichung des Modells von diesem Zustand zusammen. Für den Simulationsfehler gilt damit

$$e_t = \underbrace{\mathrm{E}_t[y \mid X] - \mathrm{E}_{t_0}[y \mid X]}_{\text{Driftanteil}} + \underbrace{\mathrm{E}_{t_0}[y \mid X] - f(X)}_{\text{betriebspunktabhängiger Anteil}} + \varepsilon. \tag{4.2}$$

Der Driftanteil ist nur bei einem Real Concept Drift von null verschieden. Der betriebspunktabhängige Anteil hingegen tritt im Verlauf erst dadurch hervor, dass sich die Fahrweise verschiebt, also durch den Covariate Shift.

Bestätigt wird die Veränderung der Anlage durch das Verhalten des Driftsignals an den Wartungen. Dort verschiebt sich sein Mittel in einem Fenster von 7 Tagen vor und nach dem Ereignis deutlich stärker als an zufällig gewählten Tagen ($2{,}64$ gegenüber $0{,}76$ kWh/t, $p = 0{,}0010$). Zwischen zwei Wartungen nimmt das Driftsignal zudem tendenziell mit der seitdem vermahlenen Menge zu ($\rho = 0{,}31$). Die Mühle benötigt bei gleichen Setpoints also umso mehr Energie, je mehr Material seit der letzten Wartung vermahlen wurde. Beide Befunde bestätigen, dass sich der Zusammenhang $P_t(y \mid X)$ bei festgehaltenen Setpoints verändert, und belegen damit einen Real Concept Drift gemäß (2.10), der sich dem Covariate Shift überlagert.

4.4.2 Ursachen

Als Ursache des Drifts kommen damit vor allem die Wartungen in Betracht. Abbildung 4.4 zeigt dazu die gemessene Höhe der vier Walzen zusammen mit den Wartungen als gestrichelte Linien. Zwischen zwei Wartungen nimmt die Walzenhöhe allmählich ab, was auf den Verschleiß zurückzuführen ist. Mit jeder Wartung springt sie durch das Zurücksetzen des Nullpunkts auf ein neues Niveau. Blass eingetragen sind die übrigen Kalibrierungen, an denen die Walzenhöhe ebenfalls springt, ohne dass eine Wartung vorausging. Wie stark die Anlage auf eine Wartung reagiert, zeigt erst das Driftsignal in Abbildung 4.3, das an den meisten Wartungen deutlich

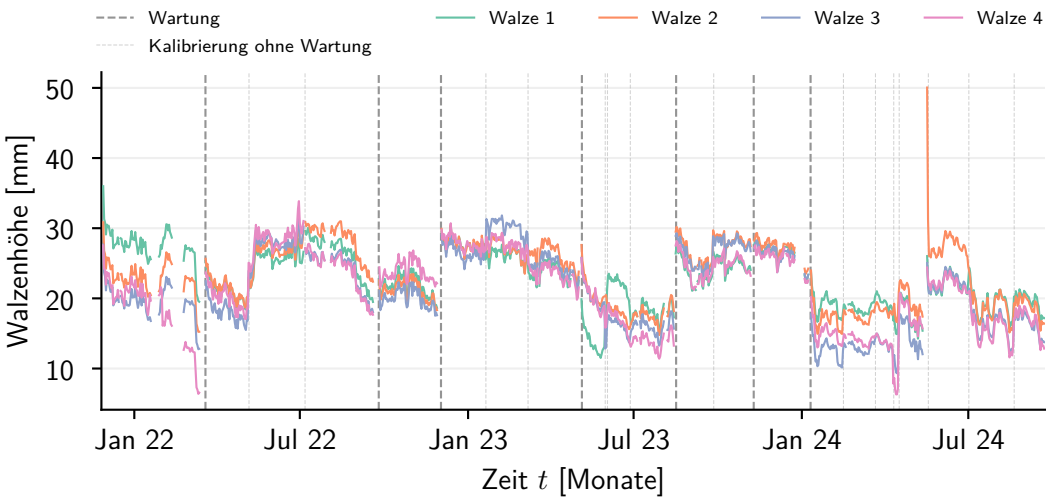


Abbildung 4.4: Gemessene Höhe der vier Walzen mit den Wartungen

abfällt.

Denkbar wäre auch, dass sich nach einer Wartung weniger die Anlage als die Fahrweise verändert, etwa indem die Anlage danach unruhiger betrieben wird. Ob dies der Fall ist, lässt sich in Anlehnung an [Ig24] über die Streuung der Betriebspunkte prüfen. Mit demselben Vergleich gegen zufällig gewählte Tage lässt sich eine Veränderung dieser Streuung an den Wartungen jedoch nicht nachweisen ($p = 0{,}200$). Die Operatoren fahren die Anlage nach einer Wartung also nicht anders als zuvor, und die Veränderung geht auf die Anlage selbst zurück. Auch der Simulationsfehler verschiebt sich an den Wartungen deutlich stärker als an zufällig gewählten Tagen ($2{,}95$ gegenüber $0{,}70$ kWh/t, $p = 0{,}0005$). Die Wartungen schlagen sich damit trotz des betriebspunktabhängigen Anteils auch im Fehler des Modells nieder. Die übrigen Kalibrierungen gehen dagegen mit keiner Veränderung der Anlage einher. An ihnen verschiebt sich das Driftsignal nicht stärker als an zufällig gewählten Tagen ($0{,}64$ kWh/t, $p = 0{,}678$). Auch der dokumentierte Wechsel des Aufgabematerials hinterlässt keine erkennbare Veränderung.

Neben den Wartungen stehen weitere Kontextgrößen zur Verfügung, deren Zusammenhang mit dem Simulationsfehler Tabelle 4.3 als Korrelationskoeffizient r nach Pearson angibt. Mit der seit der letzten Wartung vermahlenen Menge, der Walzenhöhe, der Materialfeuchte und der Zahl der Stillstände hängt der Fehler nur schwach zusammen. Dass die vermahlene Menge dennoch mit dem Drift verknüpft ist, zeigt ihr deutlich stärkerer Zusammenhang mit dem Driftsignal in Abschnitt 4.4.1. Im Simulationsfehler wird er durch dessen betriebspunktabhängigen Anteil überdeckt. Stärker hängt der Fehler mit Wassereindüsung und Walzentemperatur zusammen, ohne dass sich daraus eine Ursache des Drifts ableiten ließe.

Am stärksten hängt der Simulationsfehler mit der Ventilatorgeschwindigkeit zusammen, die selbst als Eingangsgröße in das Modell eingeht. Denkbar wäre, dass die Operatoren den Ventilator mit zunehmendem Verschleiß anders fahren und er den Drift damit lediglich widerspiegelt. Mit der seit der Wartung vermahlenen Menge und dem Driftsignal hängt die Ventilatorgeschwin-

Tabelle 4.3: Korrelation der Kontextgrößen mit dem Simulationsfehler

Kontextgröße	r
Menge seit Wartung	$0{,}18$
Walzenhöhe	$0{,}20$
Materialfeuchte	$0{,}26$
Stillstände je Woche	$0{,}11$
Wassereindüsung	$0{,}37$
Walzentemperatur	$-0{,}45$
Ventilatorgeschwindigkeit	$0{,}74$

digkeit jedoch kaum zusammen, und auch an den Wartungen verschiebt sie sich nicht stärker als an zufällig gewählten Tagen. Naheliegender ist daher, dass das Modell den Einfluss des Ventilators nicht vollständig abbildet. Dafür spricht, dass der Zusammenhang auch auf beiden Feldexperimenten besteht, in denen der Ventilator gezielt und unabhängig vom Verschleiß variiert wurde. Ursache ist wahrscheinlich die geringe Abdeckung durch das erste Feldexperiment, das nur wenige Stufen der Ventilatorgeschwindigkeit umfasst. Der Zusammenhang gehört damit zum betriebspunktabhängigen Anteil des Simulationsfehlers aus Abschnitt 4.4.1 und nicht zu den Ursachen des Drifts. Keine der gemessenen Kontextgrößen erklärt den Drift somit für sich allein.

4.4.3 Beschaffenheit

Ob der Drift alle Betriebspunkte gleichermaßen betrifft, soll zunächst anhand des Simulationsfehlers untersucht werden. In Abbildung 4.5 ist dazu der normierte Simulationsfehler nach

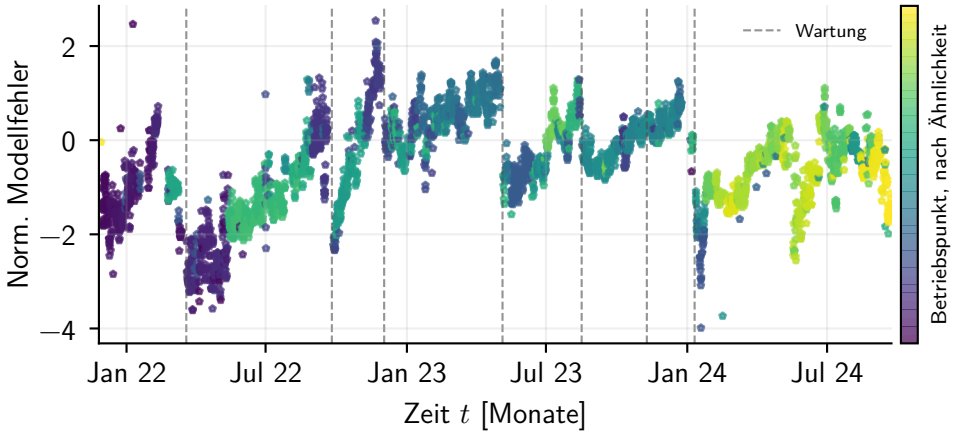


Abbildung 4.5: Normierter Simulationsfehler nach Betriebspunkt

Betriebspunkten eingefärbt, die aus einem k-Means-Clustering der Setpoints hervorgehen und deren Farben ihre Ähnlichkeit widerspiegeln. Für die Farbgebung werden die Betriebspunkte anhand ihrer mittleren Setpoints hierarchisch gruppiert und so angeordnet, dass ähnliche Betriebspunkte auf der Farbskala nebeneinander liegen. Auffällig ist, dass ähnliche Betriebspunkte über längere Phasen gefahren werden, während sich die Fahrweise über den gesamten Betrachtungszeitraum mehrfach verschiebt. Darin zeigt sich der Covariate Shift aus Abschnitt 4.4.1, der sich nicht stetig, sondern in Phasen vollzieht. Die Phasen könnten dabei auf einen Hidden Context zurückgehen. So ist denkbar, dass etwa der Zustand der Anlage oder das Aufgabematerial den Operatoren wenig Spielraum gelassen haben, die Anlage anders zu fahren. Betriebspunkte ähnlicher Farbe liegen dabei auch auf unterschiedlichen Fehlerniveaus. So liegen die grünen Betriebspunkte Mitte 2022 deutlich unter dem Niveau, das sie Mitte 2023 erreichen. Bei ähnlichen Setpoints verändert sich also der Zusammenhang zur spezifischen Energie, was den in Abschnitt 4.4.1 nachgewiesenen Real Concept Drift auch im Simulationsfehler sichtbar macht. Wie groß die Anteile von Betriebspunkt und Zeitpunkt jeweils sind, lässt sich in der Abbildung allein jedoch nicht bestimmen, da beide über die Phasen eng miteinander verknüpft sind.

4.5 Drift-Erkennung

4.6 Drift-Adaption

5 Kapitel 5

Anhang

A Tennessee-Eastman-Prozess

Tabelle A.1 listet die im Tennessee-Eastman-Prozess vordefinierten Prozessstörungen auf. Die Störungen IDV(1)–IDV(20) entstammen der ursprünglichen Prozessbeschreibung von [DV93] und umfassen Sprünge, zufällige Schwankungen, eine langsame Drift sowie klemmende Ventile; IDV(16)–IDV(20) sind dabei nicht näher spezifiziert. Die Erweiterung um IDV(21)–IDV(28) nach [BRJ15] ergänzt zusätzliche, als zufällige Schwankungen modellierte Temperatur- und Druckstörungen der Zulaufströme sowie der Kühlwasser-Umwälzeinheiten. Die Störung IDV(29) stellt die eigene Ergänzung dar. Sie modelliert nach Abschnitt 3.2.1 eine schleichende Kalibrierdrift des Strippertemperatursensors, die allein den aufgezeichneten Messwert verfälscht und den Prozess selbst unberührt lässt, und dient damit als Grundlage für einen Datenstrom mit driftbehafteten Zusammenhängen.

Tabelle A.2 führt die Setpoints auf, mit denen die sechs Betriebsmodi des TEP gefahren werden. Sie gehören zum dezentralen Regelkonzept nach [Ri96] und beschreiben den jeweils optimalen stationären Betriebspunkt nach [Ri95], übernommen in der Zusammenstellung von [RKR21]. Füllstände, Reaktordruck, Dampfventil und Rührerdrehzahl gehen als Schranken oder feste Vorgaben in die Optimierung ein und bleiben deshalb über alle Modi gleich, ebenso die Stellung des Recycle-Ventils außerhalb der Modi 3 und 6. Die drei letztgenannten Größen sind dabei keine Führungsgrößen, sondern Stellgrößen, die das Regelkonzept konstant hält. Dabei bezeichnet y_{AC} den Molanteil von A und C im Reaktorzulauf und y_{A} den darauf bezogenen Anteil von A.

Tabelle A.1: Prozessstörungen des Tennessee-Eastman-Prozesses nach [DV93], erweitert um die zusätzlichen Störungen nach [BRJ15].

Nummer	Prozessvariable	Typ
IDV(1)	A/C-Zulaufverhältnis, B-Zusammensetzung konstant (Strom 4)	Sprung
IDV(2)	B-Zusammensetzung, A/C-Verhältnis konstant (Strom 4)	Sprung
IDV(3)	D-Zulauftemperatur (Strom 2)	Sprung
IDV(4)	Eintrittstemperatur des Reaktorkühlwassers	Sprung
IDV(5)	Eintrittstemperatur des Kondensatorkühlwassers	Sprung
IDV(6)	Verlust des A-Zulaufs (Strom 1)	Sprung
IDV(7)	Druckverlust im C-Sammler – verringerte Verfügbarkeit (Strom 4)	Sprung
IDV(8)	Zulaufzusammensetzung von A, B, C (Strom 4)	Zufällige Schwankung
IDV(9)	D-Zulauftemperatur (Strom 2)	Zufällige Schwankung
IDV(10)	C-Zulauftemperatur (Strom 4)	Zufällige Schwankung
IDV(11)	Eintrittstemperatur des Reaktorkühlwassers	Zufällige Schwankung
IDV(12)	Eintrittstemperatur des Kondensatorkühlwassers	Zufällige Schwankung
IDV(13)	Reaktionskinetik	Langsame Drift
IDV(14)	Reaktorkühlwasserventil	Festklemmen
IDV(15)	Kondensatorkühlwasserventil	Festklemmen
IDV(16)	Unbekannt	Unbekannt
IDV(17)	Unbekannt	Unbekannt
IDV(18)	Unbekannt	Unbekannt
IDV(19)	Unbekannt	Unbekannt
IDV(20)	Unbekannt	Unbekannt
IDV(21)	A-Zulauftemperatur (Strom 1)	Zufällige Schwankung
IDV(22)	E-Zulauftemperatur (Strom 3)	Zufällige Schwankung
IDV(23)	A-Zulaufdruck (Strom 1)	Zufällige Schwankung
IDV(24)	D-Zulaufdruck (Strom 2)	Zufällige Schwankung
IDV(25)	E-Zulaufdruck (Strom 3)	Zufällige Schwankung
IDV(26)	A- und C-Zulaufdruck (Strom 4)	Zufällige Schwankung
IDV(27)	Druckschwankung in der Kühlwasser-Umwälzeinheit des Reaktors	Zufällige Schwankung
IDV(28)	Druckschwankung in der Kühlwasser-Umwälzeinheit des Kondensators	Zufällige Schwankung
IDV(29)	**Aufgezeichnete Strippertemperatur XMEAS(18)**	**Concept Drift**

Tabelle A.2: Setpoints der sechs Betriebsmodi des Tennessee-Eastman-Prozesses nach [Ri95; Ri96] in der Zusammenstellung von [RKR21].

		Modus 1	**Modus 2**	**Modus 3**	**Modus 4**	**Modus 5**	**Modus 6**
Produktionsrate	$\mathrm{m^3\,h^{-1}}$	22,89	22,73	18,04	36,04	23,55	20,20
Stripper-Füllstand	%	50	50	50	50	50	50
Separator-Füllstand	%	50	50	50	50	50	50
Reaktorfüllstand	%	65	65	65	65	65	65
Reaktordruck	kPa	2800	2800	2800	2800	2800	2800
Molanteil G im Produkt	mol-%	53,80	11,66	90,09	53,35	11,65	90,07
y_{A}	%	63,137	64,196	62,110	61,940	64,030	61,468
y_{AC}	mol-%	51	54,24	47,43	58,76	54,32	48,79
Reaktortemperatur	°C	122,9	124,2	121,9	128,2	124,6	123,0
Stellung Recycle-Ventil	%	1	1	77,62	1	1	71,166
Stellung Dampfventil	%	1	1	1	1	1	1
Rührerdrehzahl	%	100	100	100	100	100	100

B Vertikalrollenmühle

Tabelle B.1 fasst Aufbau und Training des in Abschnitt 4.3 eingesetzten statischen `MLP` zusammen. Das Netz bildet die Stellgrößen eines Betriebspunkts über zwei hidden Layer auf die spezifische Energie ab. Trainiert wird mit dem Adam-Optimierer auf den mittleren quadratischen Fehler. Stagniert der Fehler auf den Validierungsdaten, wird zunächst die Lernrate schrittweise reduziert und das Training schließlich vorzeitig beendet. Übernommen werden die Gewichte der Epoche mit dem geringsten Validierungsfehler. Die Trainingsdaten werden vorab je Betriebspunkt mit einem Butterworth-Filter geglättet und nach Betriebspunkten in Trainings- und Validierungsdaten getrennt.

Tabelle B.1: Aufbau und Training des statischen `MLP` der Mühle

Eingangsgrößen	Aufgabemenge, Mahltellergeschwindigkeit, Sichtergeschwindigkeit, Ventilatorgeschwindigkeit, Spanndruck, Last des Heißgaserzeugers, Stellung von Rezirkulations- und Kaminklappe
Zielgröße	Spezifische Energie
Verdeckte Schichten	2 mit je 8 Neuronen
Aktivierungsfunktion	$\tanh$, Ausgangsschicht linear
Trainierbare Parameter	153
Optimierer	Adam, Lernrate $1 \cdot 10^{-3}$
Verlustfunktion	Mittlerer quadratischer Fehler
Batchgröße	64
Epochen	höchstens 500
Early Stopping	nach 50 Epochen ohne Verbesserung
Lernratenreduktion	Faktor 0,5 nach 25 Epochen ohne Verbesserung, bis $1 \cdot 10^{-5}$
Trainingsdaten	27402 Abtastwerte aus 18 Betriebspunkten (DOE1)
Validierungsanteil	20 %, getrennt nach Betriebspunkten
Glättung	Butterworth-Filter 3. Ordnung, normierte Grenzfrequenz 0,05

Literatur

Monografien und Artikel

[Af15] Afkhami, B. et al.: Energy consumption assessment in a cement production plant. Sustainable Energy Technologies and Assessments 10, S. 84–89, 2015.

[Ba06] Baena-Garcıa, M. et al.: Early drift detection method. In: Fourth international workshop on knowledge discovery from data streams. Bd. 6, S. 77–86, 2006.

[BAK22] Bayram, F.; Ahmed, B. S.; Kassler, A.: From concept drift to model degradation: An overview on performance-aware drift detectors. Knowledge-Based Systems 245, S. 108632, 2022.

[BBL12] Bontempi, G.; Ben Taieb, S.; Le Borgne, Y.-A.: Machine learning strategies for time series forecasting. In: European big data management and analytics summer school. Springer, S. 62–77, 2012.

[BG07] Bifet, A.; Gavalda, R.: Learning from time-changing data with adaptive windowing. In: Proceedings of the 2007 SIAM international conference on data mining. SIAM, S. 443–448, 2007.

[BN06] Bishop, C. M.; Nasrabadi, N. M.: Pattern recognition and machine learning. Springer, 2006.

[BRJ15] Bathelt, A.; Ricker, N. L.; Jelali, M.: Revision of the Tennessee Eastman process model. IFAC-PapersOnLine 48 (8), S. 309–314, 2015.

[DR09] Dries, A.; Rückert, U.: Adaptive concept drift detection. Statistical Analysis and Data Mining: The ASA Data Science Journal 2 (5-6), S. 311–327, 2009.

[DV93] Downs, J. J.; Vogel, E. F.: A plant-wide industrial process control problem. Computers & chemical engineering 17 (3), S. 245–255, 1993.

[FP20] Forgione, M.; Piga, D.: Model structures and fitting criteria for system identification with neural networks. In: 2020 IEEE 14th International Conference on Application of Information and Communication Technologies (AICT). IEEE, S. 1–6, 2020.

[Ga04] Gama, J. et al.: Learning with drift detection. In: Brazilian symposium on artificial intelligence. Springer, S. 286–295, 2004.

[Ga14] Gama, J. et al.: A survey on concept drift adaptation. ACM computing surveys (CSUR) 46 (4), S. 1–37, 2014.

[GB10] Giri, F.; Bai, E.-W.: Block-oriented nonlinear system identification. Springer, 2010.

[Go16] Goodfellow, I. et al.: Deep learning. MIT press Cambridge, 2016.

[HPC12] Hoens, T. R.; Polikar, R.; Chawla, N. V.: Learning from streaming data with concept drift and imbalance: an overview. Progress in artificial intelligence 1 (1), S. 89–101, 2012.

[HS97] Hochreiter, S.; Schmidhuber, J.: Long short-term memory. Neural computation 9 (8), S. 1735–1780, 1997.

[In18] International Energy Agency: Technology Roadmap: Low-Carbon Transition in the Cement Industry, Techn. Ber., Published 5 April 2018, International Energy Agency (IEA), 2018, https://www.iea.org/reports/technology-roadmap-low-carbon-transition-in-the-cement-industry.

[Ka60] Kalman, R. E.: A new approach to linear filtering and prediction problems. 1960.

[KBG04] Kifer, D.; Ben-David, S.; Gehrke, J.: Detecting change in data streams. In: VLDB. Bd. 4, Toronto, Canada, S. 180–191, 2004.

[Ke11] Keesman, K. J.: System identification: an introduction. Springer Science & Business Media, 2011.

[Ki21] Kiranyaz, S. et al.: 1D convolutional neural networks and applications: A survey. Mechanical systems and signal processing 151, S. 107398, 2021.

[Le02] LeCun, Y. et al.: Gradient-based learning applied to document recognition. Proceedings of the IEEE 86 (11), S. 2278–2324, 2002.

[Le08] Lee, E. A.: Cyber physical systems: Design challenges. In: 2008 11th IEEE international symposium on object and component-oriented real-time distributed computing (ISORC). IEEE, S. 363–369, 2008.

[Lo11] Lopes, R. H. C.: Kolmogorov-Smirnov Test. In (Lovric, M., Hrsg.): International Encyclopedia of Statistical Science. Springer Berlin Heidelberg, Berlin, Heidelberg, S. 718–720, 2011, https://doi.org/10.1007/978-3-642-04898-2_326.

[Lu18] Lu, J. et al.: Learning under concept drift: A review. IEEE transactions on knowledge and data engineering 31 (12), S. 2346–2363, 2018.

[Ma25] Marijan, P. et al.: Concept Drift in Industrial Material Processing. In: 2025 11th International Conference on Control, Decision and Information Technologies (CoDIT). Bd. 1, IEEE, S. 647–652, 2025.

[Ma26] Marijan, P. et al.: Exploring model complexity: A model comparison. In: AIP Conference Proceedings. Bd. 3381 1. 1, AIP Publishing LLC, S. 020001, 2026.

[MBC00] McKay, M. D.; Beckman, R. J.; Conover, W. J.: A comparison of three methods for selecting values of input variables in the analysis of output from a computer code. Technometrics 42 (1), S. 55–61, 2000.

[Mo12] Moreno-Torres, J. G. et al.: A unifying view on dataset shift in classification. Pattern recognition 45 (1), S. 521–530, 2012.

[Ne20] Nelles, O.: Nonlinear System Identification: From Classical Approaches to Neural Networks, Fuzzy Models, and Gaussian Processes. Springer, Cham, 2020.

[OS10] Oppenheim, A. V.; Schafer, R. W.: Discrete-Time Signal Processing. Pearson, Upper Saddle River, NJ, 2010.

[Qu08] Quiñonero-Candela, J. et al.: Dataset shift in machine learning. Mit Press, 2008.

[RE22] Reinartz, C.; Enevoldsen, T. T.: pyTEP: A Python package for interactive simulations of the Tennessee Eastman process. SoftwareX 18, S. 101053, 2022.

[RHS20] Raab, C.; Heusinger, M.; Schleif, F.-M.: Reactive soft prototype computing for concept drift streams. Neurocomputing 416, S. 340–351, 2020.

[RHW86] Rumelhart, D. E.; Hinton, G. E.; Williams, R. J.: Learning representations by back-propagating errors. nature 323 (6088), S. 533–536, 1986.

[Ri95] Ricker, N. L.: Optimal steady-state operation of the Tennessee Eastman challenge process. Computers & Chemical Engineering 19 (9), S. 949–959, 1995.

[Ri96] Ricker, N. L.: Decentralized control of the Tennessee Eastman allenge process. Journal of Process Control 6 (4), S. 205–221, 1996.

[RKR21] Reinartz, C.; Kulahci, M.; Ravn, O.: An extended Tennessee Eastman simulation dataset for fault-detection and decision support systems. Computers & Chemical Engineering 149, S. 107281, 2021.

[SB21] Sousa, V.; Bogas, J. A.: Comparison of energy consumption and carbon emissions from clinker and recycled cement production. Journal of cleaner production 306, S. 127277, 2021.

[Sc01] Schaefer, H.: Loesche vertical roller mills for the comminution of ores and minerals. Minerals engineering 14 (10), S. 1155–1160, 2001.

[SK07] Scholz, M.; Klinkenberg, R.: Boosting classifiers for drifting concepts. Intelligent Data Analysis 11 (1), S. 3–28, 2007.

[SL19] Schoukens, J.; Ljung, L.: Nonlinear system identification: A user-oriented road map. IEEE Control Systems Magazine 39 (6), S. 28–99, 2019.

[Ti96] Tibshirani, R.: Regression shrinkage and selection via the lasso. Journal of the Royal Statistical Society Series B: Statistical Methodology 58 (1), S. 267–288, 1996.

[WB+95] Welch, G.; Bishop, G. et al.: An introduction to the Kalman filter. 1995.

[We16] Webb, G. I. et al.: Characterizing concept drift. Data Mining and Knowledge Discovery 30 (4), S. 964–994, 2016.

[Wi58] Wiener, N.: Nonlinear Problems in Random Theory. MIT Press, Cambridge, MA, 1958.

[WK96] Widmer, G.; Kubat, M.: Learning in the presence of concept drift and hidden contexts. Machine learning 23 (1), S. 69–101, 1996.

[Wo17] Woywadt, C.: Grinding process optimization—Featuring case studies and operating results of the modular vertical roller mill. In: 2017 IEEE-IAS/PCA Cement Industry Technical Conference. IEEE, S. 1–7, 2017.

Betreute studentische Arbeiten

[Ig24] Igel, S.: Concept Drift Mitigation for Deep Neural Networks related to Industrial Grinding Processes, Nr. 2024-32, Magisterarb., Kaiserslautern: Rheinland-Pfälzische Technische Universität Kaiserslautern-Landau, Lehrstuhl für Werkzeugmaschinen und Steuerungen, 2024.

Webseiten und Internetreferenzen

[Ko26] Kompetenzzentrum Klimaschutz in energieintensiven Industrien (KEI): Zementindustrie - Auf dem Weg zur klimaneutralen Industrie, Abgerufen am 2. März 2026, 2026, https://www.klimaschutz-industrie.de/themen/branchen/zementindustrie/.

[Ri05] Ricker, N. L.: Tennessee Eastman Challenge archive, Abgerufen am 19. März 2026, 2005, http://depts.washington.edu/control/LARRY/TE/download.html.

Lebenslauf

Persönliche Daten

Name: ———

Anschrift: ———

———

Geburtsdatum: ———

Geburtsort: ———

Staatsangehörigkeit: ———

Ausbildung

yyyy - yyyy Technische Universität ———

Studiengang: ———

Vertiefung: ———

yyyy - yyyy ———

———

Berufserfahrung

seit yyyy Deutsches Forschungszentrum für Künstliche Intelligenz GmbH -

FB Innovative Fabriksysteme /

Technologie-Initiative SmartFactory KL e.V., Kaiserslautern

Researcher / Teamleiter

yyyy - yyyy Technologie-Initiative SmartFactory KL e.V., Kaiserslautern

Researcher

yyyy - yyyy ———

———

———

yyyy - yyyy ———

———